Karlsruhe University of Applied Sciences

Department of Computer Science and Business Information Systems

Business Information Systems

BACHELOR´S THESIS

Efficiency vs. Performance in Green AI: An Empirical Investigation of Energy Consumption, Carbon Footprint, and Predictive Accuracy of Classification Algorithms at Varying Dataset Sizes.

Author	Eron Qorri
Matriculation number	85383
Workplace	Hochschule Karlsruhe
First Advisor	Prof. Dr. rer. nat. Christine Preisach
Second Advisor	Prof. Dr. Reimar Hofmann
Closing Date	31 July, 2026

31 July, 2026

Chairman of the Examination Board

Contents

List of Abbreviations

ACC Accuracy.

ADAM Adaptive Moment Estimation.

AI Artificial Intelligence.

AMP Automatic Mixed Precision.

API Application Programming Interface.

BERT Bidirectional Encoder Representations from Transformers.

BO Bayesian Optimization.

CF1 Carbon F1.

CO_2eq CO_2-equivalent.

CPU Central Processing Unit.

CUDA Compute Unified Device Architecture.

CV Cross-Validation.

DL Deep Learning.

DRAM Dynamic Random-Access Memory.

EPM External Power Meter.

FLOP Floating-Point Operation.

FN False Negative.

FP False Positive.

FP32 32-bit single-precision floating-point format.

GBDT Gradient Boosted Decision Trees.

gCO_2eq grams of CO_2-equivalent.

GP Gaussian Process.

GPU Graphics Processing Unit.

HPO Hyperparameter Optimization.

JIT Just-In-Time.

.

KNN K-Nearest-Neighbor.

kWh kilowatt-hour.

LOOCV Leave-One-Out Cross-Validation.

LR Logistic Regression.

ML Machine Learning.

MLP Multilayer Perceptron.

NLP Natural Language Processing.

NN Neural Network.

NVML NVIDIA Management Library.

OOB Out-of-Bag.

OvR One-vs-Rest.

PMC Performance Monitoring Counter.

RAM Random-Access Memory.

RAPL Running Average Power Limit.

ReLU Rectified Linear Unit.

RF Random Forest.

SAG Stochastic Average Gradient.

SGD Stochastic Gradient Descent.

SIMT Single Instruction, Multiple Threads.

TDP Thermal Design Power.

TN True Negative.

TP True Positive.

TPE Tree-structured Parzen Estimator.

WF1 Weighted F1-Score.

XGB Extreme Gradient Boosting.

List of Figures

List of Tables

Abstract

Green AI research has established that the energy consumption and emissions of machine learning are measurable, but its empirical scope has remained tied to large-scale deep learning on text and image data. The classical models that dominate deployed tabular classification lack a comparable baseline. This thesis addresses that gap through a systematic benchmark of five classifiers, Logistic Regression, Random Forest, XGBoost on CPU and GPU, and a neural network, on three tabular datasets spanning four orders of magnitude in size, from 178 to 11 million instances. The scope extends beyond the single training run to cover hyperparameter search, temporal variation in electricity grid carbon intensity and the inference break-even point at which cumulative deployment cost overtakes the training carbon footprint. To address the systematic undercounting of software-based estimators, all CPU energy figures are derived from direct hardware-sensor readings via CodeCarbon. A composite metric, Carbon F1, is introduced to express CO_2-equivalent emissions per unit of predictive performance.

The results show that ecological efficiency is not a property of any single model but emerges from the fit between its computational structure, the hardware it runs on and the volume of data it processes. No model leads across all three datasets. GPU acceleration becomes the more efficient choice for XGBoost only above approximately 80,000 instances. Below that threshold device initialization overhead dominates and emissions exceed those of the CPU variant. A constrained Optuna search for the MLP on HIGGS emitted roughly 54 times the carbon emissions of the final training run it enabled. Further analysis reveals that direct hardware measurement exceeded raw CodeCarbon figures by a median factor of 1.92, and shifting training away from winter evenings towards summer midday reduced absolute emissions by up to 70 percent without altering a single line of code. The contribution of this work is not a ranked list of preferred classifiers but a corrected measurement methodology, a context-sensitive efficiency metric and a lifecycle accounting framework that together make ecological model selection answerable for a given deployment context rather than as a general question.

1 Introduction

1.1 Background and Motivation

Training a single large transformer architecture can emit as much CO_2 as five cars over their entire lifespans [1]. That figure helped catalyze the Green Artificial Intelligence (AI) movement, a growing body of research that argues computational efficiency must be treated as a primary evaluation criterion alongside predictive performance [2]. The studies that established this case share a common focus: large-scale Deep Learning (DL) applied to text and images. Yet the majority of deployed Machine Learning (ML) does not look like this. Insurance companies predict churn, banks assess credit risk, particle physicists classify collision events: these are structured, row-column datasets processed by Logistic Regression (LR), gradient-boosted trees and Multilayer Perceptron (MLP). Tabular classification is the dominant workload in deployed machine learning [3, 4]. Its ecological footprint has not been systematically measured [1].

This omission has a concrete practical consequence. A practitioner who wants to make ecologically informed model choices for a tabular task lacks a comprehensive empirical basis to draw on. Whether Extreme Gradient Boosting (XGB) is more efficient than Random Forest (RF) on a 30,000-row dataset, at what dataset size Graphics Processing Unit (GPU) acceleration becomes the greener hardware choice for gradient boosting, or how the cumulative inference cost of a deployed model compares to what was spent training it: these are quantifiable operational questions that remain unanswered.

1.2 Problem Statement

The gap is structural. The Green AI literature established that training energy is a measurable and consequential quantity, but empirically it has remained tethered to large-scale deep learning: landmark studies measure transformers trained on text and express costs in GPU-hours [1, 5]. The benchmarking literature that rigorously evaluates classical models on tabular data, meanwhile, treats energy as entirely outside its scope and compares models exclusively on predictive performance [3, 4]. A systematic study of how energy consumption and predictive performance jointly scale across model families and dataset sizes on tabular data does not exist.

Two further methodological problems widen this gap. First, the software tools commonly used to estimate training emissions have been shown to systematically undercount actual energy consumption [6, 7], meaning that reported results for any study that uses them should be treated as lower bounds rather than ground truth. Second, even an accurate

measurement of a training run captures only a fraction of a model's true ecological footprint. Hyperparameter search, which for complex models on large datasets can cost more than all benchmark training runs combined, is invisible in published tables [1]. The inference phase is invisible too, even though repeated prediction over a deployment lifetime has been shown to account for the majority of a production system's cumulative energy demand [8]. Consequently, the existing literature often provides an incomplete assessment of the true ecological footprint: the scope is largely restricted to an unrepresentative class of workloads, the tools used systematically undercount the energy they measure, and the training phase alone is treated as a proxy for the full lifecycle cost.

1.3 Research Objectives

This thesis addresses these gaps through an empirical benchmark that frames model selection as a multi-objective problem, jointly evaluating predictive performance and ecological efficiency across the full lifecycle of a model. The central research question is:

> *How does the trade-off between CO_2-equivalent emissions and predictive performance scale across LR, RF, XGB, and MLP when tabular dataset size varies from thousands to millions of instances?*

Five sub-questions structure the investigation:

RQ1. Under what conditions do the emissions of a more complex model exceed the performance gain it delivers over a simpler baseline?

RQ2. How do emissions scale with increasing MLP depth and width?

RQ3. How do emissions differ between Central Processing Unit (CPU)-based and GPU-based training for equivalent tasks, and at what dataset size does GPU acceleration become ecologically justified?

RQ4. How does reducing the number of training instances on a large dataset affect model performance and emissions across different model families?

RQ5. At what prediction volume does the cumulative inference CO_2-equivalent (CO_2eq) footprint of a deployed model surpass its one-time training footprint, and how does this break-even point differ across models?

To answer these questions, the study benchmarks five classifiers on three tabular datasets spanning four orders of magnitude in size: the Wine quality [9], Default of Credit Card Clients [10] and HIGGS boson detection [11] datasets. The mentioned models were selected to represent the full spectrum of tabular classification, from a linear baseline (LR) through tree ensembles (RF, XGB) to a feedforward neural network (MLP); XGB is evaluated on both CPU and GPU hardware. A corrected energy measurement methodology addresses the systematic undercounting inherent in standard software-based estimation tools. Since CO_2eq emissions are derived from measured energy consumption, these adjusted figures

propagate into all reported emission values throughout. To support cross-model efficiency comparisons that neither raw emissions nor predictive performance alone can capture, the study introduces a custom composite metric, the Carbon F1 (CF1), which expresses CO_2eq emissions per unit of predictive performance and is defined formally in Section 4.4. A counterfactual analysis using one year of hourly electricity grid data further quantifies how much the German electricity grid's diurnal and seasonal variability affects the absolute emissions of the training runs conducted in this study.

1.4 Structure of the Thesis

The remainder of this thesis is organized as follows. Chapter 2 establishes the theoretical foundations: the Green AI paradigm, energy and carbon measurement in ML, the classification methods under study and the statistical evaluation methods. Chapter 3 surveys the empirical literature on ML energy benchmarking and identifies the structural gap between Green AI research and classical tabular classification benchmarks that this work addresses. Chapter 4 defines the datasets, models, evaluation metrics including CF1, validation strategy and hyperparameter tuning protocol, and describes the hardware and software environment, the emissions measurement infrastructure and each of the five experiments conducted. Chapter 5 presents the empirical findings across all five experimental dimensions: measurement validation, tuning overhead, the cross-dataset benchmark, hardware and scaling effects, and the lifecycle break-even analysis. Chapter 6 synthesizes the findings, acknowledges the study's limitations and provides actionable implications for Green AI practitioners alongside directions for future research.

2

Theoretical Background

This chapter establishes the theoretical foundations required to understand the empirical study presented in the remainder of this thesis. It first frames the research within the broader debate between *Red AI* and *Green AI*, then formalizes the concepts of energy consumption and carbon emissions in ML. Subsequently, the relevant classification models are described from a purely theoretical standpoint, followed by the metrics used to evaluate their predictive performance, the validation strategy and Hyperparameter Optimizations (HPOs) techniques.

2.1 Green AI and Red AI

In recent years, a paradigm has emerged in AI research that Schwartz et al. [2] refer to as "Red AI". Red AI describes research approaches that primarily aim to improve prediction performance through ever-larger datasets and parameter counts that demand massive computational resources, while largely disregarding the associated infrastructural and energetic costs. What drives this trend is that the relationship between model performance and model complexity is at best logarithmic [2]: a linear increase in performance requires an exponentially larger model or exponentially more training data. Empirically, as reported by Verdecchia et al. [12], the compute used to train state-of-the-art models has been doubling approximately every 3.4 months since 2012. This is particularly evident in large Transformer architectures. Strubell et al. [1] document that training such models can require days of continuous computation across multiple GPUs, while foundation models such as GPT-3 scale to 175 billion parameters.

In response, the concept of *Green AI* has emerged as a counter-movement within the ML community [2]. Green AI is defined as research that yields novel results while explicitly accounting for computational costs, elevating efficiency to a primary evaluation criterion alongside predictive performance. Beyond the environmental dimension, the movement also addresses inclusivity: the prohibitive financial costs of training massive models increasingly restrict participation to only some well-resourced institutions [2]. A distinction must also be made between Green AI and *AI for Green*, where Green AI concerns reducing the ecological footprint of AI systems themselves, not applying AI to environmental problems [13].

A core principle of the Green AI movement is evaluating environmental impacts across the entire lifecycle of an AI system, rather than focusing solely on the training phase [14, 15]. This full-lifecycle view encompasses data engineering, system integration, continuous retraining and inference operations [15]. Empirical work on data-centric Green AI [12] illustrates the potential of this broader perspective: simple dataset preprocessing modifications, such as extracting smaller representative subsets or reducing feature counts, can

decrease training energy consumption by up to 92 percent with negligible loss in predictive performance.

To translate these principles into practice, researchers advocate for standardized reporting of computational work, energy consumption and carbon emissions alongside performance results [2, 16]. Several tools have been developed to support this. The `ML CO2 Impact Calculator` [17] and the `experiment-impact-tracker` [16] allow practitioners to estimate carbon emissions based on hardware configuration, execution time and local electricity grid carbon intensity. At the organizational level, the Green Software Foundation's Software Carbon Intensity (SCI) specification [18] provides a standardized, consequential carbon accounting metric aimed at directly reducing AI carbon footprints rather than relying on market-based offsets [5, 15].

Realizing efficiency as a measurable criterion requires defining appropriate metrics. Floating-Point Operations (FLOPs) provide a hardware-independent estimate of computational work [2], but do not translate directly to energy or execution time, as these depend on hardware architecture, memory access patterns and parallelization [16]. Training time is more intuitive but hardware-dependent and difficult to compare across environments. Energy consumption in kilowatt-hours (kilowatt-hour (kWh)) and greenhouse gas emissions in CO_2eq sidestep these limitations by capturing the actual physical and ecological footprint of a training run. Although tied to a specific hardware configuration and electricity grid, they provide directly comparable sustainability figures when experiments are conducted under controlled conditions [5, 16].

2.2 Energy and Carbon Emissions in Machine Learning

Operationalizing ecological efficiency requires distinguishing two related but distinct quantities: energy consumption and CO_2eq emissions. This section formalizes both, along with the hardware components that drive energy draw and the measurement approaches used to obtain reliable figures.

2.2.1 From Energy to Carbon Emissions

Energy consumption. To understand the ecological footprint of ML, it is first necessary to distinguish between power and *energy consumption*. Power, measured in Watts (W) or Joules per second ($\frac{J}{s}$), represents the instantaneous rate at which electricity is drawn by a system. Energy consumption, on the other hand, is the total quantity of electricity consumed over a specific duration, calculated as the product of power and time ($E = P \cdot t$, in watt-seconds, i.e. joules) and often expressed in multiples of the watt-hour, most commonly at the kilowatt-hour (kWh) magnitude. Since 1 Wh equals 1 W sustained for one hour, or equivalently 3,600 J, the kilowatt-hour is the practical unit for training energy rather than the SI unit Joule. Unlike nominal power ratings, the actual power draw depends on hardware utilization and thermal design rather than being a fixed property of the components [16].

Carbon intensity. Translating this measured energy consumption into a quantifiable environmental impact requires the concept of carbon intensity [17]. Carbon intensity represents the

amount of greenhouse gas emissions produced per unit of electricity generated, expressed in grams of CO_2-equivalent (gCO_2eq) per kWh ($\frac{gCO_2eq}{kWh}$) and standardizes the global warming potential of various greenhouse gases into a single comparable value.

The operational carbon footprint of a ML model is derived by multiplying the total energy consumed (E, in kWh) by the carbon intensity I_C (in $\frac{gCO_2eq}{kWh}$), yielding $C = E \cdot I_C$ (in gCO_2eq) [5]. Because different geographic regions rely on vastly different fuel mixes, from coal and natural gas to nuclear and renewables, carbon intensity varies drastically by location [16]. Empirical data illustrates this range clearly: training a model in Quebec, Canada, which is heavily reliant on hydroelectricity, faces a carbon intensity of approximately 20 $\frac{gCO_2eq}{kWh}$, whereas training the same model in a fossil-fuel-dependent region such as Iowa, USA, results in a carbon intensity of over 736 $\frac{gCO_2eq}{kWh}$ [17].

Carbon intensity is not merely a static geographic property but also a temporally variable one [5]. The emissions factor of a grid can fluctuate significantly throughout the day based on the real-time availability of renewable sources, such as solar during daylight hours or wind at night. Consequently, accurately evaluating the ecological footprint of a ML model requires integrating both the spatial location and the temporal characteristics of the local electricity grid into the final gCO_2eq conversion.

2.2.2 Computing Hardware and Power Consumption

The execution of ML workloads relies on a combination of hardware components, primarily the CPU, GPU and Dynamic Random-Access Memory (DRAM) [16]. Understanding the architectural differences between these components is essential for reasoning about their respective energy profiles and the efficiency trade-offs examined in this thesis.

The CPU is a general-purpose processor designed for sequential, low-latency execution of complex, branching instruction streams. Its architecture dedicates a large share of its transistor budget to control logic, branch prediction and deep cache hierarchies, typically comprising between 4 and 32 high-frequency cores. This design excels at tasks requiring complex decision logic but offers limited parallelism for the arithmetic-intensive operations that dominate DL [19].

The GPU, by contrast, is a hardware accelerator originally designed for rendering graphics, a task that requires applying the same transformation to millions of pixels simultaneously. Its architecture instead consists of thousands of smaller, simpler processing cores organized into streaming multiprocessors, optimized for high arithmetic throughput and high memory bandwidth. This massively parallel design maps directly onto the matrix multiplications and element-wise operations that dominate both training and inference in deep learning, which is why GPUs have become the standard compute substrate for computationally intensive DL workloads [5].

DRAM serves as the system memory that holds the active dataset, model parameters, and intermediate computation buffers throughout a training run. Its per-module power draw is substantially lower than that of the CPU or GPU, typically ranging from a few watts to around 20 W depending on installed capacity and operating frequency [20]. Despite being

individually modest, this contribution is non-negligible over long training runs and must therefore be included in any complete energy accounting.

Power draw is the price of this architectural advantage. While CPUs typically draw between 10 W and 150 W under load, GPUs are significantly more power-hungry, often consuming between 250 W and 350 W [5]. As a result, the GPU typically accounts for the vast majority of total electrical energy consumed during training, often approaching three-quarters of the total draw [5].

Thermal Design Power (TDP) defines the maximum heat generated by a component that its cooling system is designed to dissipate, serving as a theoretical upper bound on sustained power draw [6]. Some simplified methodologies estimate total energy consumption by multiplying a component's TDP by execution time, but this approach is fundamentally flawed [16]: actual power draw fluctuates continuously based on hardware utilization, varying greatly between idle states and peak load. Empirical studies demonstrate that TDP-based estimation leads to severe over- or underestimations of the true energy footprint [16]. Reliable sustainability measurements therefore require direct real-time hardware readings rather than static TDP-based extrapolations.

2.2.3 Model Suitability for CPU vs. GPU

In the context of Green AI, optimizing energy efficiency requires aligning the computational profile of a ML model with the architectural strengths of the underlying hardware. While it is common practice to universally accelerate training using GPUs [2, 21], this approach is not intrinsically energy-efficient for all models. The fundamental divide in hardware suitability lies in the distinction between dense, regular parallelism and irregular, sequential branching [19].

Hardware platforms are designed with opposing optimization goals: GPUs are optimized for aggregate throughput on parallel tasks, whereas CPUs are optimized for low latency on sequential, single-threaded tasks [19]. GPUs achieve this through a *Single Instruction, Multiple Threads (SIMT)* architecture that executes thousands of lightweight threads concurrently to hide memory latency. This design heavily favors dense, regular computational tasks with uniform memory access patterns, most notably the large-scale matrix multiplications that form the computational core of Neural Network (NN) training. When operations can be uniformly distributed across thousands of cores simultaneously, the GPU achieves maximum utilization.

Not every model benefits from GPU acceleration. Models that rely on sequential decision logic, such as decision trees, process data step by step, adapting at each branch based on previous results. Running these on a GPU wastes energy: the hardware is built for parallel work, and sequential branching leaves most of its cores idle [19, 22]. For these models, a CPU is the more energy-efficient choice.

Despite the higher peak power draw of GPUs, their throughput on suitable workloads means they process substantially more training samples per joule than a CPU running the same task. A GPU can therefore deliver superior energy efficiency for well-matched

workloads despite consuming more power in absolute terms. This advantage is not universal: at small dataset scales, the GPU's higher idle and transfer overhead can make CPU training the more energy-efficient option overall [6, 21].

2.2.4 Measurement Approaches

Physical measurement. Accurately evaluating the energy consumption of ML models relies on a spectrum of methodologies, broadly categorized into physical measurements, estimation models and on-chip sensors [6]. The baseline for ground-truth measurement is the External Power Meter (EPM), which tracks power directly at the wall outlet or motherboard. While EPMs capture the true consumption of the entire system, they are often costly to deploy at scale and cannot provide fine-grained decomposition to isolate the energy used by specific software processes or individual hardware components [6].

Estimation models. To achieve component-level granularity without external hardware, many approaches utilize estimation models. These models calculate energy based on abstract hardware specifications, such as multiplying execution time by the TDP or counting FLOPs, or by leveraging Performance Monitoring Counters (PMCs) to track utilization rates [6]. However, as established in Section 2.2.1, relying solely on abstract analytical models or TDP often leads to severe inaccuracies under dynamic workloads, failing to capture the true, fluctuating real-time power draw of the hardware [16].

On-chip sensors. Consequently, modern sustainability accounting strongly prefers built-in measurements obtained from vendor-specific on-chip sensors [16]. For GPUs, the industry standard is the NVIDIA Management Library (NVML), which polls instantaneous power draw in watts during execution, from which total energy consumption is obtained by integrating the power readings over the measurement interval. For CPUs and DRAM, the standard interface is Running Average Power Limit (RAPL), provided by Intel and AMD, which accesses hardware performance counters to deliver accurate, fine-grained energy measurements directly from the CPU package [6].

Practical constraints. Despite its high accuracy, a significant practical limitation of RAPL is its strict dependency on the underlying operating system [16]. RAPL interfaces are primarily supported on Linux environments and often require elevated administrator privileges to access the hardware counters securely. When direct sensor access is unavailable due to these constraints, software tracking tools often revert to problematic TDP-based estimation models as a fallback, which severely compromises measurement accuracy [6].

2.2.5 Software Measurement

From fragmented tools to a standard. To practically implement the sustainability accounting principles advocated by the Green AI movement, researchers require standardized tools that bridge the gap between physical hardware metrics and environmental impacts. Early efforts were fragmented [23]: web-based calculators such as the ML CO_2 Impact calculator [17] and Green Algorithms [24] relied on manually entered hardware specifications and broad estimates, while later Python packages such as `experiment-impact-tracker` [16] and

`CarbonTracker` [25] polled hardware sensors directly but diverged in methodology and carbon-intensity data, producing inconsistent measurements across the literature [23].

CodeCarbon. To unify these fragmented efforts into a reliable standard, the CodeCarbon [26] initiative was established [23]. CodeCarbon is an open-source software library that evolved directly from previous tracking projects, serving as the official successor to the `Energy Usage` tool [6] and integrating the efforts of the ML CO_2 Impact project [23]. By providing a unified tracking interface, it enables practitioners to quantify the environmental cost of their models and report these metrics transparently [26].

To evaluate a model's energy consumption, the framework leverages the direct measurement approaches introduced in Section 2.2.4. Specifically, it interfaces with vendor-specific on-chip sensors, such as NVML for GPUs and RAPL for Intel and AMD processors, to collect fine-grained power draw measurements directly from the underlying hardware [6]. By periodically polling these hardware counters during execution, the tool computes the total energy consumed by the computing components in kWh.

Once the physical energy consumption is recorded, the framework applies the $C = E \cdot I_C$ conversion from Section 2.2.1: it determines the carbon intensity of the local electricity grid from the geographic location of the computing infrastructure and multiplies the measured kWh figure by this value. The raw result follows directly from the carbon intensity unit defined earlier and is therefore in gCO_2eq, which CodeCarbon stores internally as $kgCO_2eq$ in its output files; this study converts those values to gCO_2eq for presentation, as the measured footprints range from sub-gram to tens of grams and are more legibly expressed at that magnitude [5]. By combining hardware-level energy tracking with grid-level carbon intensity data in a single standardized framework, it automates the translation from computational work to ecological impact [16].

2.3 Supervised Classification

In ML, supervised classification is the task of inferring a predictive model from historical, annotated data in order to make accurate categorical predictions on new data [27]. Mathematical notation in the following subsections is local to each model description; symbols such as $\mathcal{L}$, γ, λ, and T are standard in their respective literatures and may carry different meanings across subsections.

Supervised Learning. Formally, given a training dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$ consisting of n examples, where each $x_i \in \mathcal{X}$ represents a vector of input features and $y_i \in \mathcal{Y}$ represents the corresponding ground-truth label, the objective is to learn a mapping function $f : \mathcal{X} \to \mathcal{Y}$ that reliably predicts the target output for any given input [4].

Classification. The structure of the label space $\mathcal{Y}$ determines the nature of the classification task. In binary classification, $\mathcal{Y} = \{0, 1\}$. When more than two categories are involved it becomes a multiclass problem with $\mathcal{Y} = \{1, \ldots, M\}$ [4]. Multiclass problems are typically handled via One-vs-Rest (OvR) strategies, which decompose the problem into M binary classifiers, each distinguishing one class from all others, or by extending binary formulations

with a Softmax output, which maps a raw score vector $z \in \mathbb{R}^M$ to a normalized probability distribution via $\text{softmax}(z)_k = e^{z_k} / \sum_{j=1}^{M} e^{z_j}$, where z_k is the raw score for class k and j indexes all M classes in the denominator, ensuring all class probabilities sum to one.

The fundamental objective when learning f is achieving strong *generalization*: the model must perform reliably on new, unseen data drawn from the same underlying distribution rather than merely memorizing training inputs [28]. A central challenge is *overfitting*, where a model excessively adapts to the specific "sampling noise" present in the training data, fitting it so closely that predictive performance on new, unseen data actively degrades [28, 29]. The opposing failure mode is *underfitting*, where the model is too simple to capture the true data patterns. Training therefore requires finding the right balance between these two extremes.

To optimize f during training, a loss function $\mathcal{L}(y, f(x))$ quantifies the discrepancy between predicted outputs and true labels [29]. Minimizing $\mathcal{L}$ with respect to the model's parameters is performed by a *solver*, typically a gradient-based optimization algorithm such as gradient descent, which iteratively adjusts the parameters in the direction of the negative gradient $-\nabla_\theta \mathcal{L}$ to reduce the loss [29]. For classification, the standard objective is Cross-Entropy Loss. In the binary case this reduces to the negative Bernoulli log-likelihood, while for multiclass problems it generalizes to Categorical Cross-Entropy, formalized in Equation (2.1).

$$\mathcal{L} = -\sum_{k=1}^{M} y_k \log\big(p_k(x)\big) \tag{2.1}$$

y_k is the binary indicator for the true class k and $p_k(x)$ is the model's predicted probability for that class. Cross-Entropy produces steeper, more informative gradients when a confident prediction is wrong and provides a direct probabilistic interpretation of the model output.

2.3.1 Logistic Regression

LR, formalized as a statistical classification method by Cox [30], is a parametric linear model that remains a standard baseline for binary classification tasks where interpretability and computational efficiency are paramount.

Model. For a binary classification task ($\mathcal{Y} = \{0, 1\}$), LR predicts the probability that an instance x belongs to the positive class. To ensure the output is bounded strictly between 0 and 1, the linear combination of input features and learned weights w with bias b is passed through the logistic sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$, yielding the prediction model in Equation (2.2).

$$P(y = 1 \mid x) = \sigma(w^\top x + b) = \frac{1}{1 + e^{-(w^\top x + b)}} \tag{2.2}$$

This functional form stems from modeling the log-odds (logit) as a linear function of the inputs, as expressed in Equation (2.3) [27].

$$\ln\left(\frac{P(y = 1 \mid x)}{P(y = 0 \mid x)}\right) = w^\top x + b \tag{2.3}$$

Training. Parameters are estimated via Maximum Likelihood Estimation, which is equivalent to minimizing the Binary Cross-Entropy loss and requires iterative numerical solvers since no closed-form solution exists [27]. This study uses the Stochastic Average Gradient (SAG) solver [31], which is particularly well-suited for large datasets: rather than recomputing the full gradient over all training samples at each step, SAG maintains a running average of recently computed individual gradients, substantially reducing both memory usage and required training time. For multiclass problems, Multinomial LR with a Softmax function is applied [27].

Advantages and Limitations. A key advantage of LR is its interpretability: each feature weight directly represents its directional influence on the log-odds of the prediction. It also trains quickly and is robust to overfitting on simple datasets. Its fundamental limitation is linearity: without manual feature engineering, the model cannot capture non-linear patterns or feature interactions [27].

2.3.2 Random Forest

A decision tree classifies instances by routing them through a sequence of axis-aligned threshold tests ($x_j \leq t$) until a leaf node is reached, where the majority class of the training samples arriving at that leaf is assigned. Trees learn piecewise constant functions through a greedy algorithm that minimizes Gini impurity at each split, making them naturally suited to the irregular, non-smooth patterns common in tabular data [29]. When grown deeply, however, a single tree memorizes sampling noise rather than true structure, producing high variance and poor generalization. RFs, formally introduced by Leo Breiman [32], address this by combining a large collection of tree predictors, where each tree depends on the values of a random vector sampled independently and with the same distribution for all trees in the forest.

Training Procedure. The algorithm constructs this ensemble using a two-fold randomization strategy. First, it employs a technique known as *bagging* (bootstrap aggregating), originally proposed by Breiman [33], where each tree is grown on a new training set drawn at random with replacement from the original data. RFs extend this by additionally randomizing feature selection during the tree-growing phase: instead of evaluating all available features to determine the optimal split at each internal node, the algorithm selects the best split from a randomly chosen, restricted subset of features [32]. The constituent trees are typically grown to their maximum depth and are not pruned, meaning no branches are removed after growth to reduce model complexity. To classify a new input vector, each tree in the ensemble casts a unit vote and the forest outputs the majority class.

Advantages. Breiman theoretically proved that the predictive quality of a RF is governed by the interplay of two key parameters: the *strength* of the individual tree classifiers and the *correlation* between them [32]. Correlation here means the degree to which trees make similar predictions on the same data points. Decorrelated trees, by contrast, make diverse predictions and thus commit different kinds of errors. An optimal forest consists of individually strong trees that produce decorrelated predictions. By combining bagging

with random feature selection, the algorithm deliberately decreases the correlation between individual trees, which substantially reduces the variance of the overall ensemble without substantially compromising the predictive strength of the base learners. Building on this analysis, Breiman [32] further demonstrated via the *Strong Law of Large Numbers* that as the number of trees grows, the generalization error almost surely converges to a limiting value, guaranteeing that the ensemble is inherently robust against overfitting with respect to the number of trees. While this convergence argument in the original paper was somewhat informal, formal consistency for a close variant of Breiman's algorithm under standard assumptions was later established by Scornet, Biau and Vert [34].

A crucial computational advantage of this architecture, particularly relevant for evaluating the efficiency of ML models, is its inherent parallelizability [35]. Because the individual trees are entirely independent of one another, the training process can be distributed independently across multiple processing cores, allowing RFs to scale efficiently on multicore CPU architectures [32, 35]. GPU-accelerated implementations also exist (e.g., RAPIDS cuML, Intel Extension for Scikit-learn), though GPU support remains implementation-dependent rather than a universal property of the algorithm.[1] Additionally, the bagging mechanism naturally provides an internal validation mechanism: since each tree is trained on a bootstrap sample (approximately 63% of the data), the remaining samples, termed *Out-of-Bag (OOB)* samples, form the complement set. For each training sample, the forest can evaluate it using only the subset of trees that did not include it in their training bootstrap, yielding an unbiased generalization error estimate without requiring a separate validation split [32]. In scikit-learn, this mechanism is optional and requires setting the `oob_score=True` parameter at initialization.

Limitations. Despite this parallelism, RFs carry a significant computational burden on large datasets. Each tree must be grown independently on a bootstrap sample of the full training set and because the trees are deep and unpruned, the cost per tree grows with both the number of samples and the number of features evaluated at each split [35]. This cost is then multiplied across all B trees in the ensemble. As the dataset size grows into the millions of samples, the aggregate training cost becomes substantial, limiting the practical applicability of RFs on very large datasets without subsampling or other approximations.

2.3.3 Gradient Boosting and XGBoost

RFs and Gradient Boosting represent fundamentally different ensemble strategies. RFs combine many independent strong learners: deep, complex trees trained in parallel on bootstrap samples, where the ensemble strength comes from diversity and aggregation. In contrast, Gradient Boosted Decision Trees (GBDT) operate sequentially, constructing an ensemble of weak learners (shallow trees) where each successive tree is trained to correct the errors of the previous iterations, iteratively improving overall performance [29]. Introduced by Friedman [29], the gradient boosting machine frames this sequential training as a form of

[1] GPU acceleration for RFs is not universally available across all libraries. Support depends on the specific implementation: scikit-learn runs on CPUs only, while specialized libraries like RAPIDS cuML provide GPU-accelerated versions.

gradient descent in function space. Instead of adjusting numerical parameters, the algorithm iteratively adds a new decision tree to the ensemble that is fitted to the negative gradient of the loss function with respect to the predictions of the previous iteration. This allows the model to minimize arbitrary differentiable loss functions, making it a highly flexible and powerful approach for both regression and classification tasks. Throughout this framework, $\hat{y}_i$ denotes the model's predicted output for training instance i, where the hat symbol ($\hat{\cdot}$) is standard notation distinguishing a model's prediction from the corresponding true label y_i. After each boosting round, a new tree is added whose outputs correct the residual errors of the current ensemble, progressively updating $\hat{y}_i$ toward y_i. Figure 2.1 illustrates the architectural contrast between the two strategies: the left panel shows RF's parallel structure, in which all trees are trained independently on bootstrap samples and their predictions are aggregated by majority vote, while the right panel shows GBDT's sequential pipeline, in which each successive tree is fitted to the errors of the previous ensemble and the predictions are summed to progressively reduce the residual.

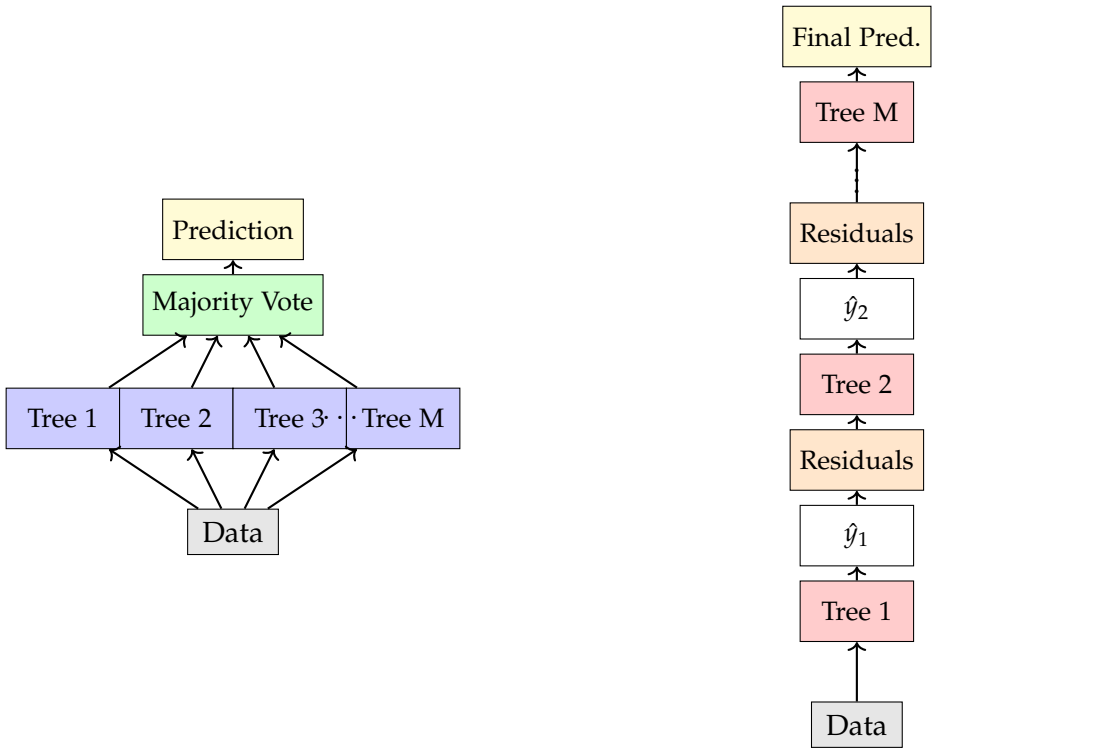


Figure 2.1: RF vs GBDT: Parallel (breadth) vs Sequential (depth) ensemble strategies.

Regularized Objective. XGB, introduced by Chen and Guestrin [36], significantly extends this foundational concept to improve both predictive performance and computational scalability. To combat the overfitting risks inherent in boosted ensembles (where each tree iteratively fits residual errors of its predecessors, risking the memorization of training noise), XGB modifies the training objective by augmenting the task-specific loss function with an explicit

regularization term [36].

$$\mathcal{L} = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \Omega(f), \quad \Omega(f) = \gamma T + \tfrac{1}{2}\lambda \|w\|^2 \tag{2.4}$$

In Equation (2.4), n is the number of training instances, y_i the true label and $\hat{y}_i$ the predicted value for instance i and $l(\cdot)$ is a differentiable task-specific loss function. For binary classification, for example, l takes the form of the logistic loss $l(y_i, \hat{y}_i) = y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)$. The regularization term $\Omega(f)$ penalizes model complexity directly: T is the number of leaf nodes of tree f, $w \in \mathbb{R}^T$ the vector of leaf output weights, $\gamma \geq 0$ a penalty controlling tree size and $\lambda \geq 0$ the L2 regularization coefficient that shrinks the leaf weights toward zero to reduce variance. Furthermore, while standard gradient boosting relies only on first-order gradients, XGB utilizes a second-order Taylor expansion of the loss function [36]. By calculating both the gradient and the Hessian (second derivative) for each training instance, the algorithm can optimize the tree structure more directly and rapidly converge on the optimal split points.

Advantages and Limitations. XGB also implements *shrinkage* and *column subsampling* to further prevent overfitting [36]. Shrinkage scales each new tree's contribution by a learning rate to slow convergence, which is enabled by default in the XGB package via `eta=0.3`. Column subsampling introduces feature randomization analogous to RF, but whereas RF draws a fresh random feature subset at every individual split node, XGB's `colsample_bytree` selects a random subset of features once per tree, so that all nodes within that tree compete only over the same fixed subset, though this mechanism must be explicitly configured, as the default value of `colsample_bytree=1.0` retains all features and effectively disables it.

Despite these algorithmic optimizations, the exact greedy split-finding procedure shared by all tree-based methods requires sorting continuous feature values across all candidates, a bottleneck that becomes prohibitive at the scale of millions of instances [36]. To accelerate this, XGB was extended to natively leverage GPUs [22]. Formalized by Mitchell and Frank, the GPU-accelerated tree construction algorithm exploits the massive parallelism and high memory bandwidth of modern GPU architectures. Instead of relying on exact sorting, the *histogram method* bins continuous feature values into discrete buckets and aggregates gradient statistics per bin. By evaluating split points using highly optimized parallel primitives over these histograms, the entire decision tree can be constructed directly in device memory [22]. This histogram-based split-finding approach allows XGB to scale efficiently to millions of instances, achieving significant speedups over multicore CPU implementations while fully accommodating the algorithm's second-order mathematical formulations. In their evaluation, Mitchell and Frank demonstrated the memory efficiency of this approach by processing a large-scale dataset of 11 million training instances and 28 features entirely within the memory of a single GPU, confirming its practical viability for large-scale classification tasks [22]. The histogram binning itself constitutes a genuine algorithmic efficiency gain in the Green AI sense: by reducing split-finding from an exact sort over all n samples to an aggregation over a fixed number of bins, it reduces the total computational work performed rather than merely accelerating it [36]. Mapping this

algorithm onto a GPU, however, is a separate step that does not further reduce total work. As Schwartz et al. [2] argue, hardware acceleration redistributes computational cost from time to power draw without diminishing the underlying workload, so the GPU speedup alone does not constitute an efficiency gain in the ecological sense [2].

2.3.4 Multilayer Perceptron

Deep learning is a subfield of ML concerned with learning hierarchical representations of data through compositions of multiple processing layers [37]. Rather than relying on hand-crafted features, deep learning models learn increasingly abstract representations directly from raw input, with each layer transforming its input into a more abstract representation that the next layer builds upon. The primary architecture through which this hierarchical representation learning is realized is the *artificial neural network*, a family of models loosely inspired by biological neural connectivity, in which computation is organized as a directed graph of parameterized units called neurons [37].

The MLP is the foundational instance of this family: a feedforward neural network architecture composed of sequential layers of neurons that apply linear transformations followed by non-linear activation functions. The Universal Approximation Theorem [38] provides the mathematical justification for this architecture, guaranteeing that a single hidden layer is sufficient to approximate any continuous function arbitrarily closely, given enough neurons in that layer. However, unlocking this immense representational capacity requires scaling the network's depth and width, which inherently increases both the model's susceptibility to overfitting and the computational burden of training.

Backpropagation and Optimization. Unlike decision trees, MLPs are optimized continuously using *backpropagation* [39], which propagates the gradient of the loss function backwards through the network layer by layer via the chain rule to compute each parameter's contribution to the error and Stochastic Gradient Descent (SGD) or its variants to minimize a predefined loss function. In practice, SGD is applied in its mini-batch form, where the gradient is computed not over the full dataset nor a single sample, but over small random subsets of the training data, trading gradient accuracy for computational efficiency and introducing beneficial noise that helps escape local minima. To accelerate this optimization process in modern applications, the Adaptive Moment Estimation (ADAM) solver is frequently deployed. ADAM is a computationally efficient, first-order gradient-based optimization algorithm [40]. It computes individual adaptive learning rates for different parameters from estimates of the first and second moments of the gradients. By maintaining exponential moving averages of the gradient and the squared gradient and applying an initialization bias correction to counteract initial estimates being biased toward zero, Adam effectively adapts to the geometry of the objective function.

Activation Functions. Historically, training deep architectures was severely hindered by the *vanishing gradient problem*. When utilizing traditional saturating non-linearities like the logistic sigmoid, the gradients of saturated neurons rapidly approach zero during backpropagation, making it nearly impossible for earlier layers to update effectively [41].

The widespread adoption of the Rectified Linear Unit (ReLU) largely resolved this bottleneck. By computing $y = \max(0, x)$, ReLU prevents gradients from vanishing for positive inputs and yields highly sparse representations [42]. Furthermore, its gradient is exactly 1 for positive inputs and 0 otherwise, making it highly efficient for training deep networks.

Batch Normalization. As MLPs scale in depth, they introduce considerable internal instability during training. During gradient descent, the distribution of inputs to any given hidden layer constantly changes as the parameters of all preceding layers are updated, a phenomenon termed *internal covariate shift* [43]. This forces the optimizer to use excessively low learning rates, substantially slowing down convergence. To address this, Batch Normalization explicitly standardizes the intermediate activations across each mini-batch to maintain a stable mean of zero and variance of one, before applying a learned affine transformation with parameters γ and β [43]. The original authors hypothesized that fixing these distributions was the primary reason the network could tolerate substantially higher learning rates without diverging, but the precise mechanism has since been debated. Later work demonstrates that this distributional stability has little to do with the success of the method. Instead, the Batch Normalization reparametrization inherently creates a smoother optimization landscape [44]. It is this smoothness that makes backpropagation highly resilient to parameter scale and allows the use of larger learning rates, ultimately resulting in substantially faster training times.

Regularization. Because an MLP will begin to memorize the sampling noise of the training data if training proceeds without constraint, regularization is required. *Dropout* provides a computationally elegant solution by randomly deactivating a fraction of hidden units during each forward pass, preventing neurons from developing complex co-adaptations where they solely rely on specific other units to correct their mistakes [28]. At test time, scaling the outgoing weights approximates an equally weighted geometric mean of all these shared-weight subnetworks, reducing generalization error without the prohibitive cost of training multiple large models [28]. *Early Stopping* complements this by monitoring a separate validation set and halting training once the validation error stops decreasing [45]. Beyond acting as an implicit regularizer, it serves as a critical efficiency mechanism, saving significant computational time by avoiding redundant training epochs.

Tree-based models vs. deep learning on tabular data. On tabular data, tree-based ensembles consistently outperform neural networks in low- and medium-data regimes, with the gap narrowing only when training instances scale into the millions [3, 4]. Three structural properties explain this advantage. Trees learn piecewise constant functions suited to the irregular, non-smooth target functions typical of tabular data, whereas neural networks are biased toward overly smooth solutions. Axis-aligned splits preserve the independent meaning of individual features and are robust to uninformative ones, while MLPs treat all features symmetrically and degrade when many carry no signal. Finally, neural networks require substantially more data to generalize without overfitting, making them ill-suited to smaller tabular problems [3]. This fundamental trade-off between predictive gain and

computational cost reinforces the need to evaluate models not solely on classification quality, but also on their overall energy efficiency.

2.4 Dataset Scaling and Learning Curves

In predictive modeling, the relationship between the volume of available training data and the resulting classification performance is formally modeled using a learning curve [46]. Empirical learning curves consistently demonstrate that performance improves as more data becomes available, but the rate of improvement eventually diminishes: the error typically decreases following a power law $\varepsilon(n) \propto n^{-\alpha}$ with $\alpha > 0$, meaning each additional doubling of training data yields ever smaller gains [27, 46]. This creates a fundamental tension in the context of Green AI: while computational and energy costs scale roughly linearly with dataset size, performance yields only sublinear growth, making it increasingly ecologically expensive to achieve stronger results simply by adding more data [2]. Achieving a marginal performance gain at the upper end of a learning curve demands a disproportionately large investment in additional data, time and energy.

Conversely, when dataset sizes are heavily restricted, classifiers generally experience degraded performance because smaller datasets contain fewer representative examples, leaving the model unable to generalize underlying patterns [47]. The risk of overfitting also increases: with limited samples, a model may memorize training noise rather than learn true structure. Crucially, however, the overall performance of a classifier depends more heavily on how well the dataset represents the underlying data distribution than on its sheer size [47]. A small but highly representative dataset can yield better generalization than a large but biased one [47].

Different classes of ML models exhibit different sensitivities to dataset scale. DL architectures are highly data-dependent: they must fit many parameters via backpropagation, meaning more data directly enables better adjustment and generalization [47], though beyond a certain scale even neural networks encounter the diminishing returns described by the learning curve above. Tree-based models are also sensitive at the smaller end of the scale, as recursive partitioning means predictions at deeper nodes rest on increasingly small subsets of examples, making tree depth unstable when the initial dataset is limited [32, 47]. In contrast, simpler linear models serve as robust baselines: their constrained functional form imparts a higher model bias but a much lower variance, making them far less prone to overfitting on limited datasets [27].

A complex model's classification performance must therefore be weighed against its computational footprint and the diminishing returns of the learning curve. This trade-off is especially pronounced in a Green AI context, where the ecological cost of scaling up model complexity or dataset size may exceed the marginal performance gain.

2.5 Evaluation Metrics

Assessing the ecological efficiency of a ML model requires more than measuring its predictive performance in isolation. A model that achieves strong predictive performance at

disproportionate computational cost is not unconditionally preferable to one that scores slightly lower but is significantly more resource-efficient [2]. This section therefore introduces the theoretical basis for three families of metrics. The first, performance metrics, covers classification quality and measures how accurately a model predicts class labels. The second, resource and model statistics, quantifies the energy expenditure and associated carbon emissions of training. The third, composite metrics, integrates both dimensions into a single score that captures the trade-off between predictive performance and ecological cost. The specific metrics and their experimental implementation are described in Section 4.4.

2.5.1 Performance Metrics

Confusion Matrix. At the core of evaluating classification quality is the *confusion matrix*, which cross-tabulates a model's predictions against the true labels. For a binary classification problem, this matrix consists of four fundamental components: True Positive (TP), True Negative (TN), False Positive (FP) and False Negative (FN). This concept extends naturally to multiclass problems by tracking predictions across a $M \times M$ grid, forming the basis for deriving all subsequent performance metrics [27].

Table 2.1: Binary confusion matrix with the four fundamental prediction outcomes.

	Predicted Positive	**Predicted Negative**
Actual Positive	TP	FN
Actual Negative	FP	TN

Accuracy. The most intuitive of these metrics is accuracy, defined as the overall proportion of correct predictions: $\text{ACC} = \frac{TP+TN}{N}$, where TP and TN are the correctly classified instances of each class and N is the total number of instances. While simple to interpret, accuracy exhibits a critical vulnerability when applied to datasets characterized by an uneven distribution of labels, known as *class imbalance*. For example, in a dataset where 99% of the instances belong to a single majority class, a trivial classifier that ignores all input features and simply predicts the majority class every time will achieve 99% accuracy. Despite this high score, the model learns no underlying patterns and fails completely at identifying the minority class.

Precision, Recall and F1-Score. This limitation motivates the use of more robust metrics, specifically precision and recall. Precision ($\frac{TP}{TP+FP}$) measures the proportion of positive predictions that are actually correct, while recall ($\frac{TP}{TP+FN}$) measures the proportion of actual positive instances successfully identified. Unlike accuracy, precision and recall evaluate performance entirely independently of true negatives, making them highly resilient to datasets where a large, heterogeneous majority class dominates the negative counts [48]. Because improving precision often comes at the expense of recall and vice versa, the F1-score combines them into a single metric calculated as their harmonic mean, formalized in Equation (2.5).

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{2.5}$$

The harmonic mean is explicitly chosen over the arithmetic mean because it heavily penalizes extreme values. A model that achieves near-perfect recall but abysmal precision will yield a low F1-score, forcing the classifier to balance both dimensions effectively. Consequently, the F1-score provides a more encompassing summary of overall model performance and overcomes the representation problems caused by imbalanced data distributions.

F1 Aggregation Strategies. When dealing with multiclass or imbalanced datasets, the F1-score can be aggregated in several ways. Macro F1 calculates the metric independently for each class and averages them, treating all classes equally regardless of their size [48]. Micro F1 aggregates the global contributions of TP, FP and FN across all classes before calculating the score, an approach that inherently favors larger classes [48]. Weighted F1-Score (WF1) calculates the score for each class independently but averages them using the true support of each class as a weight:

$$\mathrm{WF1} = \sum_{i=1}^{M} \frac{n_i}{N} \cdot F_{1,i} \tag{2.6}$$

where M is the number of classes, n_i is the true support of class i, $N = \sum_{i=1}^{M} n_i$ is the total number of instances, and $F_{1,i}$ is the F1-score of class i.

Among these aggregation strategies, WF1 is particularly well-suited to imbalanced settings. By weighting each class's score according to its true support, it yields a more representative picture of overall model performance than either Macro F1, which gives disproportionate influence to rare classes, or simple accuracy, which is entirely dominated by the majority class.

2.5.2 Resource and Model Statistics

Measuring classification quality captures only one dimension of model evaluation. Evaluating models under the Green AI paradigm requires moving beyond pure predictive quality to systematically measure the physical, computational, and environmental costs of achieving that performance [2]. To formalize this cost, [2] propose that the total computational and ecological cost of a ML result scales linearly with three factors: the cost of executing the model on a single example (E), the size of the training dataset (D), and the number of hyperparameter experiments (H). These three factors together define the scope of resource measurement in any Green AI evaluation.

Execution Time and Latency. Elapsed real time, measured in seconds, is a highly practical efficiency metric. It is commonly decomposed into two phases: total training time and inference latency (seconds per sample). Inference latency is particularly critical because deployed models are often queried millions of times. In production environments, the cumulative demand of repeated inference can account for up to 90 % of total ML energy costs, vastly eclipsing the initial training footprint [8]. Because execution time is highly dependent on the specific hardware and background system utilization, it cannot serve as a standalone proxy for ecological impact.

Energy Consumption and Carbon Emissions. To capture the true ecological footprint of ML, resource tracking must extend to physical energy and carbon emissions. A critical distinction

arises here: compute efficiency does not strictly equal energy efficiency, which in turn does not equal carbon efficiency [49]. Structural savings in execution time do not translate to emission reductions if the hardware is severely underutilized or if the computation is performed on an electricity grid heavily reliant on fossil fuels. Operational carbon emissions, expressed in gCO_2eq, are therefore the more complete ecological metric, as they incorporate both the measured energy consumption in kWh and the carbon intensity I_C of the local electricity grid. The foundational definitions and measurement methodology for these quantities are established in Section 2.2.

2.5.3 Composite Metrics

Composite Efficiency Metrics. To evaluate ML models across both dimensions at once, researchers increasingly employ composite efficiency metrics that weigh predictive performance against these resource expenditures in a single score. The general methodological motivation behind these composite metrics is to provide a unified heuristic for identifying models that offer the optimal trade-off between classification quality and ecological impact [2]. In the literature, such approaches include the Energy-Efficiency Ratio, Performance-per-Watt [8] and FLOPs-per-Sample (or per-inference) [2].

Theoretical Limitations. However, composite metrics possess inherent theoretical weaknesses. First, the resource-based components of these metrics, such as energy consumed or execution time, are intrinsically hardware-dependent. A composite score calculated in one environment is tied to the specific computational architecture and local energy grid, meaning it cannot be universally generalized or directly compared across different settings [2, 5]. Second, the weighting between performance and resource cost is highly scenario-dependent. Any composite metric must therefore make an explicit assumption about the relative value of predictive performance against resource cost, and that assumption is inherently context-dependent and should be stated transparently alongside the metric definition [16].

2.6 Hyperparameter Optimization

ML models are governed by two distinct categories of configurable quantities. Parameters are internal to the model and are learned directly from training data by minimizing a loss function [50]. In parametric models such as NNs, they take the form of fixed-size weight vectors; in non-parametric models such as decision trees, they manifest as split thresholds whose count grows with the data. Hyperparameters, by contrast, are external configuration choices that must be fixed prior to training: examples include the learning rate of an optimizer, the maximum depth of a tree, or the number of estimators in an ensemble [50]. Because hyperparameters govern the model's capacity, its regularization strength, which controls the degree to which overfitting is penalized and its optimization dynamics, which determine how the learning process converges toward a solution, their values have a direct and often decisive impact on generalization performance [50, 51].

The goal of HPO is to identify the configuration from a predefined search space Λ that minimizes the expected generalization error, where Λ is the Cartesian product of the

individual value ranges of each hyperparameter, as shown in Equation (2.7) and the error is estimated via Cross-Validation (CV) [51]

$$\Lambda = \Lambda_1 \times \Lambda_2 \times \cdots \times \Lambda_d \tag{2.7}$$

where Λ_i denotes the value range of the i-th hyperparameter and d is the total number of hyperparameters. As the number of hyperparameters grows, exhaustive evaluation of Λ becomes exponentially expensive [50, 51]. The following subsection first introduces the cross-validation procedure used to estimate the generalization error within each trial, after which the combinatorial explosion of the search space motivates the more efficient search strategies described thereafter.

2.6.1 Cross-Validation

Train-Test Split. The simplest approach to evaluating a model's generalization capability is the *hold-out method*, where the available data is partitioned into a single training and test split (for example, an 80/20 ratio). While computationally efficient, this approach is highly sensitive to the specific data partition and exhibits high variance in its performance estimate, particularly when the dataset is small [27]. To mitigate this instability in settings that require repeated model comparisons, such as HPO, K-fold CV is widely employed as a more stable estimation procedure [27].

K-Fold. The algorithm partitions the dataset into K roughly equal-sized folds. The model is trained K times. In each iteration, $K-1$ folds are used to fit the model, while the single held-out fold is used for testing [27]. The CV estimator is formalized in Equation (2.8).

$$\mathrm{CV}(K) = \frac{1}{K} \sum_{k=1}^{K} L\big(y_k, \hat{f}^{-k}(x_k)\big) \tag{2.8}$$

In Equation (2.8), L is the chosen evaluation metric, y_k are the true labels of fold k, x_k are the corresponding input features of fold k and $\hat{f}^{-k}$ denotes the model trained on all folds except the k-th [27]. The standard deviation of L across all K folds provides a quantitative measure of the model's stability across different data samples.

Selecting the number of folds K introduces a fundamental bias-variance trade-off. A small value of K is computationally less expensive but yields a higher estimation bias, as the model is trained on a significantly smaller fraction of the original data [27]. Conversely, setting $K = N$, known as Leave-One-Out Cross-Validation (LOOCV), trains the model N times using $N-1$ samples for each iteration. While LOOCV produces a nearly unbiased estimate of the test error, it is computationally prohibitive for large datasets and can exhibit high variance. Because the N training sets differ by only a single sample, the resulting estimators are highly correlated. Unlike independent estimators, whose averaged variance shrinks by a factor of $\frac{1}{N}$, the variance of correlated estimates does not diminish at this rate, so the averaging step of LOOCV provides little variance reduction [27]. To balance these competing factors, $K = 5$ and $K = 10$ are established as the standard recommendations in

statistical learning, offering an optimal compromise between computational feasibility, low bias and acceptable variance [27].

Stratified Cross-Validation. When evaluating datasets characterized by class imbalance, standard random partitioning may result in folds that do not accurately represent the underlying target distribution. To address this, *stratified CV*, a variant that ensures the original class ratio is preserved within every fold, is commonly applied. In practice, stratification is combined with random shuffling of the data before fold construction, so that the ordering of instances in the source file does not systematically bias any fold. The random seed used for shuffling must be fixed to make the fold assignment reproducible. The same class imbalance that motivates a support-weighted metric such as WF1 must also be faithfully reproduced in each fold's test set, ensuring that the evaluation procedure and the metric it produces remain coherent.

Selection Bias. A critical theoretical distinction arises when CV is employed not only for performance evaluation but also for HPO. If the identical CV folds are used to both select the optimal hyperparameters and estimate the final generalization error, the resulting performance metric exhibits an optimistic bias. This occurs because the hyperparameters are explicitly chosen to maximize performance on these specific held-out folds, which in the HPO context function as *validation sets*, leading to a form of indirect data leakage where the configuration implicitly overfits the validation sets. The magnitude of this bias is governed by two factors. First, it is inversely related to the size of the validation set: a small held-out fold introduces high variance in the model selection criterion, giving the optimizer opportunity to exploit the statistical peculiarities of that specific sample rather than genuine generalizable signal [52]. Increasing the validation set size directly reduces this variance, causing the selected hyperparameters to cluster more tightly around the true optimum [52]. Second, the bias grows with the number of hyperparameters to be tuned, as a larger search space provides more degrees of freedom through which the optimizer can inadvertently fit validation noise rather than the underlying generalization surface [52].

2.6.2 Grid Search and Random Search

Grid Search (GS). The most traditional approach to hyperparameter tuning is GS, an exhaustive strategy that evaluates the Cartesian product of a user-specified, finite set of values for each parameter [50]. While conceptually simple, GS suffers severely from the *curse of dimensionality* [50, 51]. As the number of hyperparameters increases, the computational complexity grows exponentially. For instance, optimizing five hyperparameters with ten candidate values each across three models and three datasets would require an exhaustive evaluation of 900,000 distinct configurations. This combinatorial explosion renders GS computationally prohibitive for complex ML architectures with limited budgets [50].

Random Search (RS). To overcome the inefficiency of exhaustive evaluation, RS was proposed as a highly effective alternative [51]. Instead of testing predetermined values on a rigid grid, RS selects a predefined number of parameter combinations randomly from the specified search space distributions [50]. Empirical and theoretical analyses demonstrate that RS is

significantly more efficient than GS in high-dimensional spaces [51]. The key insight is that the mapping from hyperparameter configurations to model performance, the so-called response surface, behaves as if it only has a few relevant dimensions, even when many hyperparameters are nominally present. In practice, only a small subset of hyperparameters substantially impacts the generalization error, while the others leave the response surface nearly flat and can be varied freely without consequence [51]. Because GS forces allocations across all dimensions equally, it wastes the vast majority of its computational budget exploring irrelevant parameters. RS, by contrast, tests a unique value for every parameter in every trial, allowing it to thoroughly explore the relevant subspaces with far fewer evaluations [51].

Limitations. Despite their differences in candidate selection, both GS and RS share a fundamental methodological limitation: they are entirely uninformed algorithms [50]. Every trial in their search process is independent, meaning neither method utilizes the performance results of previously evaluated configurations to influence the selection of future points [50]. Consequently, they inevitably waste massive amounts of time and computing resources evaluating poorly performing areas of the configuration space without learning from past failures [50]. This inherent inefficiency establishes the primary motivation for adopting sequential, history-dependent strategies such as Bayesian Optimization (BO).

2.6.3 Bayesian Optimization

BO offers a sequential, model-based approach that utilizes the history of previously evaluated configurations to intelligently determine the next hyperparameter values to test [50]. BO operates using two primary components: a probabilistic *surrogate model* that approximates the true, computationally expensive objective function and an *acquisition function*, such as Expected Improvement (EI), which selects the optimal next point for evaluation [50, 53]. The acquisition function explicitly balances the fundamental optimization trade-off between *exploration* (sampling from under-explored regions of the parameter space where uncertainty is high) and *exploitation* (sampling from regions already known to yield promising results) [50].

Tree-structured Parzen Estimator (TPE). While traditional BO relies on Gaussian Processs (GPs) as surrogate models to directly estimate the conditional probability $p(y|x)$ of the objective value y given the hyperparameters x, the TPE takes a fundamentally different approach [53]. Instead of asking "given these hyperparameters, what performance can I expect?", TPE inverts the question: "given that a trial turned out well or poorly, what kinds of hyperparameter values were likely to produce it?" [53]. Formally, this corresponds to modeling $p(x|y)$, the distribution of hyperparameter configurations x conditioned on the observed performance y, rather than $p(y|x)$ directly. TPE implements this by splitting all past trials at a performance threshold γ into two groups and fitting a separate density to each: $\ell(x)$ captures the distribution of configurations that performed well and $g(x)$ captures those that performed poorly [50, 53]. The next candidate is then chosen to maximize the

ratio $\frac{\ell(x)}{g(x)}$, favoring configurations that are likely under the good density and unlikely under the poor one [53].

Conditional Hyperparameters. This inverse modeling formulation grants TPE a significant advantage in complex configuration spaces. Because the Parzen estimators are organized in a tree structure, TPE naturally handles conditional hyperparameters, that is, parameters that only exist or only matter depending on the value of another parameter [50]. For example, a dropout rate is only relevant if dropout is enabled at all. If it is disabled, the dropout rate has no effect on performance regardless of its value. A standard GP treats all hyperparameters as independent continuous dimensions and cannot represent such if-then relationships. TPE's tree structure, by contrast, can branch on a parent hyperparameter's value and only then consider its dependent children, faithfully reflecting the actual structure of the search space [50, 53].

2.6.4 Computational Comparison

Furthermore, TPE offers a substantial computational advantage over both GPs and exhaustive search methods. While GP-based models scale cubically with the number of past observations $\mathcal{O}(n^3)$, making them prohibitively slow as the search history grows, TPE achieves a much lower time complexity of $\mathcal{O}(n \log n)$ [50]. By sequentially directing the search only toward the most promising regions and maintaining low computational overhead per trial, sequential strategies like TPE bypass the exponential combinatorial explosion that plagues GS, enabling the efficient optimization of complex ML architectures with many hyperparameters [50].

Table 2.2: Comparison of HPO strategies (n: trials, d: hyperparameters).

Method	Strategy	History	Complexity
GS	Exhaustive	No	$\mathcal{O}(n^d)$
RS	Random sampling	No	$\mathcal{O}(n)$
BO (GP)	Sequential, model-based	Yes	$\mathcal{O}(n^3)$
TPE (Optuna)	Sequential, model-based	Yes	$\mathcal{O}(n \log n)$

2.6.5 Optuna as a Practical Framework

Optuna is a modern HPO framework designed around a *define-by-run* Application Programming Interface (API) [54], which allows the search space to be constructed dynamically during runtime rather than requiring a static definition beforehand. Unlike traditional tools that require the search space to be statically defined prior to execution, Optuna constructs the hyperparameter search space dynamically within the objective function during runtime, accommodating complex dependencies without relying on rigid external configurations. Optuna utilizes the TPE as its default sampling algorithm, allowing for efficient exploration of these dynamically generated spaces. To ensure reproducibility, the stochasticity of the TPE sampler can be controlled by fixing a global random seed.

Ultimately, the adoption of an efficient framework like Optuna is not merely a mathematical convenience, but a fundamental Green AI strategy. The total computational and ecological cost of a ML result grows linearly with the number of hyperparameter trials executed [2]. By utilizing BO to strategically minimize the number of required evaluations, the overall energy consumption and resulting carbon footprint of the entire model development process are significantly reduced.

2.7 Chapter Summary

This chapter established the theoretical foundations for the empirical study. The Green AI framework motivated the treatment of ecological cost as a first-class evaluation criterion alongside predictive performance. The energy and emissions section formalized how power draw translates into carbon footprint through grid carbon intensity, established why TDP-based estimation is unreliable and introduced CodeCarbon as the standardized measurement tool. The models section described the five classifiers under examination, their inductive biases on tabular data and the conditions under which GPU acceleration is genuinely energy-efficient rather than merely faster. Dataset scaling and learning curves established the sublinear relationship between data volume and performance gains, reinforcing the Green AI argument that ecological costs can outpace predictive returns. The evaluation metrics section introduced WF1 as a robust classification quality metric for imbalanced datasets and outlined the theoretical limitations of composite efficiency metrics. CV was formalized as an estimation procedure used within hyperparameter optimization, with stratification linking directly to the metric choice under class imbalance. Finally, HPO was covered from exhaustive search through BOs to Optuna, with HPO trial count identified as an ecological cost in its own right.

3

Related Work

This chapter surveys empirical studies that have measured the energy consumption and carbon emissions of ML systems. Each section closes by identifying what prior studies leave unmeasured or unresolved, collectively motivating the specific design choices of this study.

3.1 Empirical Energy Benchmarks for Machine Learning

As empirical benchmarking has replaced theoretical complexity analysis as the primary mode of model evaluation, a body of literature has emerged that measures the actual energy consumption and carbon emissions of training ML systems. This literature has, however, grown unevenly: the studies that sparked the Green AI movement focused almost exclusively on large-scale DL, while the empirical benchmarks covering tabular data and classical models ignored energy entirely.

The case for treating energy as a first-class evaluation metric was made most forcefully by early measurements of DL workloads. Strubell et al. [1] were among the first to directly measure hardware power interfaces during Natural Language Processing (NLP) model training, finding that a single large transformer architecture could emit as much CO_2eq as five cars over their entire lifespans and helping to establish the Green AI research agenda. Follow-up work examined the structural drivers of this footprint. Dodge et al. [5] expanded the empirical scope to 16 cloud computing regions, demonstrating that hardware choice and grid carbon intensity are as consequential as algorithmic decisions, with the same training run emitting between 7,000 and 26,000 gCO_2eq depending on the data center. Desislavov et al. [8] extended this perspective to the inference phase, showing that the operational footprint of deployed models can rival the cost of training. A step toward more modest workloads was taken by Henderson et al. [16], whose experiment-impact-tracker framework demonstrated that everyday reinforcement learning experiments on standard hardware generate clearly measurable emissions, establishing ecological efficiency as a concern for ordinary research rather than exclusively for large-scale industrial training. Despite this broadening scope, the entire strand of Green AI benchmarking has remained focused on NNs trained on unstructured data such as text and images.

The empirical literature on tabular data took a different path entirely. A series of large-scale benchmarks asked which model family generalizes best to structured, heterogeneous datasets and consistently found that classical models hold their own against DL. Grinsztajn et al. [3] evaluated 45 tabular datasets and found tree-based models such as RF and XGB to outperform deep NNs, a result echoed by Gorishniy et al. [4] in a separate benchmark that reaffirmed the dominance of gradient boosting on this data type. Both studies, however,

treated energy as entirely outside their scope: runtime was noted as a secondary metric, but physical power consumption and carbon emissions went unmeasured.

Only recently have some studies begun to bridge these two strands. Rodriguez et al. [6] evaluated software-based energy trackers against physical hardware meters, measuring classical ML models alongside DL models as part of their methodology. Verdecchia et al. [12] took a data-centric approach, empirically measuring how reducing data points and features affects the training energy of six AI models and finding that data-centric dataset modifications can substantially reduce training energy, though the performance cost varies depending on whether data points or features are reduced. These contributions are important, but they remain limited to datasets of a few thousand instances and do not examine how energy and predictive performance co-evolve across the orders-of-magnitude variation in dataset size that characterizes real-world applications.

Research Gap. The existing literature therefore reveals a clear gap at the intersection of two well-developed but isolated fields. Where Green AI benchmarks have established energy as a measurable, consequential quantity, they have done so exclusively for DL on unstructured data [1, 5]. Where experimental benchmarks have rigorously evaluated classical models on tabular data, they have focused entirely on predictive performance [3, 4]. A systematic analysis of how energy consumption, carbon emissions and predictive performance jointly scale with dataset size for classical classification models remains absent.

3.2 Dataset Size Effects on Performance and Energy Consumption

The tension between data volume and model quality has a specific shape. Schwartz et al. [2] conceptualize this in their Equation of Red AI, which models the computational cost of a result as growing linearly with training dataset size. At the same time, performance does not grow linearly with data: empirical learning curves consistently follow an inverse power law, meaning that performance gains become increasingly marginal as the training set expands [46]. The practical consequence is that energy costs scale proportionally with data volume while performance returns diminish, making large datasets progressively less efficient investments. Koshute et al. [46] formalize the point where these curves cross as the Sufficient Training Set Size (STSS): the minimum volume beyond which additional data is ecologically unjustifiable. A complementary finding from Althnian et al. [47], based on medical classification tasks, adds that the key factor is not absolute size but representativeness, meaning that a small, well-sampled dataset can generalize as well as a much larger but biased one.

The energy implications of dataset reduction have been directly and concretely measured by Verdecchia et al. [12], who trained six AI models while systematically reducing both data points and features in 10% increments. Two distinct patterns emerge from their results. Reducing the number of features yields energy savings of up to 76% while largely preserving predictive performance, making it a near-cost-free optimization. Reducing the number of data points produces larger savings, up to 92.16% for RF, but at a meaningful performance cost that must be weighed explicitly. The effect is also model-specific: K-Nearest-Neighbor

(KNN) achieves a maximum reduction of only 61.72% from fewer rows, while ensemble methods such as RF are considerably more sensitive to training set size. Beyond dataset modifications, the study finds that switching to a less complex model can alone yield energy savings of up to two orders of magnitude, establishing model selection as one of the most powerful levers for ecological efficiency.

Research Gap. These findings rest, however, on a single dataset of 5,574 instances, run exclusively on a standard quad-core CPU and without GPU-accelerated or DL baselines. Whether the same energy-performance patterns hold at the scale of millions of instances and whether they generalize across fundamentally different model families, remains empirically unexplored.

3.3 CPU vs. GPU Efficiency: Empirical Evidence

Whether GPU training is more energy-efficient than CPU training cannot be answered by measuring power draw alone. A GPU that draws five times the wattage of a CPU can still consume less total energy, if it finishes the same training task more than five times faster. The relevant quantity is energy, specifically power integrated over time and how that relationship plays out depends heavily on the model and the dataset [55].

The power side of this equation is well established empirically. Dodge et al. [5] report observed GPU draws of 250 W to 350 W under load from cloud training environments, compared to 10 W to 150 W for CPUs. During Bidirectional Encoder Representations from Transformers (BERT) training, a workload representative of high-utilization DL, the GPU alone accounted for 74% of total system electricity. For NN architectures of this type, the elevated power draw is more than offset by execution speed: the parallelism of DL maps efficiently onto GPU hardware and training that would take days on a CPU completes in hours. For this class of workload, the GPU is the more energy-efficient device.

Classical ML models on tabular data present a different picture. Mitchell and Frank [22] measured GPU acceleration for XGB specifically, finding speedups of $3\times$ to $6\times$ over a four-core desktop CPU and $1.2\times$ over a dual-socket server with 24 CPU cores combined. These gains are real but considerably smaller than those achieved for NNs. Given the GPU's substantially higher power draw, a $3\times$ to $6\times$ speedup may not be enough to recover the additional energy cost and Mitchell and Frank measured only execution time, not energy consumption.

Research Gap. The question of where the break-even point lies for models like XGB therefore remains open. A dataset may exist below which the CPU is the ecologically preferable device and above which the GPU's speedup compensates for its power draw, but this has not been empirically established for gradient boosting on tabular data.

3.4 Measurement Tool Accuracy: Software Estimation vs. Hardware Sensors

As Green AI research has grown, so has demand for accessible tools that quantify the energy consumption and carbon emissions of model training. The emerging ecosystem of software trackers and analytical estimators has simplified emissions measurement. However, recent empirical work reveals a significant problem: different tools often report strikingly different results for the same experiment. These discrepancies are not random but represent systematic errors in the underlying estimation models.

Several measurement approaches exist, ranging from hardware power meters at the wall outlet to on-chip sensors and software estimation models [6]. In practice, cost and access constraints push most researchers toward estimation-based tools, despite evidence that these introduce substantial and directional errors.

How large these errors can be is documented by Bannour et al. [23], who compared six tracking tools on the same NLP training runs. Tools relying on analytical models and user-supplied parameters systematically overestimated consumption, while those reading directly from hardware sensors produced more consistent results. The underlying reason for this divergence is that analytical approaches cannot capture the dynamic fluctuations of real hardware. Henderson et al. [16] demonstrated this empirically: TDP-based estimates significantly over- or underestimate actual energy compared to sensor readings, because they ignore real-time utilization shifts and dynamic memory effects and FLOPs-based estimates often show little correlation with actual consumption across architectures due to hardware-specific firmware behaviors. Rodriguez et al. [6] confirmed this pattern in practice, finding that purely analytical tools consistently deviated from measured ground truth, while sensor-based tools remained closer, though still subject to platform-specific constraints.

More recent work has moved toward direct validation of widely adopted trackers against physical ground truth. Fischer [7] compared CodeCarbon against external power meters across a range of workloads, finding that CodeCarbon consistently underestimates total system power draw by 20 to 30%, as software profilers cannot account for cooling overhead or power supply inefficiencies. Critically, this underestimation is substantially worse for CPU-only workloads than for GPU workloads, with relative errors exceeding GPU-equivalent errors by more than an order of magnitude in some configurations, a finding with direct consequences for any study benchmarking classical ML, which runs predominantly on CPU.

3.5 Seasonal and Geographic Carbon Intensity Effects

The carbon footprint of a ML experiment is not determined solely by model and hardware choices. Because electricity grids draw on different fuel mixes depending on geography, time of day and season, two identical training runs executed on different infrastructure can produce vastly different emissions, making geographic and temporal factors a distinct dimension of Green AI research.

The geographic component of this variability has been measured at scale. Dodge et al. [5] documented factors of up to 40 between the most and least carbon-intensive cloud regions, establishing workload placement as a practical lever for emissions reduction alongside hardware and model choice.

Temporal variation tells a more nuanced story. Dodge et al. [5] find that diurnal variation is meaningful: starting a training run at midnight rather than during peak solar generation hours can increase emissions by up to 8% in certain regions and they propose time-shifting or dynamically pausing workloads based on real-time grid data. Seasonal variation, however, proves negligible across their 12-month tracking of globally distributed cloud instances, leading them to conclude that month-to-month emissions differences within a single cloud region are small.

Research Gap. This conclusion may not transfer to local electricity grids with a high share of seasonally dependent renewables, which can exhibit considerably stronger swings than global cloud averages suggest.

3.6 Hyperparameter Tuning Efficiency

To achieve state-of-the-art predictive performance, ML models require extensive hyperparameter tuning. Since this process involves training architectures multiple times across various configurations, a critical tension emerges between the ecological cost of finding the optimal configuration and the performance benefits it yields.

The Red AI paradigm has normalized evaluating only the final, best-performing model, a practice that obscures the computational budget spent during the development and tuning phases [2]. Practical measurements reveal that this hidden cost can be substantial. Strubell et al. [1], in an empirical case study tracking the development of a NLP pipeline, quantified this overhead directly. A single tuning cycle evaluating 24 configurations multiplied the computational cost of the baseline model by a factor of 24. Further the complete development process, which required training 4,789 model variants to find the optimal hyperparameters, consumed 239,942 GPU hours in total. This demonstrates that the tuning phase can multiply the baseline energy footprint of a single model by several orders of magnitude.

To reduce these impacts, researchers must move away from exhaustive methods like grid search. Lacoste et al. [17] explicitly state that grid search is highly inefficient in terms of both model performance and environmental impact and that replacing it with random or model-based search strategies can accelerate convergence and directly reduce carbon emissions. Experimental benchmarks support this: Bergstra and Bengio [51] show that as few as 8 to 32 random trials can consistently match or outperform exhaustive grid search depending on the size of the search space. Going further, Bergstra et al. [53] illustrate the practical scale of this problem on a Deep Belief Network with 32 hyperparameters: evaluating only two values per parameter under grid search yields over four billion combinations, which at roughly one hour of training each would require nearly half a million years of compute. By distributing asynchronous TPE proposals across five GPUs, they evaluated only 200 configurations in 24 hours of wall time and identified a result that outperformed expert-tuned baselines [53].

Research Gap. While the literature clearly documents the empirical tuning costs for DL models in NLP and Computer Vision [1], there is a lack of equivalent benchmarks for classical ML on tabular data. How much energy and carbon emissions can be saved by reducing the number of tuning trials for models like RF or XGB and at what trial count the marginal performance gain ceases to justify the ecological cost, remains unquantified.

3.7 Lifecycle Emissions: The Ecological Cost of Deployment and Inference

While the ML community has heavily scrutinized the energy consumption and carbon emissions of the training phase, this narrow focus risks obscuring the true ecological cost of an AI system's complete lifecycle. A critical tension thus emerges: does the cumulative energy consumed during the deployment phase eventually outweigh the initial, highly-publicized training footprint?

Industry-scale empirical studies demonstrate that the inference phase is often the dominant factor in a model's total energy consumption. As models are integrated into everyday applications, they are queried millions of times, creating a multiplicative effect where the ongoing energy demand of continuous deployment quickly eclipses the one-time cost of training [8]. This dominance is reflected in industry estimates cited by Patterson et al. [55], who report that both NVIDIA and Amazon Web Services estimated that 80 to 90% of total ML computing demand in large-scale deployed systems is attributable to inference. Wu et al. [21] provide further empirical support at the production level, finding that for deployed language models at Meta, the inference phase accounts for 65% of total lifecycle resource consumption, significantly outweighing the training phase.

Consequently, recent research has begun explicitly measuring the inference footprint of pre-trained models at deployment scale. By systematically benchmarking energy consumption across diverse model architectures, Luccioni et al. [56] found that multipurpose generative models require orders of magnitude more energy per inference than task-specific systems, even when controlling for the number of model parameters. They also formalize the concept of a cost parity point: the number of predictions at which cumulative inference emissions surpass the initial training footprint, providing a concrete metric for lifecycle analysis. Accurately evaluating this break-even requires accounting for hardware specializations, as Desislavov et al. [8] note that inference efficiency depends heavily on whether a model can fully exploit the underlying accelerator's optimizations.

Research Gap. Despite the growing acknowledgment of inference costs, the current literature exhibits a significant blind spot. The lifecycle analyses and inference benchmarks provided by these studies rely almost exclusively on complex DL architectures applied to unstructured data: Luccioni et al. [56] evaluate only text and image modalities, Desislavov et al. [8] focus on ImageNet and GLUE benchmarks, Patterson et al. [55] restrict their analysis to large NLP models and Wu et al. [21] examine DL recommendation and language models. For classical ML models operating on tabular data, direct experimental evidence on lifecycle emissions is near-absent. It therefore remains unquantified at what number of predictions

the cumulative inference carbon footprint of a model such as LR or RF surpasses its training footprint and how this lifecycle scaling compares across models of varying complexity.

3.8 Composite Efficiency Metrics: Balancing Performance and Ecological Cost

The singular focus on predictive performance in ML has historically obscured the underlying computational costs of model training, raising a critical methodological question: How can researchers systematically evaluate both model quality and environmental footprint within a single, standardized framework?

To counteract the tendency to achieve marginal performance gains through massive computational expenditure, Schwartz et al. [2] argue that efficiency must be elevated to a primary evaluation criterion alongside predictive performance. Measuring algorithmic progress exclusively through predictive performance is fundamentally incomplete. Hernandez and Brown [57] demonstrate that true progress must be quantified by holding performance constant and measuring the reduction in computational cost required to achieve it. Taken together, these theoretical arguments advocate for composite metrics that mathematically integrate compute costs with model capabilities to provide a holistic view of algorithmic efficiency.

In practice, the ML community has already recognized the necessity of such combined measurements. Coleman et al. [58] introduced composite metrics such as Time-to-Accuracy and Cost-to-Accuracy in the DAWNBench benchmark, demonstrating that the community accepts end-to-end metrics that evaluate the time and financial budget required to reach a predefined quality threshold. Building upon this precedent, the MLPerf benchmark suite [59] formally adopted Time-to-Train as its primary evaluation metric, deliberately shifting the focus away from raw performance scores towards the holistic efficiency of the training system.

While these benchmarks validate the use of composite metrics, their efficiency components rely exclusively on execution time or financial cost [58, 59]. As established in previous sections, neither time nor cost necessarily correlates with the actual carbon footprint, particularly when hardware power draw and regional grid carbon intensities vary [49]. Recent theoretical frameworks have begun to recognize this shortcoming by proposing conceptual metrics, such as the Job Quality per Cost Rate (JQCR) [15], which aims to balance the quality of a computing job against its sustainability costs [15]. However, this approach remains strictly conceptual and lacks a defined mathematical formula linking specific performance measurements to carbon emissions [15].

Research Gap. To date, no established, mathematically operationalized composite metric explicitly incorporates a direct ecological component alongside predictive performance.

4 Methodology and Implementation

This chapter documents both the design decisions and the concrete implementation of the experimental pipeline. It covers the hardware and software environment, the selected datasets and models, the evaluation metrics, the validation strategy and hyperparameter tuning approach, the emissions and inference measurement infrastructure including a custom CPU power monitoring solution, and the results storage architecture. The final section defines the individual experiments that make up this study, directly preceding the results reported in Chapter 5.

4.1 Experimental Setup

4.1.1 Hardware Setup

All experiments were run on a single local machine to guarantee reproducibility and enable direct hardware energy monitoring. The system specifications are listed in Table 4.1.

Table 4.1: Hardware specification of the experimental system

Component	Specification
CPU	AMD Ryzen 7 5700X (8 cores / 16 threads, 3.6 GHz base)
GPU	NVIDIA RTX 4070 (12 GB GDDR6, 5,888 CUDA cores)
DRAM	32 GB DDR4
Operating System	Windows 11

The pairing of a mid-range consumer GPU (RTX 4070) with a desktop CPU (Ryzen 7 5700X) enables direct CPU-versus-GPU training comparisons on a single machine, which is central to answering RQ 3.

4.1.2 Software Environment and Reproducibility

All experiments were implemented in Python 3.13.0. The full implementation is publicly available [60], where a `requirements.txt` file is provided so that the exact library versions used in this study can be reproduced. The main libraries used are described below.

noch private

Scikit-learn [31] served as the foundation of the evaluation pipeline. It provided the `LogisticRegression` and `RandomForestClassifier` implementations, as well as the `StratifiedKFold` (with `shuffle=True`) for CV within hyperparameter tuning and `StandardScaler` for feature normalization which is used for LR and MLP. It also supplied `make_pipeline` for chaining preprocessing and model steps, and `make_scorer` together with WF1 for evaluation metrics. Final model evaluation uses a single `model.fit(X_train,`

`y_train)` call followed by `model.predict(X_test)`, with WF1 computed on the held-out test partition using `f1_score(average="weighted")`.

The XGBoost library [61] was used for the gradient boosting model, as it provides a highly optimized implementation that supports both CPU and GPU training.

For the MLP, PyTorch [62] was used as the DL backend. To integrate the PyTorch model into the scikit-learn evaluation pipeline, the Skorch library [63] was used, which wraps PyTorch models in a scikit-learn-compatible interface.

Carbon emissions and energy consumption were tracked using CodeCarbon [26], which is described in detail in Section 4.7.

4.2 Datasets

Dataset size is a primary driver of both computational demand and energy consumption during model training [6]. To evaluate how this relationship varies across different scales, three classification datasets were selected, varying significantly in size and enabling a systematic evaluation of how model performance and energy consumption scale across different levels of data complexity. The jump from 30,000 to 11,000,000 instances is large, spanning nearly three orders of magnitude. The datasets are summarized in Table 4.2.

Table 4.2: Overview of datasets used in this analysis

Dataset	Instances	Features	Classes	Scale
Wine [9]	178	13	3	Small
Default of Credit Card Clients [10]	30,000	23	2	Medium
HIGGS [11]	11,000,000	28	2	Large

The Wine [9] dataset contains 178 instances across 13 features, where each feature describes a chemical property of wines from the same region in Italy, derived from three different cultivars.

The Default of Credit Card Clients [10] (hereafter: Credit) dataset contains 30,000 instances across 23 features and focuses on predicting payment default among credit card clients in Taiwan.

The HIGGS [11] dataset is the largest dataset used in this study, comprising 11,000,000 instances across 28 features, seven of which are high-level features derived by physicists to help discriminate between the two classes [64]. The classification goal is to distinguish between signal events produced by Higgs bosons and background noise processes.

A central `config.py` stores all dataset-related settings as a single source of truth: file paths, target column names, optional row limits (`nrows`), and dataset-specific properties such as column names and label offsets. The global constants `RANDOM_STATE` and `CV_FOLDS` are also defined here. The `load_data` function reads each dataset using the appropriate mechanism (`read_csv` for Wine and Credit, `read_parquet` for HIGGS), applies any configured preprocessing steps, and returns the feature matrix X and target vector y, as visualized in Figure 4.1. RF is explicitly excluded from HIGGS: training times at eleven million rows are prohibitive, so the script exits early for that combination.

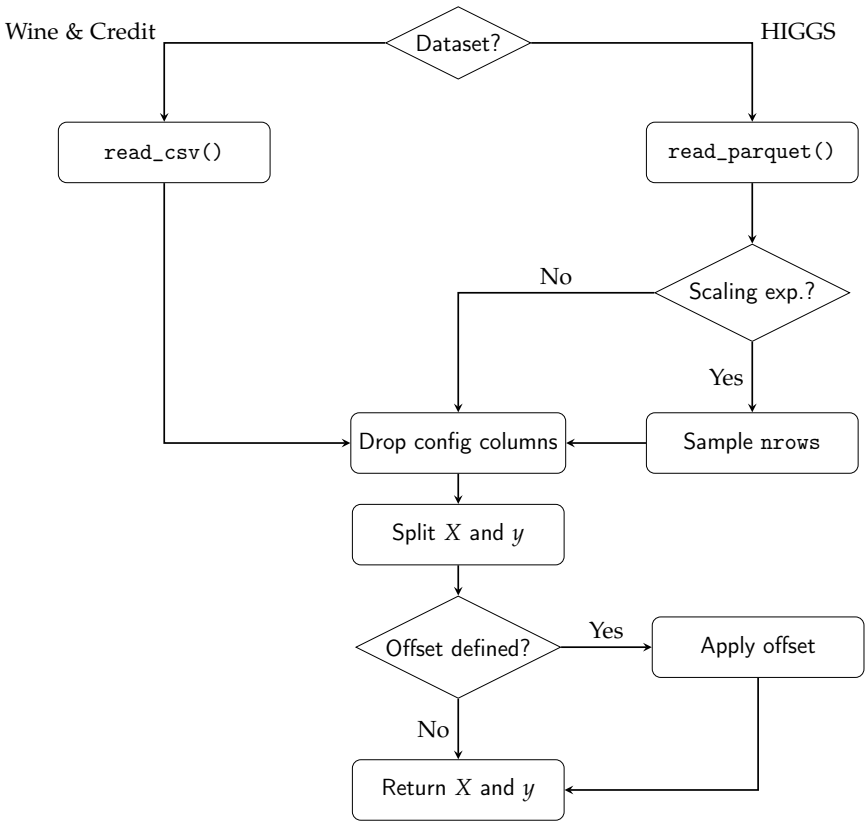


Figure 4.1: Visualization of the data loading pipeline within `load_data()`. The `nrows` sampling step only applies during the scaling experiment. The label offset (-1) is applied only to the Wine dataset to shift class labels from $\{1, 2, 3\}$ to $\{0, 1, 2\}$.

A thin wrapper around `load_data`, called `load_data_split`, serves as the single entry point for all model and tuning scripts. It calls `load_data` and immediately applies a stratified 80/20 train–test split using `train_test_split` with `RANDOM_STATE` and `TEST_SIZE = 0.2` from `config.py`, returning the four arrays `X_train`, `X_test`, `y_train` and `y_test`. Centralizing the split in one function ensures that tuning and training scripts see identical partitions. The rationale for this design is discussed in Section 4.5.

4.3 Models

Four classification methods were selected to span a range of model complexity, from a linear baseline to a deep NN. The table below summarizes their roles and hardware assignments.

Table 4.3: Overview of classification models used in this study

Model	Type	Device	Role
LR	Linear	CPU	Baseline
RF	Ensemble	CPU	Mid-complexity
XGB	Gradient Boosting	CPU & GPU	Mid-complexity
MLP	NN	GPU	High-complexity

4.3.1 Logistic Regression

Role. LR serves as the lightweight baseline in this study. Despite its simplicity as a linear classifier, it often achieves competitive performance, making it well suited to quantifying whether the additional training cost of more complex models yields a meaningful gain in predictive performance. As a gradient-based model, it requires feature scaling, and a standard scaler is applied to normalize all input features to zero mean and unit variance before training.

Implementation. The algorithm is embedded in a scikit-learn pipeline with a `StandardScaler` as the first step. The solver is set to `sag`, chosen for its efficiency on large datasets as described in Section 2.3.1, and `max_iter` is capped at 1000 to ensure convergence. No HPO is applied.

Threading Constraint. It is worth noting that scikit-learn's `LogisticRegression` is single-threaded for the core optimization step regardless of solver choice: none of the available solvers (`lbfgs`, `newton-cg`, `sag`, `saga`, `liblinear`) parallelize a single model fit. The `n_jobs` parameter of `LogisticRegression` only applies to OvR multiclass decomposition, which provides no benefit for the two binary-classification datasets in this study. As a result, training runs on a single CPU core and produces roughly 10% utilization on the eight-core system used here. The implications of this single-threaded constraint for energy comparability are discussed in Section 4.7.

4.3.2 Random Forest

Role. RF was selected as a mid-complexity representative of tree-based ensemble methods. Relevant to this study, RF does not support GPU-accelerated training and is therefore run exclusively on the CPU. It is the only model excluded from one dataset combination: due to prohibitive training runtimes at 11 million instances, it is not evaluated on HIGGS in the main benchmark. It is, however, included in the dataset scaling experiment for subset sizes up to 500,000 instances, where runtimes remain manageable. This provides data on the point at which RF becomes computationally impractical relative to the other models. All other model–dataset combinations are included in the main benchmark.

Implementation. RF requires no feature scaling and is parallelized across all available CPU cores via `n_jobs=-1`, which directly influences the energy consumption tracking and runtime measurements. As established above, it is excluded from the HIGGS dataset. The dataset-specific hyperparameters, determined via Optuna as detailed in Section 4.6, are listed in Table 4.4.

Table 4.4: Optimized hyperparameters for the RF classifier obtained via Optuna optimization.

Parameter	Wine	Credit
`n_estimators`	250	285
`max_depth`	20	5
`min_samples_split`	15	17
`min_samples_leaf`	6	8
`max_features`	`sqrt`	`None`

4.3.3 XGBoost

Role. Building on the tree-based approach, XGB was chosen as a second mid-complexity model, with the key distinction that it also supports GPU-accelerated training. This makes it possible to directly compare the energy consumption and training time of CPU- and GPU-based tree ensemble methods under identical conditions, which is central to RQ 3. The suitability of GPU-accelerated XGB for datasets at this scale has been demonstrated by Mitchell and Frank, who show that the histogram-based GPU implementation can process a dataset of 10 million instances with 28 features entirely within GPU memory [22].

CPU and GPU Variants. The classifier is evaluated in two variants: one restricted to the CPU and one using GPU acceleration. To isolate hardware as the sole variable, the configuration dictionaries are kept identical for both executions. The only difference is the device parameter, which is set to GPU for the second variant. The resulting hyperparameters are applied to both variants without modification, which is valid because the histogram-based algorithm is the same on both devices and the optimal configuration is not expected to depend on the execution device. Minor floating-point differences between the CPU and GPU histogram builds can still produce slightly different splits, but these do not shift the tuned hyperparameters in any systematic way.

Objective Configuration. XGB has a specific technical requirement for handling target variables: while RF natively adapts to different class structures, XGB requires explicit instructions for its learning objective. As mentioned in Section 4.2, the Wine dataset consists of three distinct categories, so the objective parameter is set to `multi:softmax`, paired with the `mlogloss` evaluation metric and a specified number of classes. The Credit and HIGGS datasets contain only two categories, so their objective is configured as `binary:logistic` and evaluated using the standard `logloss` metric.

Feature Handling and Crossover Analysis. The algorithm processes the unscaled feature matrix X directly and uses the globally defined random state to guarantee deterministic execution. The CPU versus GPU comparison is derived in the plotting pipeline by reading all HIGGS subset rows from `results.csv` and fitting a log-log model separately for each device using `numpy.polyfit`, with the energy crossover point computed analytically from the intersection of the two fitted lines. The resulting configurations used for the final training phase are detailed in Table 4.5.

Table 4.5: Optimized hyperparameters for the XGB classifier across all datasets.

Parameter	Wine	Credit	HIGGS
`objective`	`multi:softmax`	`binary:logistic`	`binary:logistic`
`eval_metric`	`mlogloss`	`logloss`	`logloss`
`num_class`	3	N/A	N/A
`n_estimators`	123	405	406
`max_depth`	8	5	8
`learning_rate`	0.0773	0.0149	0.2263
`subsample`	0.8832	0.8686	0.8910
`colsample_bytree`	0.6082	0.9229	0.8222
`min_child_weight`	10	3	3

4.3.4 Multilayer Perceptron

Role. The MLP was selected as the most complex model, representing the NN category. Its highly configurable architecture, particularly the number of hidden layers and neurons per layer, makes it well suited for the architectural complexity experiments conducted in this study, where these parameters are systematically varied to assess their impact on predictive performance and energy consumption. Like LR, the MLP also requires standard scaling, which is applied identically.

Dynamic Architecture. The implementation uses PyTorch as the primary DL framework combined with the Skorch library, which wraps the PyTorch model in a scikit-learn-compatible interface required by the pipeline and tuning infrastructure. A dynamic architecture was chosen to ensure the network structure directly reflects the optimal parameters found during the Optuna tuning process. At runtime, the script reads the best parameter set from a JSON file generated during the tuning phase. It extracts the number of hidden layers via `n_layers` and the neuron count for each individual layer via `layer_i`. These values are then assembled into a list called `layer_sizes`. This list is subsequently passed to the `MLPModule` class. This class constructs the full network dynamically upon instantiation, making the architecture entirely data-driven. The resulting layer-wise architecture, including parameter counts, is shown in Table 4.6. The specific configurations resulting from this process for each dataset are detailed in Table 4.7.

Device and Type Handling. To minimize the overall training time, the implementation checks for Compute Unified Device Architecture (CUDA) availability using `torch.cuda.is_available()`. The algorithm runs on the GPU if one is present and automatically falls back to the CPU otherwise. PyTorch requires highly specific numeric types and NNs react sensitively to implicit type mismatches. Therefore, the feature matrix X is explicitly cast to `float32` and the target vector y to `int64` before the training begins.

Training Configuration. Input features are standardized using a `StandardScaler` embedded in the scikit-learn `Pipeline`. The `NeuralNetClassifier` provided by Skorch receives all architecture parameters through a specific naming convention. It uses the `module__*` prefix to forward keyword arguments directly to the constructor of the `MLPModule`. The ADAM

Table 4.6: Resolved layer-wise architecture of the tuned MLP for each dataset. Each hidden block consists of a Linear layer, Batch Normalization, ReLU and Dropout. Parameter counts include weights, biases and Batch Normalization scale and shift parameters.

(a) Wine

Lyr	In/Out	Par.
HL 1	$13 \to 64$	1,024
HL 2	$64 \to 256$	17,152
Out	$256 \to 3$	771
Total		**18,947**

(b) Credit

Lyr	In/Out	Par.
HL 1	$23 \to 128$	3,328
HL 2	$128 \to 128$	16,768
HL 3	$128 \to 128$	16,768
HL 4	$128 \to 64$	8,384
Out	$64 \to 2$	130
Total		**45,378**

(c) HIGGS

Lyr	In/Out	Par.
HL 1	$28 \to 512$	15,872
HL 2	$512 \to 256$	131,840
HL 3	$256 \to 256$	66,304
Out	$256 \to 2$	514
Total		**214,530**

optimizer and the `CrossEntropyLoss` criterion are used consistently across all datasets. The learning rate and the batch size are taken directly from the tuning results and are therefore tailored to each dataset. A fixed random seed is applied using `torch.manual_seed` to make runs reproducible across iterations. On CPU this yields exact reproducibility, while on GPU results are reproducible only up to the residual non-determinism of cuDNN kernels, which is not eliminated by seeding alone and would additionally require the deterministic execution flags that were not enabled here.

Table 4.7: Optimized hyperparameters for the MLP across all datasets.

Parameter	Wine	Credit	HIGGS
`n_layers`	2	4	3
`layer_sizes`	[64, 256]	[128, 128, 128, 64]	[512, 256, 256]
`dropout_rate`	0.0448	0.2058	0.1041
`lr`	0.0007	0.0008	0.0048
`batch_size`	1024	4096	8192

Early Stopping. The maximum number of training epochs is set to 200. This value acts as an upper boundary rather than a strict target. An early stopping callback (`skorch.callbacks.EarlyStopping`) monitors the validation loss after each individual epoch. It halts the training process immediately if no improvement is observed for 10 consecutive epochs. This mechanism prevents unnecessarily long training runs and provides an additional layer of protection against overfitting. Consequently, the upper limit of 200 epochs is rarely reached in practice.

4.4 Evaluation Metrics

To answer the research questions of this study, two categories of metrics were employed: classification performance and ecological cost.

Classification Performance. WF1 was selected to measure predictive performance, as it provides a standardized basis for comparing models across datasets of varying class distributions.

Ecological Cost. CO_2eq emissions in gCO_2eq were chosen as the primary resource metric, directly reflecting the ecological cost of model training. Although CodeCarbon derives these figures from measured energy consumption in kWh, emissions are the primary reported metric, as they incorporate the carbon intensity of the local grid and therefore represent a more complete measure of ecological impact than raw energy alone. The underlying component energy is nonetheless retained where the analysis requires it, specifically for the scaling and lifecycle experiments.

Computational Overhead. Training time in seconds was included as a secondary resource metric, as it captures computational demand independently of hardware-specific power draw. Furthermore, inference time was measured across all experimental setups to account for the operational footprint. In large-scale deployment scenarios, the cumulative energy demand of repeated predictions can often rival or even exceed the initial training costs [8]. Together, these metrics enable a direct assessment of whether gains in predictive performance justify the associated increase in training cost.

Composite Efficiency Metric. Inspired by the efficiency framing proposed in Schwartz et al. [2], a custom composite metric was developed for this study, named CF1. Equation (4.1) defines this metric as the ratio of training CO_2eq emissions (in milligrams of CO_2-equivalent ($mgCO_2$eq)) to the WF1 (in percent):

$$\mathrm{CF1} = \frac{C_{\mathrm{mg}}}{WF1} \tag{4.1}$$

where C_{mg} denotes emissions in $mgCO_2$eq. Milligrams are chosen as the unit because training emissions in this study range from roughly 0.01 gCO_2eq to 90 gCO_2eq, which divided by WF1 scores near 70–99 % yields CF1 values in the range of approximately 0.1 to 1,000, which are human-readable integers that would become unwieldy fractions in grams. Lower values indicate a more ecologically efficient model: less carbon expenditure per unit of predictive performance. Concretely, a CF1 of 10 means that each percentage point of WF1 the model achieves costs 10 $mgCO_2$eq on average. Put differently, a model with twice the CF1 spends twice as much carbon per unit of predictive output. Unlike normalization-based composite metrics, CF1 is an absolute measure: its value is independent of which other models appear in the same comparison, making it meaningful even when a single model is evaluated in isolation.

It should be noted that C_{mg} is inherently location- and time-dependent, as emissions are tied to the carbon intensity of the local electricity grid, which varies both by region and over time [49]. Consequently, CF1 scores reflect the ecological efficiency of the specific experimental setup used in this study and may not generalize directly to other energy infrastructures or time periods.

Measurement Scope. Emissions are tracked separately for three distinct phases: hyperparameter tuning, model training and inference. In all three cases, data loading and preprocessing are deliberately excluded from the measurement interval, as their cost scales with dataset size rather than model architecture and would therefore introduce noise into cross-model comparisons. This ensures that reported emissions reflect only the algorithmic cost of each phase.

4.5 Validation Strategy

Train–Test Split. To obtain an unbiased estimate of generalization performance, the evaluation methodology employs a strict separation between model development and final testing. A shuffled, stratified 80/20 train–test split is applied to each dataset prior to any modeling steps. The resulting training partition is used exclusively for HPO and model fitting, while the held-out test set is reserved solely for the final evaluation.

Cross-Validation Strategy. Within the HPO phase, CV is used as the internal evaluation strategy on the training partition. Specifically, shuffled Stratified K-Fold CV with $k = 5$ folds is applied, a value chosen for its established balance between computational cost and estimation stability. Random shuffling is applied before fold construction to ensure that the order of instances in the original dataset does not influence fold composition. Stratified splitting then ensures that each fold preserves the original class distribution. This is particularly relevant for the Credit dataset, whose target is markedly imbalanced at roughly 78 % non-default to 22 % default. Wine is moderately balanced across its three classes (about 33/40/27 %) and HIGGS is close to even at about 53/47 %, so for these two stratification primarily guarantees that the existing class proportions are reproduced exactly in every fold rather than correcting a severe skew. This approach is directly coherent with the choice of WF1 as the primary metric and evaluation objective.

Final Evaluation. Final model evaluation is performed by training a single model on the full training partition and scoring it on the unseen test set. By strictly withholding the test data from the CV loop, this protocol ensures that the hyperparameters are not implicitly optimized for the test set, thereby preventing over-fitting during the model selection phase.

Reproducibility. To ensure reproducibility, a fixed random state was used for all splits, CV folds, model initialization, and data shuffling. The specific value and its technical implementation are described in Section 4.1.2 above.

Wine Dataset Limitation. A practical limitation applies to the Wine dataset: an 80/20 split of its 178 instances leaves approximately 36 test samples, which yields a noisier point estimate than the thousands or millions of test instances available for Credit and HIGGS. This is a structural constraint of the dataset size rather than a methodological choice, and the split ratio is kept uniform across all datasets to preserve comparability.

4.6 Hyperparameter Optimization

Rationale. Hyperparameter tuning was applied to all models except LR. Without tuning, default parameters tend to favor certain model families over others, potentially skewing both performance and training cost comparisons [65]. LR is deliberately excluded: its role as a lightweight linear baseline depends on its simplicity and tuning it would undermine that function.

Framework and Budget. BO-based methods require substantially fewer evaluations than grid or random search to reach competitive configurations, which directly reduces the energy cost of the tuning process itself. Because of that, Optuna was selected as the optimization framework, using TPE as its search strategy. Each run was configured to execute 40 trials per model–dataset combination, a budget chosen to balance search thoroughness against ecological overhead. All trials operate exclusively on the training partition and tuning was performed separately for each dataset. The optimization objective is WF1 in all cases.

Budget Limitation. The uniform 40-trial budget does not account for differences in search space dimensionality: the MLP's conditional space, whose effective dimensionality ranges from five to eight depending on sampled depth, is substantially harder for TPE to map than the low-dimensional spaces of RF or XGB. The implications for reported MLP performance figures and the rationale for keeping the budget symmetric are discussed in Section 6.3.

Implementation. The tuning logic is encapsulated in three dedicated scripts (`tune_xgb.py`, `tune_rfc.py` and `tune_mlp.py`), one per tunable model. All three follow the same structural pattern and differ only in the model instantiated and the search space queried from the trial object. The configuration shared across all three scripts is summarized in Table 4.8.

Table 4.8: Shared tuning configuration applied to all three Optuna scripts.

Setting	Value
Number of trials	40 (overridable via `N_TRIALS` env var)
Dataset selection	Command-line argument
Optimization metric	WF1 (maximize)
Sampler	`TPESampler` seeded with `RANDOM_STATE = 42`
CV strategy	`StratifiedKFold`, 5 folds, `shuffle=True`
Tuning data	Training partition only (80 % of full dataset)

Each parameter in Table 4.9 maps to a mechanism introduced in Chapter 2. For RF these are the tree structure and feature randomization controls. For XGB they are the shrinkage rate, column subsampling, and leaf-complexity penalty. For the MLP they are depth, width, dropout, learning rate, and batch size.

Objective Function. After loading and splitting the data via `load_data_split()`, each script defines an `objective(trial)` function that draws a candidate configuration from the trial object, instantiates the classifier and returns the mean WF1 from a shuffled `StratifiedKFold` CV on the training partition via `cross_val_score`. The held-out test set is never accessible to the optimizer. The rationale for this split-first design is discussed in Section 4.5 above.

Table 4.9: Hyperparameter search spaces defined in the Optuna tuning scripts.

Model	Parameter	Type	Range / Values
RF	`n_estimators`	integer	[100, 500]
	`max_depth`	integer	[3, 20]
	`min_samples_split`	integer	[2, 20]
	`min_samples_leaf`	integer	[1, 10]
	`max_features`	categorical	{`sqrt`, `log2`, `None`}
XGB	`n_estimators`	integer	[100, 500]
	`max_depth`	integer	[3, 8]
	`learning_rate`	float (log)	[0.01, 0.3]
	`subsample`	float	[0.6, 1.0]
	`colsample_bytree`	float	[0.6, 1.0]
	`min_child_weight`	integer	[1, 10]
MLP	`n_layers`	integer	[1, 4]
	`layer_i` (per layer)	categorical	{64, 128, 256, 512}
	`dropout_rate`	float	[0.0, 0.5]
	`lr`	float (log)	$[10^{-4}, 10^{-1}]$
	`batch_size`	categorical	{1024, 4096, 8192}

Everything else, such as emissions tracking and timing, is handled outside the objective function.

Reproducibility. Each study is created via `optuna.create_study(direction="maximize")` and parameterized with a `TPESampler` seeded with `RANDOM_STATE=42`. Seeding the sampler is what makes the trial sequence reproducible across re-runs. Without it, the suggested hyperparameter combinations would differ on every execution and the resulting tuning emissions would no longer be comparable. The optimization itself is launched through `study.optimize(objective, n_trials=N_TRIALS)`.

Emissions Tracking. To capture the energy footprint of the tuning process itself, the entire `study.optimize` call is wrapped in a CodeCarbon `EmissionsTracker` with a script-specific `project_name` (e.g. `tune_mlp_higgs`), so that tuning runs are clearly distinguishable from training runs in the `emissions/` directory. The same tracker is used across all three scripts and is configured identically to the trackers used during training, as described in Section 4.7 below.

Persisting Results. Once a study completes, the best score, the corresponding hyperparameter dictionary, the duration, the emissions reported by the tracker and the trial count are written to a shared `best_params.json` file via the `save_best_params()` utility. The function performs an upsert keyed by model and dataset, so that re-running a single tuning configuration overwrites only its own entry and leaves the other results untouched. The model scripts introduced in Section 4.3 consume this file through the corresponding `load_best_params()` helper, which decouples tuning from training entirely: the final training pipeline never instantiates Optuna, it merely reads the previously persisted hyperparameter dictionary and passes it to the classifier constructor.

4.7 Emissions Measurement

Measuring energy consumption and carbon emissions is the central objective of this study and the measurement infrastructure therefore required particularly careful design. This section describes how CodeCarbon was integrated into each script, how measurement coverage was defined across model and dataset combinations and what hardware-specific constraints and limitations apply to the collected values.

CodeCarbon Integration. Energy consumption and carbon emissions were tracked using CodeCarbon as described in Section 2.2.5. In each model script, an `EmissionsTracker` instance is created before the training run begins and configured with two key parameters: `output_dir`, which points to the shared `emissions/` directory at the project root and `project_name`, which uniquely identifies the run. The tracker is started by calling `tracker.start()` immediately before `model.fit(X_train, y_train)` and stopped via `tracker.stop()` once training completes. The return value of `tracker.stop()` is a single float representing the total CO_2eq emissions in kilograms accumulated over the entire tracked interval. This value is what gets passed to `save_results()` and written to `results.csv`.

Measurement Boundaries. The placement of the tracker boundaries was a deliberate choice. Data loading, preprocessing and the train–test split happen before `tracker.start()`, so they are excluded from the measurement. Only the single training run on `X_train` falls inside the tracked interval. This ensures that the reported emissions value reflects the cost of model fitting exclusively and that values are directly comparable across models and datasets without interference from data handling overhead.

Run Identification. To allow fine-grained analysis, each model–dataset combination is tracked by a dedicated `EmissionsTracker` instance, identified by a unique `project_name` following the pattern `<model>_<dataset>`, for example `lr_wine`, `rf_credit`, or `xgb_gpu_higgs`. This scheme makes individual runs immediately identifiable in the `emissions/` directory, where CodeCarbon writes one CSV file per tracked run in addition to returning the scalar value in code. Tuning scripts follow the same pattern preceded by `tune_`, for example `tune_mlp_higgs` or `tune_xgb_wine`. In total, the experiment produces tracked emissions records for 14 training runs and up to eight tuning runs. The separate CSV files serve as a raw audit trail, while the values extracted via `tracker.stop()` are the primary source for the analysis presented in Chapter 5.

Overwriting CPU Measurement. CodeCarbon's TDP-based CPU estimation on Windows was replaced in this study by a custom `CPUPowerMonitor` class implemented in `models/power_monitor.py`. It runs in a background thread and polls the CPU Package Power sensor every 0.25 seconds via the LibreHardwareMonitor library (`HardwareMonitor` Python package [66]). This library reads the CPU Package Power sensor directly from the hardware, providing true power values rather than a TDP approximation. In practice, CodeCarbon underestimated CPU power by a factor of roughly $9\times$ compared to the hardware sensor (e.g. 6.6 W vs. 57.8 W for LR on the Credit dataset). The corrected emissions value in `results.csv` therefore replaces CodeCarbon's CPU contribution with the hardware-measured value, while GPU

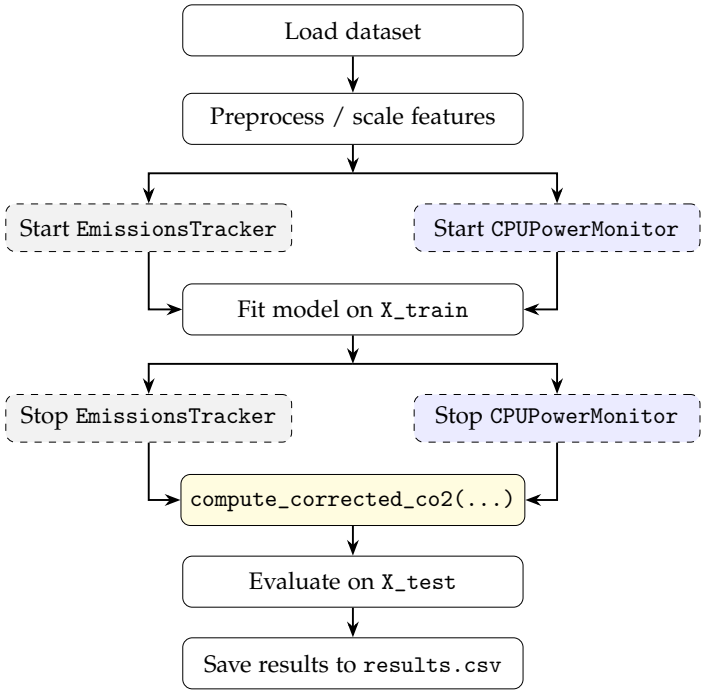


Figure 4.2: Training and emissions tracking procedure per model-dataset combination. Dashed gray boxes indicate CodeCarbon `EmissionsTracker` calls and dashed blue boxes indicate `CPUPowerMonitor` calls (background thread, 0.25 s sampling). Both run in parallel during training. After training, `compute_corrected_co2` replaces CodeCarbon's TDP-based CPU estimate with the hardware sensor value.

and Random-Access Memory (RAM) energy are still taken from CodeCarbon. The resulting corrected total energy (hardware CPU plus CodeCarbon GPU and RAM) is then converted to CO_2eq using the same static carbon intensity factor of 381 $\frac{gCO_2eq}{kWh}$ that CodeCarbon applies for Germany (Section 4.10.4). Applying one consistent factor across all energy components is what makes the recorded `co2eq_kg` reversible back to energy in Exp. 4 and what yields an identical effective intensity in the lifecycle analysis of Exp. 5. The original CodeCarbon total is retained as `co2eq_codecarbon_kg` for reference.

Limitations. This approach does come with limitations. Because `LibreHardwareMonitor` requires privileged access to hardware sensors, all measurements in this study were conducted with the process running under administrator privileges. Without elevated rights, the `CPUPowerMonitor` fails to retrieve sensor data and the corrected emissions estimate cannot be computed. Beyond the access requirement, the approach is inherently platform-specific. LibreHardwareMonitor is a Windows-only solution, adopted here precisely because Intel RAPL, the interface CodeCarbon uses for direct CPU energy readings on Linux, is not accessible on Windows. The hardware-corrected emissions values reported in this study are therefore tied to this Windows-specific stack and cannot be reproduced directly on other operating systems.

A secondary limitation is that CodeCarbon measures at the process level, meaning background system activity on Windows 11 contributes to the recorded energy. This effect is expected to be small relative to the dominant training workload but cannot be fully eliminated.

A final structural limitation concerns CPU parallelization asymmetry: LR's single-threaded SAG solver operates at roughly 10 % CPU utilization on this system, while RF and XGB saturate all eight cores, causing LR's emissions to overstate its true algorithmic cost. The full analysis of this asymmetry and its consequences for cross-model comparisons are discussed in Section 6.3.

4.8 Inference Latency Measurement

Inference latency was measured separately after training, outside the emissions tracker. For each model, a single row from the dataset was passed through the trained model repeatedly and the elapsed time was captured using `time.perf_counter()`, yielding a hardware-level estimate of per-sample prediction latency.

Single-Row Design. Only a single row was passed to the model rather than a full batch. This was intentional: batching would measure throughput rather than the actual time it takes to process one request, which is the more relevant figure in deployment. A single-prediction latency can be extrapolated to real-world usage, where a model may handle millions of requests and the per-sample cost accumulates into a meaningful energy expenditure.

Measurement Procedure. The model used for this measurement is the pipeline fitted on `X_train` during the main training run, loaded from disk via `joblib`. A single row from `X_test` is used as the inference input. One warmup call is issued and discarded before the monitor starts: by running it outside the measurement window, neither the latency artifacts from Just-In-Time (JIT) compilation and cache misses nor the associated energy spike are included in the reading. A `CPUPowerMonitor` is then started and a `while` loop driven by `time.perf_counter()` calls `model.predict()` repeatedly for a fixed 30-second budget. Each iteration is individually timed and its elapsed time appended to a list. Once the budget expires, the monitor is stopped, returning the total window energy and the count n of predictions completed. The median of the recorded per-prediction timings serves as the inference latency estimate, providing robustness against transient system load spikes.

Time Window Rationale. A fixed time window rather than a fixed repetition count was chosen deliberately to ensure reliable CPU energy measurement across all models. The `CPUPowerMonitor` polls CPU package power every 0.25 seconds. Models with very short per-prediction latency would complete a fixed-count loop in well under one polling interval, yielding zero power samples and making any energy estimate meaningless. A 30-second window guarantees at least 120 power samples for every model regardless of per-prediction speed, producing a statistically stable reading across the full range of model families.

Per-Prediction Energy. The total CPU energy accumulated over the 30-second window is divided by the number of predictions made during that interval to obtain `energy_per_inference_wh`: the directly measured energy cost per single prediction, expressed in Wh rather than kWh because per-prediction values fall several orders of magnitude below 1 kWh. This is more principled than multiplying a separately recorded latency by a separately recorded power figure, as both quantities are derived from the same physical measurement

window. Since training involves gradient computation, backward passes and tree-building whereas inference requires only a forward pass, inference-phase CPU power is substantially lower than training-phase power. Using training power as a proxy would overestimate per-prediction energy and undercount the break-even threshold. The direct measurement avoids this distortion. One caveat applies at the cheapest end of the model range: for predictors with sub-microsecond compute such as LR, the measured per-prediction energy is dominated by harness overhead, the `while` loop, the `perf_counter` calls and the scikit-learn pipeline dispatch, rather than by the model arithmetic itself. The reported figure for these models should therefore be read as an upper bound on the true inference cost, which is conservative for the break-even analysis since it shifts the crossover towards inference rather than flattering it.

Break-even Derivation. These two quantities, `energy_per_inference_wh` and the training emissions C_{train} (in gCO_2eq, taken from `results.csv`), feed into a novel lifecycle break-even analysis (`run_lifecycle_analysis.py`), introduced in this thesis to bridge the gap between one-time training cost and cumulative deployment cost. The core idea is to estimate at what prediction volume the growing inference footprint overtakes the fixed training footprint, yielding a concrete, dataset-specific break-even count for each model. The emissions per single prediction, $C_{\text{inference}}$, are:

$$C_{\text{inference}} = \frac{\texttt{energy\_per\_inference\_wh}}{1000} \times I_C \tag{4.2}$$

where the division by 1000 converts Wh to kWh and I_C is the static 381 $\frac{gCO_2eq}{kWh}$ factor that CodeCarbon applies for the German grid (Section 4.7), so that $C_{\text{inference}}$ is expressed in gCO_2eq. The break-even prediction count follows directly as

$$N_{\text{break-even}} = \frac{C_{\text{train}}}{C_{\text{inference}}} \tag{4.3}$$

The full methodology is discussed in Section 4.10.5.

4.9 Results Storage and Orchestration

Training Results. The results of every model run are written to `results/results.csv`, which serves as the central data source for all subsequent analysis and visualisation. Each row represents one complete model-dataset combination and is appended to the file rather than overwriting it, so that results from multiple runs accumulate without data loss. The file is managed using Python's `csv.DictWriter`, which writes a header row on first creation and appends subsequent rows without repeating the header. A consequence of the append-only design is that re-running a configuration produces duplicate rows, which must be filtered during analysis. Table 4.10 lists all columns.

The primary emissions value is `co2eq_kg`, which combines the hardware-sensor CPU reading with CodeCarbon's GPU and RAM estimates as described in Section 4.7. The column `co2eq_codecarbon_kg` retains the original CodeCarbon total for reference. The difference between the two columns is examined in Chapter 5.

Table 4.10: Column schema of `results.csv` including component-level energy tracking.

Column	Format	Description
`timestamp`	datetime	Date and time of the run
`model`	string	Model name
`dataset`	string	Dataset name
`nrows`	int / `all`	Subset size (`all` for the full dataset)
`f1`	float	Test WF1
`co2eq_kg`	float	Corrected CO_2eq (Hardware CPU + CC GPU/RAM)
`co2eq_codecarbon_kg`	float	Original CC estimate (static 381 g/kWh factor)
`cpu_power_hw_w`	float	Average CPU package power (W)
`cpu_energy_hw_wh`	float	Total CPU energy (Wh)
`gpu_energy_wh`	float	Total GPU energy from CodeCarbon (Wh)
`ram_energy_wh`	float	Total RAM energy from CodeCarbon (Wh)
`training_time_s`	float	Total training wall-clock time (s)

Inference Results. Inference latency is stored separately in `results/inference_time.csv`, as it is measured outside the emissions tracker and has a different structure. Like `results.csv`, it uses append-only writes via `csv.DictWriter`. Table 4.11 shows its columns.

Table 4.11: Column schema of `inference_time.csv`.

Column	Format	Description
`timestamp`	datetime	Date and time of the run
`model`	string	Model name
`dataset`	string	Dataset name
`nrows`	int / `all`	Subset size (`all` for the full dataset)
`inference_time`	float	Median single-row prediction latency (s)
`co2eq_kg`	float	Corrected CO_2eq of the training run
`cpu_power_inference_w`	float	Average CPU package power during 30 s inference window (W)
`energy_per_inference_wh`	float	Directly measured CPU energy per prediction (Wh)
`n_inference_reps`	int	Number of predictions made during the 30 s window

Orchestration Scripts. Five core orchestration scripts coordinate the data collection and analysis phases of the experiment. Each model script is launched sequentially as an isolated subprocess rather than being imported directly. This is important because CodeCarbon operates at the Python process level: running multiple trackers within the same process risks state leakage between measurements. Subprocess isolation ensures that each tracker starts and stops cleanly in a fresh runtime environment. However, as noted in Section 4.7, the hardware sensors still measure whole-system power draw during this isolated execution, meaning Windows background activity remains part of the final recorded footprint.

`run_all.py` is the main entry point for the full cross-dataset benchmark. It runs in two sequential phases. Phase 1 executes all tuning scripts for RF, XGB and MLP across their respective datasets, saving the best parameters to `best_params.json`. Phase 2 then runs all five model scripts across all three datasets using those saved parameters (except RF on the HIGGS dataset as mentioned earlier).

The remaining four scripts handle specific experiments and post-processing. `run_scaling_multiseed.py` trains all models across nine HIGGS subset sizes under three independent

random seeds (42, 123 and 999), using a fixed held-out test set and training subsets drawn from the remaining train pool. The full methodology is described in Section 4.10.2. The same runs cover both the scaling analysis and the CPU/GPU XGB comparison, so no separate breakeven script is needed. `run_mlp_variation.py` runs the MLP architecture grid experiment. `run_inference_power.py` re-measures inference latency and CPU package power for each model–dataset combination using a dedicated 30-second measurement window, keeping inference measurements separate from the training emissions tracker. Finally, `run_lifecycle_analysis.py` computes the lifecycle break-even analysis from the collected training and inference data, with the methodology described in Section 4.10.5.

4.10 Experiments

The research questions are operationalized through five experiments, each isolating a specific dimension of the ecological-performance trade-off. **Exp. 1** trains all five models on all three datasets, forming the empirical backbone of the study and addressing the overall efficiency trade-off across model complexity and dataset scale (RQ 1). **Exp. 2** holds all variables constant except HIGGS training size, using a three-seed replication strategy to attribute observed differences exclusively to data volume (RQ 4). **Exp. 3** trains a systematic grid of MLP architectures on HIGGS to isolate the effect of depth and width on emissions and performance (RQ 2). **Exp. 4** is purely retrospective, re-weighting recorded training energy against one year of hourly German grid data to bound the uncertainty in CodeCarbon's static intensity factor. Unlike the other four, it is not tied to a specific research question. Instead it validates the emissions measurement that underpins all of them, quantifying how far the reported figures could shift under realistic timing rather than answering a question about the models themselves. **Exp. 5** combines the training footprint with per-prediction inference latency to estimate at what prediction volume cumulative inference emissions surpass training emissions (RQ 5).

4.10.1 Main Cross-Dataset Benchmark

The main cross-dataset benchmark (Exp. 1) trains all five models on all three datasets (14 model–dataset combinations, with RF on HIGGS excluded as described in Section 4.2), directly addressing RQ 1. All combinations are evaluated under identical conditions: dataset-specific hyperparameters from the Optuna tuning runs, a stratified 80/20 train–test split with a fixed random state and the corrected emissions measurement pipeline described in Section 4.7.

For each of the 14 runs, WF1 is computed on the held-out test set, while emissions and training time are recorded as totals over the single training run. Inference latency is measured separately on a single held-out sample after training. From these values, the CF1 is computed during analysis and serves as the primary composite metric for comparing the performance-to-cost ratio across models and datasets.

The benchmark is designed to support comparison along two axes simultaneously. On the model complexity axis, ranging from LR through the tree ensembles to MLP, it exposes

at what point additional complexity stops producing performance gains that justify the associated energy cost. On the dataset scale axis, spanning three orders of magnitude from Wine to HIGGS, it reveals how the relative standing of models shifts as training data grows. The inclusion of both XGB CPU and XGB GPU as separate entries also directly feeds into the CPU versus GPU efficiency comparison developed in the scaling experiment.

4.10.2 Dataset Scaling Experiment

Scope. Exp. 2 is restricted to HIGGS, the only dataset large enough to make this experiment meaningful: with 11 million instances, it can be subsampled across a wide enough range for the scaling behavior to become visible. Wine and Credit are too small to show any trend across multiple subset sizes.

Split. The experiment covers nine HIGGS subset sizes: 1,000, 10,000, 50,000, 100,000, 200,000, 500,000, 1,000,000, 5,000,000 and the full train pool of approximately 8,800,000 instances. The range starts at 1,000 rows to capture the low-data regime where models may not yet converge and where resource costs are dominated by fixed overhead rather than computation. The upper endpoint is the train pool rather than the complete 11-million-instance dataset, because 20 % of HIGGS is reserved as a fixed held-out test set, as described below. All five models are trained at the six smaller subset sizes (up to 500,000 instances). RF is excluded from the three larger sizes (1,000,000 and above) because training times at that scale become prohibitive, consistent with its exclusion from the full HIGGS benchmark. Including it at the smaller subsets provides a direct view of where its scalability breaks down relative to the gradient-boosting methods, yielding 42 model–subset combinations in total. Subsets are drawn via stratified sampling to preserve the original class distribution across all sizes.

Random Seed. To obtain stable and reproducible estimates at each subset size, every model–subset combination is trained under three independent random seeds: 42, 123 and 999. Before any subset is drawn, HIGGS is partitioned once into a fixed 80 % train pool (approximately 8,800,000 rows) and a fixed 20 % held-out test set (approximately 2,200,000 rows) using a single deterministic split with random state 42. This partition is shared across all seeds and all subset sizes: the test set never changes. For each seed and each subset size, a stratified subsample of the requested size is drawn exclusively from the train pool. This design ensures that the WF1 estimate at every point on the scaling curve is evaluated against the same held-out population, eliminating the confound that would arise if both training data and test data changed simultaneously with each subset size. Model initialization, weight randomization and data shuffling are all seeded identically per run. The reported emissions, training time and WF1 values at each subset size are the means across the three runs. This replication strategy averages out subsample-specific noise that can otherwise produce non-monotonic artefacts in the scaling curves when a single draw happens to yield an atypical class distribution or feature density at a particular size.

Tuning Limitation. The hyperparameters used at every subset size are those obtained from tuning on the full HIGGS dataset and are not re-optimized per subset. Re-tuning at each size would conflate scale effects with configuration effects and make it impossible to attribute

observed differences to dataset size alone. The practical consequence is that at smaller subsets the hyperparameters are over-specified for the data, so performance figures at 1,000 or 10,000 instances reflect a model configured for 11 million rows, not the best achievable result at that size. The direction of this over-specification is conservative relative to the study's primary claims: at small scales, reported WF1 figures are lower bounds on what a properly tuned model could achieve. The experiment therefore neither overstates the performance advantage of larger data nor artificially flatters models at intermediate scales. The energy and emissions trends, which are the central focus of the analysis, are unaffected by this choice, since they reflect the cost of executing the fixed configuration and are not confounded by any tuning-induced variation.

Measurement. For each combination, emissions, training time and WF1 are recorded using the same measurement pipeline as the main benchmark. Because XGB is the only model present in both a CPU and a GPU variant, the scaling data also directly supports the CPU versus GPU comparison: it shows at which dataset size the GPU's higher power draw starts to be offset by its faster training time, making the energy crossover visible as part of the broader scaling picture rather than as an isolated finding. Exp. 2 thus directly addresses RQ 4 and feeds into RQ 3.

4.10.3 MLP Architecture Variation

Structure. Exp. 3 trains the MLP on HIGGS across a systematic grid of 12 architectures: three widths (128, 256 and 512 neurons per layer) crossed with four depths (one, two, three and four hidden layers). Each layer in a given configuration uses the same width, which keeps the design space tractable and ensures the depth and width axes can be interpreted independently of one another. The full grid is shown in Table 4.12 together with the resulting parameter counts on HIGGS (28 input features, 2 output classes), which range across more than two orders of magnitude.

Table 4.12: MLP architectures evaluated in the variation experiment, with trainable parameter counts for HIGGS. Counts include linear layer weights and biases as well as Batch Normalization scale and shift parameters, matching the convention of Table 4.6.

Depth	Width 128	Width 256	Width 512
1 layer	4,226	8,450	16,898
2 layers	20,994	74,754	280,578
3 layers	37,762	141,058	544,258
4 layers	54,530	207,362	807,938

Dataset. HIGGS is the only one of the three datasets where architectural variation has a meaningful effect to measure. Wine has too few instances to support a deep network at all and Credit is structurally simple enough that even LR achieves WF1 comparable to a tuned MLP, making any architecture a wash in performance terms. With 11 million instances and a non-trivial signal-versus-background classification task, HIGGS gives the network enough room to actually distinguish between configurations.

Tuning. The non-architectural hyperparameters (learning rate, dropout rate and batch size) are deliberately not taken from the HIGGS tuning results. The Optuna run optimized those values jointly with one specific architecture, so reusing them would silently favor that architecture against the rest of the grid. The experiment instead fixes neutral, architecture-independent defaults: a learning rate of 10^{-3} (the ADAM default from Kingma and Ba [40]), a dropout rate of 0.3 (a standard mid-range regularization value) and a batch size of 16,384 (chosen for computational efficiency). This choice may produce slightly suboptimal absolute results for any single configuration, but the relative comparison across the grid becomes considerably more defensible because no architecture starts with a hidden advantage from the tuning phase.

Training Procedure. Each of the 12 runs follows the same measurement pipeline as the main benchmark: a stratified 80/20 train–test split with the shared random state, emissions and training time tracked over the single training run, WF1 computed on the held-out test set and inference latency measured separately. The resulting data supports two orthogonal comparisons. Holding width constant while increasing depth reveals how layer stacking contributes to emissions and predictive performance. Holding depth constant while increasing width reveals the same for neuron count. Together, these two views directly address RQ 2: how energy consumption and CO_2eq output scale with NN complexity.

4.10.4 Temporal Carbon Intensity Analysis

Motivation. CodeCarbon converts measured energy into CO_2eq using a static carbon intensity factor of 381 $\frac{gCO_2eq}{kWh}$ for Germany, which is the default value used by CodeCarbon. This single number averages out two sources of variation that the literature has identified as significant for the German electricity grid: a strong diurnal cycle driven by solar production and a seasonal swing driven by the winter coal share versus summer renewable share [5]. The static factor is therefore convenient but unrepresentative of the actual emissions of any training run that was not coincidentally executed at the annual mean intensity. This analysis quantifies how much the reported emissions of this study would have differed under realistic timing.

Methodology. The experiment is purely retrospective and requires no retraining. Hourly carbon intensity data for the German bidding zone from the year 2024 is fetched from the ElectricityMaps API [67], covering 8,760 observations across all four seasons and the full diurnal cycle. The actual energy E in kWh consumed by every training run is derived from the recorded emission mass $m_{\mathrm{CO_2eq}}$ (the corrected emissions column `co2eq_kg`) by inverting the static factor. This inversion is exact because $m_{\mathrm{CO_2eq}}$ was constructed by applying that same carbon intensity of 381 $\frac{gCO_2eq}{kWh}$ to the corrected total energy, as described in Section 4.7.

Scenarios. Each run's energy is then re-multiplied by carbon intensity values drawn from the API data to produce counterfactual emissions under the eight scenarios defined in Table 4.13. These scenarios isolate three different sources of variation: the global average across the full year, the extremes of best- and worst-case timing and the structurally informed seasonal-diurnal combinations where renewables or fossil sources dominate.

Table 4.13: Carbon intensity scenarios used in the counterfactual analysis. Each scenario is computed from one year of hourly ElectricityMaps data for the German bidding zone.

Scenario	Definition
Static (CodeCarbon)	Fixed annual factor used by CodeCarbon, 381 g/kWh
Year mean	Mean over all 8,760 hourly observations
Year best hour	Single lowest-intensity hour of the year
Year worst hour	Single highest-intensity hour of the year
Winter mean	Mean over December, January, February
Summer mean	Mean over June, July, August
Summer midday	Mean of summer hours 11–14 UTC (peak solar)
Winter evening	Mean of winter hours 17–20 UTC (peak coal and gas)

Analysis questions. Exp. 4 evaluates two questions. First, how large is the gap between the static-factor estimate and the realistic best- and worst-case timing for the same workload. If the gap is small, CodeCarbon's static factor is defensible as a practical approximation. If the gap is large, the reported emissions of this study should be read as one point on a wide distribution rather than as a fixed value. Second, the analysis identifies whether the diurnal or the seasonal axis dominates the variation, which has direct implications for practitioners deciding when to schedule large training jobs on local infrastructure. The script outputs a heatmap of mean intensity by hour-of-day and month, plus a per-run counterfactual CSV with the recomputed emissions for every scenario.

4.10.5 Lifecycle Break-even Analysis

Motivation. Exp. 5 directly addresses RQ 5. In production, a deployed model is queried repeatedly and the cumulative inference footprint grows linearly with the number of predictions made. This experiment quantifies at what prediction volume it surpasses the fixed training footprint and how the break-even point differs across model families.

Approach. The analysis is purely post-hoc and requires no additional training runs. It combines two data sources already collected during the main benchmark: the corrected training CO_2eq from `results.csv` and the directly measured `energy_per_inference_wh` from `inference_time.csv`. Per-prediction CO_2eq is computed using Equation (4.2), introduced in Section 4.8, and the break-even prediction count follows as mentioned in Equation (4.3).

Fallback. When `energy_per_inference_wh` is unavailable, the script falls back to

$$\frac{t_{\text{inference}} \times P_{\text{infer}}}{3{,}600{,}000} \times I_C \tag{4.4}$$

where P_{infer} is the average CPU package power recorded during the same 30-second window.

5

Results and Discussion

The chapter is organized in nine sections that together address the five sub-questions set out in the introduction. The first section places the study's aggregate compute and emissions budget in context. The second establishes the hardware-corrected measurement baseline on which all subsequent comparisons rest. The third quantifies the tuning overhead that standard benchmark tables omit. The main analysis proceeds in four thematic blocks: the cross-dataset benchmark, the CPU–GPU hardware comparison, the HIGGS scaling and MLP architecture variation experiments, and finally the temporal and lifecycle analyses that extend the efficiency picture beyond training time alone.

5.1 Aggregate Experimental Footprint

Across 737 CodeCarbon-tracked runs conducted between March and June 2026, the experiments in this study consumed approximately 17.3 kWh of electricity and 255 hours of compute time, with CodeCarbon logging a cumulative 6,550 gCO_2eq, equivalent to driving approximately 55 km in an average German car or charging a smartphone roughly 860 times on the German grid, computed using a fleet-average factor of 118 gCO_2eq/km [68] and an assumed charge energy of 20 Wh at a grid intensity of 381 gCO_2eq/kWh. Given the systematic underestimation documented in Section 5.2, the corrected total is substantially higher. The 6,550 gCO_2eq should be read as a lower bound. Hyperparameter tuning for the MLP on HIGGS alone accounts for 4,325 gCO_2eq of that figure, approximately two thirds of the logged total, illustrating that the experimental overhead of finding a competitive configuration can dwarf the cost of the benchmark runs themselves.

The distribution of cost is highly uneven when viewed by run type rather than by count. The 40 tuning runs across all models and datasets represent just 5 % of all logged runs, yet are responsible for 69 % of the study's total emissions and 67 % of total compute time. The 14 model-dataset combinations that form the core benchmark, each measured as a single training run with already-optimized hyperparameters, account for only a small fraction of the ecological investment required to produce them. This asymmetry is not an anomaly of the experimental design but a structural feature of how ML research incurs cost: the visible artifact, the final trained model, represents only the tip of an iceberg whose bulk lies in the iterative search that precedes it.

5.2 Emissions Measurement Validation

Any comparison of model efficiencies is only as valid as the measurements it rests on. As described in Section 4.7, CodeCarbon's TDP-based CPU estimate was replaced in this study

by direct hardware-sensor readings. Figure 5.1 shows the ratio between the two for every model-dataset combination.

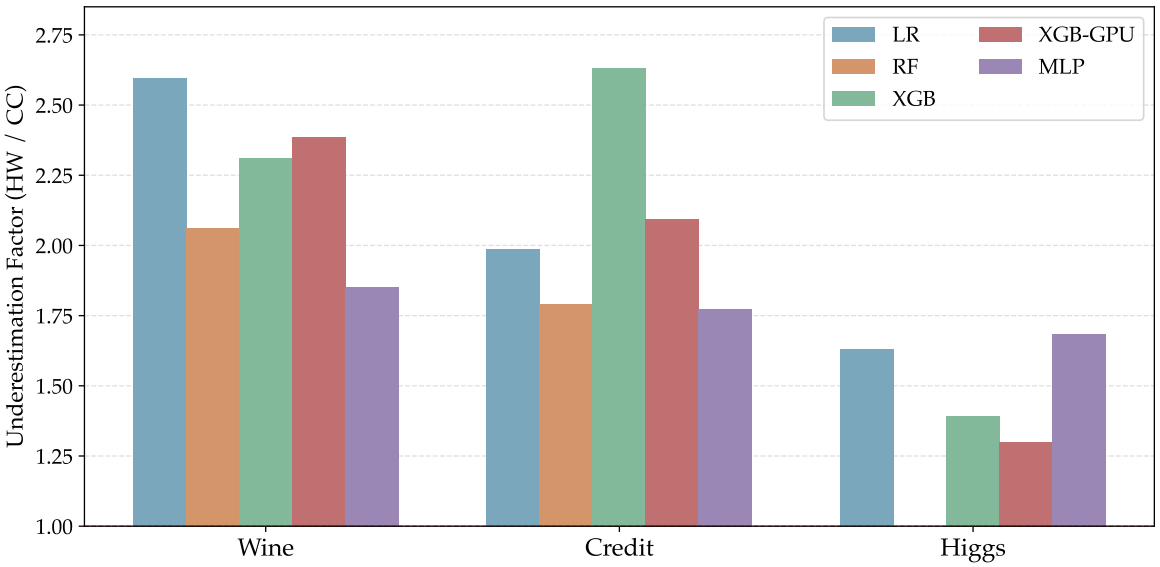


Figure 5.1: CodeCarbon underestimation factor (hardware-corrected / CodeCarbon) per model and dataset. All values exceed 1, confirming systematic undercounting. Higher factors for Wine and Credit reflect TDP-based estimation being least accurate on short training runs. RF has no HIGGS measurement.

Every one of the 14 model-dataset combinations shows a hardware-corrected value that exceeds the raw CodeCarbon recordings. The ratio ranges from 1.30 to 2.63 with a median of 1.92, meaning CodeCarbon, as deployed without modification, would have captured on average only about 52 % of the true emissions value. Fischer [7] found a consistent underestimation of 20 to 30% when validating CodeCarbon against external power meters across a range of workloads. The CPU-intensive runs in this study exceed even that range. LR on Wine reaches a ratio of 2.59, meaning CodeCarbon captured less than 39 % of the actual energy. The cause is the TDP fallback described in Section 4.7. Where RAPL is unavailable, CodeCarbon estimates CPU power from the thermal design envelope rather than measuring it and that envelope does not track how the chip actually behaves under load [16].

The error is not uniformly distributed, and two structural factors account for most of the variation. Short runs are hit hardest. Wine combinations, where training times range from 0.01 to 2.3 seconds, show a mean ratio of 2.24, while HIGGS combinations (16 to 4,754 seconds) land at 1.50. On a sub-second run the TDP-based integrator barely accumulates anything while the chip executes a real burst of work. As duration grows and load sustains, the heuristic becomes a closer approximation and the ratio shrinks. The GPU share is the second factor. XGB GPU on HIGGS, where the GPU accounts for 61 % of total energy and the CPU for only 30 %, produces the lowest ratio at 1.30. Since the correction touches only the CPU slice, a smaller CPU fraction produces a smaller total distortion even when the underlying power discrepancy is comparable to other runs. Fischer [7] document the same

asymmetry, finding that CPU-only workloads showed errors exceeding GPU equivalents by more than an order of magnitude in some configurations.

The measurement error is largest for the runs that correspond to the models Green AI positions as the efficient choice. LR on Wine carries a ratio of 2.59, XGB CPU on Credit 2.63. These are the models a practitioner would reach for to minimize ecological footprint, and they are precisely the ones where TDP-based estimation fails most severely. Without correction, they would appear even more energy-efficient relative to the heavier workloads than the corrected figures show, because the measurement error and the efficiency claim point in the same direction. The efficiency advantage of lightweight models is confirmed by the corrected data, but the gap is smaller than uncorrected measurements would suggest. All emissions values in the remainder of this chapter use the hardware-corrected figures. The relative rankings are unchanged, but the absolute scale is accurate. Published studies relying on uncorrected CodeCarbon values on comparable hardware should be read as reporting lower bounds rather than ground truth.

5.3 Hyperparameter Tuning Overhead

Benchmark tables that compare training emissions report only a fraction of a model's true ecological initialization cost. Before a single benchmark run is executed, each model must be brought to a competitive configuration through hyperparameter search and the footprint of that search is never reflected in the training figure. For computationally inexpensive models on small datasets this omission is negligible. For the MLP on HIGGS it is not: as shown in Figure 5.2, a deliberately constrained 40-trial TPE search required 72.3 hours and emitted 3,180 gCO_2eq, equivalent to approximately 27 km of car travel [68] or roughly 418 full smartphone charges on the German electricity grid, a figure that exceeds the combined training emissions of all Wine and Credit runs in this study. This is not the outcome of an exhaustive search of the kind documented by Strubell et al. [1], who found that large NLP development pipelines required hundreds of thousands of GPU hours to converge. It is the cost of the minimum credible search budget, chosen specifically to limit overhead.

Table 5.1: Hyperparameter tuning cost per model and dataset (40 Optuna TPE trials each). Duration is in hours. Emissions are hardware-corrected. Best WF1 is the highest achieved across all 40 trials. LR used sklearn defaults and was not tuned. RF was not tuned on HIGGS; literature defaults were used instead.

Dataset	Model	Duration (h)	Emissions (gCO_2eq)	Best WF1 (%)	CF1 ($mgCO_2eq$/WF1)
Wine	RF	0.02	0.88	97.9	9.0
	XGB	0.02	1.12	97.9	11.4
	MLP	**0.01**	**0.67**	**99.3**	**6.7**
Credit	RF	0.09	6.09	80.0	76.1
	XGB	**0.03**	**1.92**	80.2	**23.9**
	MLP	0.19	8.64	**80.3**	107.7
HIGGS	XGB	**0.67**	**54.26**	75.9	**714.7**
	MLP	72.33	3,182.5	**78.2**	40,682

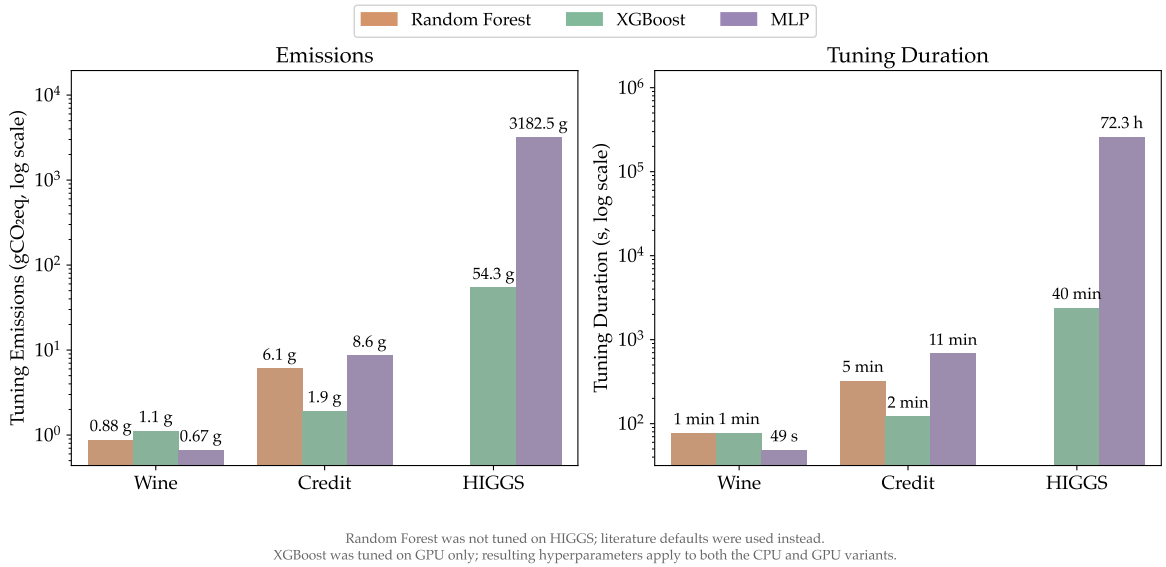


Figure 5.2: Emissions and duration of every Optuna tuning study (40 trials each). Bars show tuning duration. Emission totals are annotated to the right. RF was not tuned on HIGGS; literature defaults were used instead. XGB was tuned on GPU only; the resulting hyperparameters apply to both the CPU and GPU variants.

How expensive that search becomes is structurally determined by how well a model's training procedure maps onto available hardware during each trial evaluation. On the Credit dataset, this difference is already quantifiable. XGB completed its tuning study in 122 seconds and 1.92 gCO_2eq, accelerating each trial on GPU. RF arrived at a functionally identical result, a WF1 of 80.0 % against XGB's 80.2 %, in 326 seconds and 6.09 gCO_2eq: three times the tuning cost for a predictive difference that is statistically negligible. The gap reflects an architectural property that scales with search rather than just with training: constructing an ensemble of independent decision trees requires sequential or coarse-grained parallel computation that does not exploit GPU cores. Every trial inherits that cost and forty trials compound it. This is why RF was excluded from HIGGS tuning altogether, with literature defaults substituted instead. Subjecting that per-trial cost to eleven million rows across forty iterations would have been ecologically unjustifiable relative to any predictive gain it could have delivered, an assessment the scaling analysis in Section 5.6 corroborates.

The consequence for how the benchmark results in the following sections should be interpreted is direct. The training emissions in the benchmark table represent the cost of a single run with already-optimized hyperparameters. For the HIGGS MLP, the initialization cost that precedes that run amounts to 3,180 gCO_2eq against a training footprint of 58.5 gCO_2eq: tuning is roughly 54 times costlier than the benchmark run it makes possible. That debt does not reset at deployment. A nested CV approach for hyperparameter evaluation, as described in Section 4.5, would have multiplied these costs fivefold, adding weeks of additional compute on HIGGS alone. Whether the MLP's efficient inference architecture can serve enough predictions to offset that initialization cost over a realistic deployment lifetime is a question the lifecycle analysis in Section 5.9 addresses directly.

Translating tuning costs into the CF1 framework extends this picture beyond absolute emissions. Figure 5.3 reports the $\frac{mgCO_2eq}{WF1}$ achieved across each tuning study, using the best

WF1 reached over 40 trials as the denominator. On Wine, all three models are tuned at a comparable efficiency of 6–12 $\frac{mgCO_2eq}{WF1}$, with differences small enough to be practically irrelevant at this scale. On Credit, XGB separates from the field: its tuning CF1 is three times lower than RF's and more than four times lower than MLP's (Table 5.1). The gap reflects the same structural property that appears in training: XGB's histogram-based search is computationally light per trial, while tree-ensemble and gradient-descent-based searches carry proportionally higher per-trial costs. On HIGGS, the disparity becomes extreme. MLP's tuning CF1 is 57 times larger than XGB's, reflecting the combination of long per-trial evaluations on eleven million rows and a modest WF1 gain achieved over the full 40-trial budget. The tuning CF1 therefore reveals a cost axis that raw emissions alone obscure: whether the performance gains unlocked by a search budget are proportionate to its ecological price.

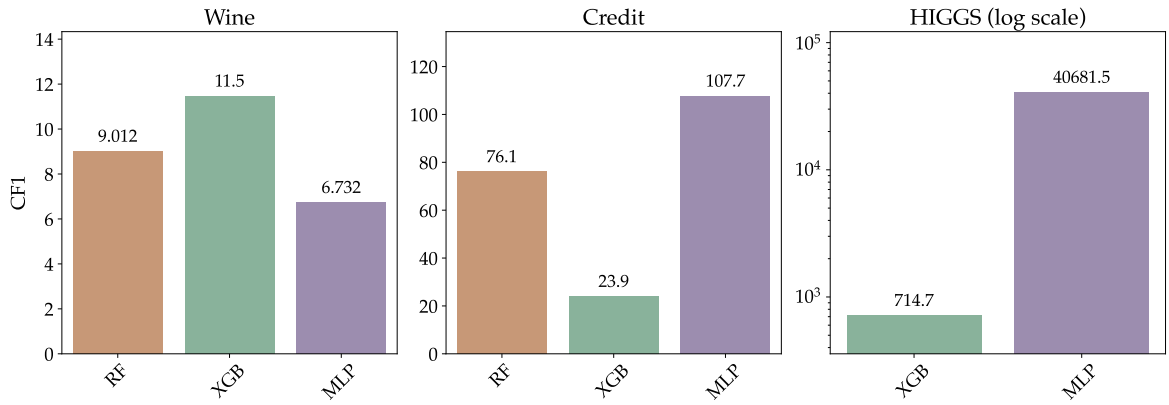


Figure 5.3: Tuning CF1 ($\frac{mgCO_2eq}{WF1}$) per tuning study, using the best WF1 across 40 trials as denominator. Lower is better. The HIGGS panel uses a log scale. RF was not tuned on HIGGS; literature defaults were used instead.

Table 5.2 makes the value proposition of hyperparameter tuning concrete by comparing the full initialization cost (HPO plus one training run) against simply training with framework defaults, restricting the comparison to HIGGS where the default configurations are defined as follows. For XGB, the library defaults are used directly: 100 estimators, maximum tree depth of 6, and a learning rate of 0.3. For the MLP, no authoritative library default exists because the implementation uses a custom `PyTorch` module rather than `scikit-learn`'s `MLPClassifier`. A minimal baseline was therefore constructed to approximate common framework conventions: Adam with a learning rate of 0.001 and an early stopping patience of 10, both matching `MLPClassifier`'s defaults, with two hidden layers of 100 units each. The unit count follows `MLPClassifier`'s default of `hidden_layer_sizes=(100,)`, and the depth is extended to two layers as the smallest configuration that constitutes a genuinely multi-layer network.

For both XGB variants the HPO study absorbs more than 95 % of the total initialization budget: 54.3 g out of a combined 57.1 g for the CPU variant. Against a default training cost of just 0.5 g, the 1.9 percentage point WF1 gain from tuning carries an additional 56.6 gCO_2eq, roughly 30 g per WF1 percentage point gained. For the MLP the structural picture differs in one direction. Poorly matched defaults extend convergence, so the untuned

model requires more epochs before early stopping triggers and produces a higher training cost (84.8 g) than the tuned variant (58.5 g). Better hyperparameters therefore directly reduce training energy as a side effect. Despite this training-side advantage, the HPO study itself costs 3,182.5 g, 54 times the tuned run and 37 times the default run, making the total initialization budget 3,241 g against 85 g: a factor of 38 for a WF1 gain of 1.6 percentage points. In all three models the HPO study absorbs more than 95 % of the initialization budget, making the benchmark training figures a systematically small fraction of each model's true ecological cost to produce. Whether the performance premium justifies the cost is deployment-dependent: for a system serving millions of queries the additional WF1 points may translate into a meaningful outcome improvement; for a low-volume application competitive performance is available at a fraction of the price by accepting library defaults.

Table 5.2: Tuned versus default hyperparameters on HIGGS (full 11 M rows). Total tuned combines the 40-trial Optuna study with one final training run on the tuned configuration. Default uses scikit-learn and XGB library defaults with no tuning. The XGB HPO study ran once on GPU; its cost applies to both variants. LR uses fixed defaults in both settings and is excluded; RF is excluded as it is not evaluated on HIGGS.

	Emissions (gCO_2eq)				WF1 (%)		
Model	HPO	Tuned train	Total tuned	Default	Tuned	Default	Δ
XGB (CPU)	54.3	2.763	57.1	0.469	76.0	74.1	+1.9
XGB (GPU)	54.3	0.393	54.7	0.123	76.0	74.1	+1.9
MLP	3,182.5	58.461	3,241.0	84.770	78.5	76.9	+1.6

5.4 Main Cross-Dataset Benchmark

Exp. 1 spans 14 model-dataset combinations across three datasets that differ by three orders of magnitude in size, directly addressing RQ 1. Figures 5.4 and 5.5 lay out the two dimensions of the comparison. The CF1 ranking synthesizes both dimensions. The central finding is that no model dominates across all three settings. Ecological efficiency is not a fixed property of any single model. It emerges from the alignment between a model's computational profile, the hardware it runs on and the scale of the task it faces. Each of the three datasets exposes a different axis of that contingency. The emission figures in the per-dataset tables below reflect a single training run with already-optimized hyperparameters. A consolidated overview of all 14 model-dataset combinations is provided in Table D.1 in the Appendix. The one-time initialization cost of producing those parameters and the WF1 premium they deliver over untuned defaults are quantified in Section 5.3.

The Low-Data Regime: When Hardware Overhead Dominates. Focusing first on the Wine dataset, the leftmost cluster in Figure 5.4 reveals that all five models converge on a WF1 of 97.2 %. With predictive differentiation absent, the entire comparison reduces to emissions, where Figure 5.5 shows a threefold spread across models that deliver the exact same outcome. The most striking observation within that spread is that XGB GPU emits 1.3 times as much gCO_2eq as XGB CPU for an identical result.

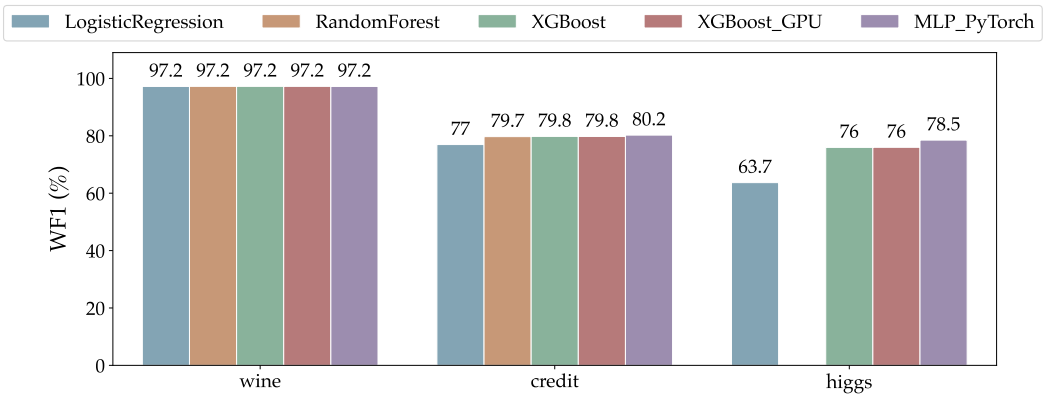


Figure 5.4: WF1 across all 14 model-dataset combinations. On Wine, all five models converge on a WF1 of 97.2 %, making emissions the only meaningful dimension of comparison. On HIGGS, LR saturates well below the non-linear models and MLP achieves the highest single result in the study.

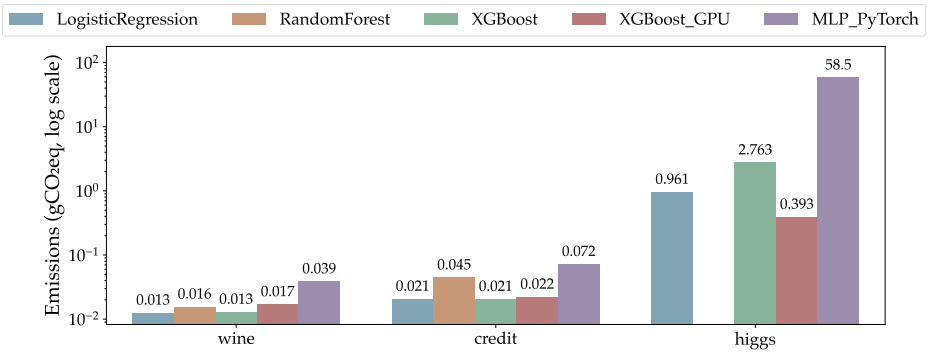
The mechanism behind this reversal is the fixed cost of GPU initialization and data movement. On a dataset of 178 rows, a Wine training run completes in fractions of a second. Device initialization and host-to-device memory transfers via the PCIe interface consume a fixed block of time and energy regardless of how little data follows. Consequently, the thousands of parallel cores the GPU provides remain almost entirely idle for the actual computation. The overhead dominates before the workload can even begin. This is the structural condition under which SIMT parallelism becomes a liability rather than an advantage: when data volume is too small to saturate the available multiprocessors, the architecture's fixed entry costs cannot be amortized.

The CF1 ranking (Table 5.3) surfaces this disproportionality directly. Since all five models achieve identical WF1, the ranking collapses to emissions alone: LR leads, while MLP costs three times as much for the same outcome. Applying complex and power-hungry hardware to a problem that a linear model solves equally well is not merely suboptimal but ecologically wasteful.

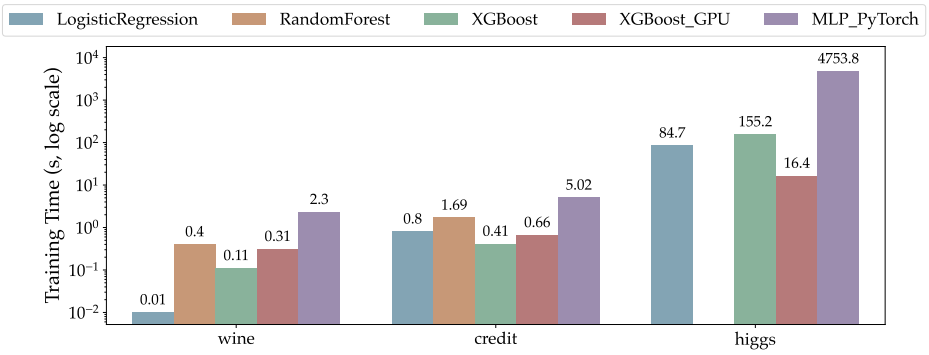
Table 5.3: Wine benchmark results for all models.

Dataset	Model	WF1 (%)	Emissions (gCO_2eq)	Time (s)	CF1 ($\frac{mgCO_2eq}{WF1}$)
	LR	97.2	**0.013**	**0.01**	**0.130**
	RF	97.2	0.016	0.4	0.161
Wine	XGB (CPU)	97.2	**0.013**	0.11	0.133
	XGB (GPU)	97.2	0.017	0.31	0.175
	MLP	97.2	0.039	2.3	0.399

The Medium-Data Regime: Structural Efficiency Disparities. Moving to the medium-scale Credit dataset (center cluster in Figure 5.4 and Figure 5.5), genuine predictive differentiation appears for the first time. As Table 5.4 shows, the critical data point is RF: it matches XGB CPU's WF1 to one decimal place while incurring more than twice the emissions cost. Two models, statistically indistinguishable predictive outcomes and a more-than-twofold difference in ecological cost.



(a) Hardware-corrected emissions (log scale).



(b) Training time (log scale).

Figure 5.5: Hardware-corrected emissions and training time (log scale) across all 14 model-dataset combinations.

The explanation is structural rather than incidental. RF constructs an ensemble of independently grown decision trees, evaluating candidate splits at each node across every tree in full. XGB's boosted tree construction, by contrast, uses highly-optimized histogram-based split-finding and grows trees sequentially, each correcting the residual errors of the last. This procedure achieves equivalent predictive capacity at a fraction of the computational cost. Crucially, this is the same structural property that inflated RF's tuning cost in Section 5.3: the per-tree overhead that makes tuning expensive makes training expensive for identical reasons and no amount of hyperparameter optimization removes it.

The CF1 column in Table 5.4 captures this disproportionality directly. XGB CPU leads despite near-identical WF1 to RF; LR ranks second even though it posts the lowest predictive score on this dataset, because its near-zero training cost outweighs the WF1 shortfall. MLP finishes last: a 0.4 percentage point lead over XGB CPU comes at 3.5 times the training emissions. The Credit ranking therefore illustrates two complementary efficiency failure modes: the structural overhead of tree-ensemble computation and the disproportionate carbon cost of marginal neural-network gains. When model families differ structurally in computational profile, choosing the right family is at least as consequential for ecological efficiency as any hardware decision [12].

Table 5.4: Credit Card Default benchmark results for all models.

Dataset	Model	WF1 (%)	Emissions (gCO_2eq)	Time (s)	CF1 ($\frac{mgCO_2eq}{WF1}$)
Credit	LR	77.0	**0.021**	0.8	0.269
	RF	79.7	0.045	1.7	0.560
	XGB (CPU)	79.8	**0.021**	**0.41**	**0.258**
	XGB (GPU)	79.8	0.022	0.66	0.279
	MLP	**80.2**	0.072	5.0	0.899

The High-Data Regime: Hardware Saturation and Role Reversals. Finally, as shown in the rightmost clusters of Figure 5.4 and Figure 5.5, both dimensions of the comparison shift decisively at eleven million rows. RF is excluded from this dataset for the reasons established in Section 5.3: the per-trial training cost made a full 40-trial HIGGS run ecologically unjustifiable. As Table 5.5 shows, LR saturates well below the non-linear models, while MLP achieves the highest WF1 in the entire benchmark at the highest single-run training cost in the study. Most strikingly, XGB GPU emits one seventh of what XGB CPU does for an equivalent outcome. On Wine, the GPU was a liability. On HIGGS, it is the ecologically dominant choice.

The reversal follows directly from the SIMT execution model discussed in Chapter 2. At the scale of eleven million rows, the data volume finally saturates the GPU's parallel cores. The fixed costs of device initialization and memory transfer that dominated the Wine result are now fully amortized across the dataset. XGB's histogram-based tree construction maps naturally onto this architecture, distributing split-finding workload across thousands of cores in a way that sequential CPU-bound computation cannot match. LR's ceiling, by contrast, is independent of hardware: a linear decision boundary cannot partition the HIGGS feature space with sufficient fidelity regardless of the computational effort invested. The MLP, meanwhile, requires iterative backpropagation passes over eleven million samples, driving its energy consumption to the highest single-run cost in the entire benchmark.

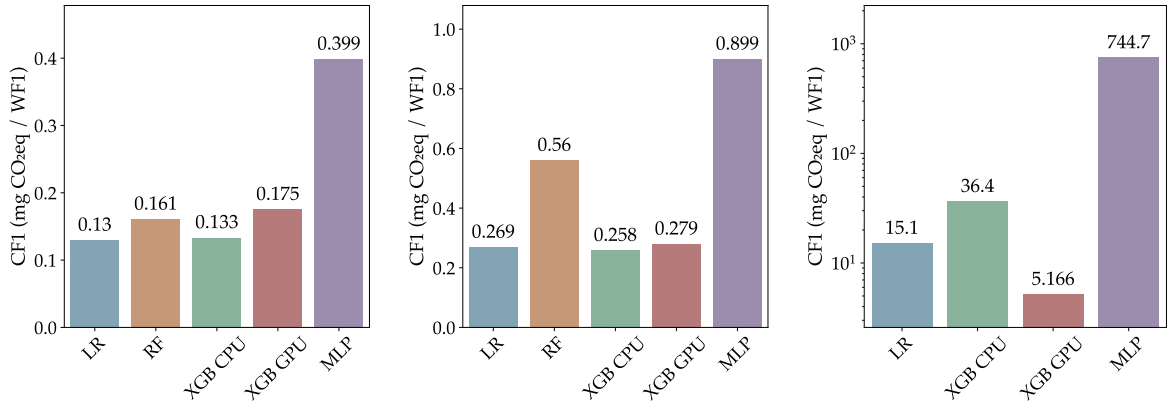
The CF1 column in Table 5.5 inverts relative to raw WF1. XGB GPU leads by a wide margin while MLP, despite holding the highest WF1 in the entire benchmark, finishes last. Its 2.5 percentage point predictive lead over XGB GPU cannot compensate for the order-of-magnitude difference in training emissions at this scale. Whether deployment-scale inference changes this assessment is the subject of Section 5.9.

Table 5.5: HIGGS benchmark results for all models.

Dataset	Model	WF1 (%)	Emissions (gCO_2eq)	Time (s)	CF1 ($\frac{mgCO_2eq}{WF1}$)
HIGGS	LR	63.7	0.961	84.7	15.09
	XGB (CPU)	76.0	2.763	155	36.38
	XGB (GPU)	76.0	**0.393**	**16.4**	**5.17**
	MLP	**78.5**	58.461	4,754	744.7

Synthesis: The Contingency of Ecological Efficiency No single model leads across all three datasets. LR wins on Wine, where a linear boundary suffices and hardware overhead dominates. XGB CPU wins on Credit, where the right model family delivers equivalent

performance at a fraction of the cost of its nearest competitor. XGB GPU wins on HIGGS, where dataset scale finally amortizes the fixed costs of GPU execution.



(a) Wine **(b)** Credit **(c)** HIGGS (log scale)

Figure 5.6: CF1 ($mgCO_2eq$ per WF1 percentage point) per model and dataset. Lower is better. The HIGGS panel uses a log scale. No single model leads across all three datasets.

As shown in Figure 5.6, the CF1 rankings shift substantially between datasets, and in each case the shift is mechanistically traceable: to hardware initialization overhead at small scale, to structural properties of the model family at medium scale and to SIMT saturation at large scale. At each scale, the metric responds to a different dominant cost factor, not just a reshuffled ranking. This is the empirical answer to the study's primary research question. Which model is ecologically efficient is always a function of computational profile, hardware and task scale, not a property of the model in isolation [21].

5.5 XGBoost: CPU vs. GPU

As established in the main benchmark, hardware acceleration is not unconditionally efficient. Figure 5.7 isolates the performance of XGB on CPU and GPU across all three datasets, revealing a contrast in ecological cost that shifts fundamentally with dataset scale.

On Wine and Credit, GPU acceleration provides no emissions benefit but rather an ecological liability. The GPU consistently emits more and trains slower than the CPU on both small datasets. This inversion occurs because the fixed overhead of device initialization and host-to-device memory transfer via the PCIe interface completely dominates actual computation time, as analyzed in Section 2.2.3. The thousands of available cores remain starved of data, leaving the GPU's higher power draw unamortized. On Credit the gap narrows to near parity, but GPU execution still fails to pay for itself.

The HIGGS result reverses the picture completely. The GPU variant is seven times more emissions-efficient and nine times faster, as roughly nine million training rows finally saturate the GPU's SIMT architecture [19]. The histogram-based tree construction distributes efficiently across thousands of cores, and the fixed memory transfer costs that dominated at small scale are now fully amortized by the parallelization gains.

To pinpoint where this shift occurs, Figure 5.8 maps the emissions crossover across intermediate HIGGS subsets from Exp. 2 (explored further in Section 5.6). The analysis

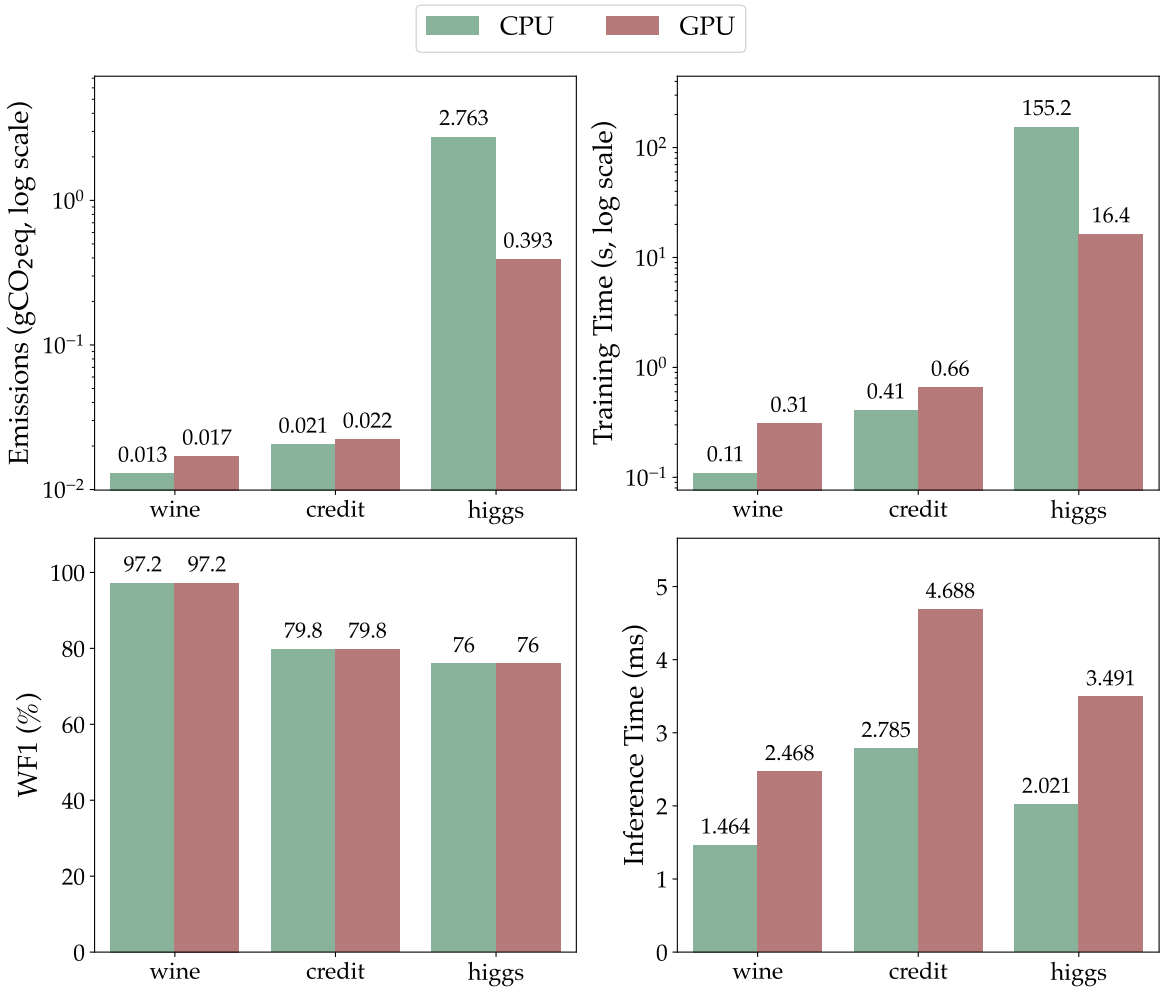


Figure 5.7: XGB CPU vs. GPU across datasets. Top row: Emissions in gCO_2eq and training time in seconds (log scale). Bottom row: WF1 and per-sample inference time. WF1 is identical across variants on all three datasets. The ecological and speed differences are the operative comparison.

identifies a hard break-even threshold at approximately 80,000 rows. Below this volume, the GPU's higher instantaneous power draw remains unamortized, transforming the accelerator into an ecological penalty. Above it, the compounding speedup of massive parallelization outpaces the wattage premium and the emissions gap widens favorably with each additional row.

This finding provides a direct, empirical answer to RQ 3: the ecological viability of GPU acceleration on tabular data is strictly bounded by dataset scale. More importantly, it challenges a pervasive assumption in modern machine learning. In practice, GPU execution is frequently treated as a universal default: a switch flipped to "accelerate" computation regardless of the task. The evidence here demonstrates that for tabular data, this default is fundamentally flawed. Blindly applying a GPU to a dataset of ten thousand rows does not accelerate learning. It merely burns energy to initialize a device and transfer data across a PCIe bus. True Green AI requires practitioners to actively break this habit and align their hardware choices with the complexity of their workloads, treating accelerators not as an unconditional default, but as a specialized tool justified only by sufficient scale.

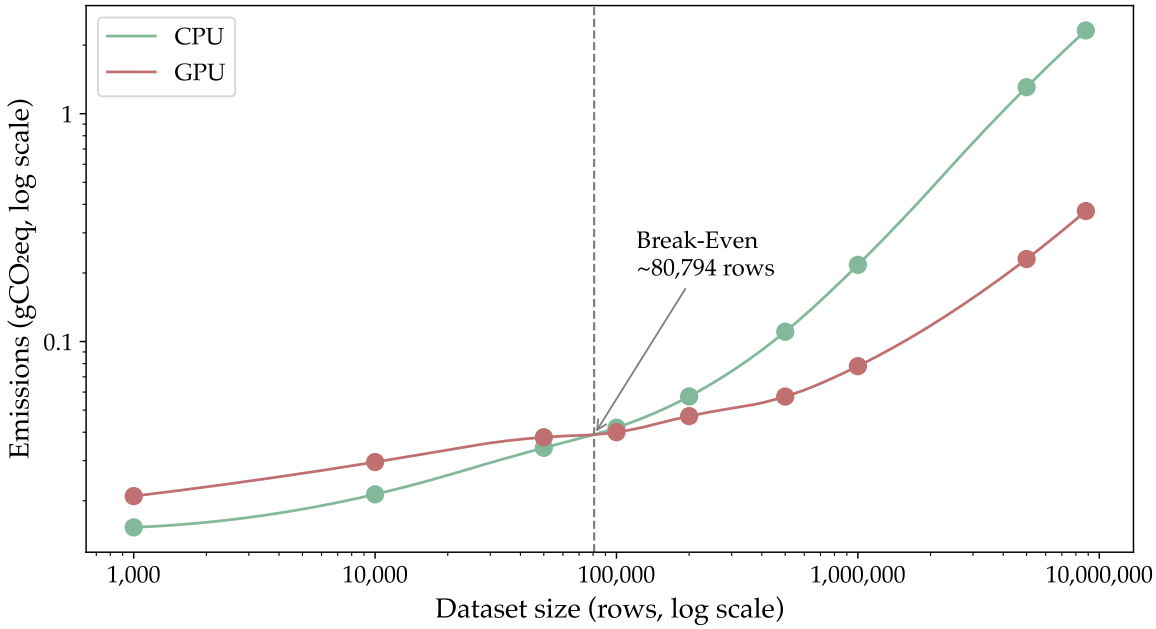
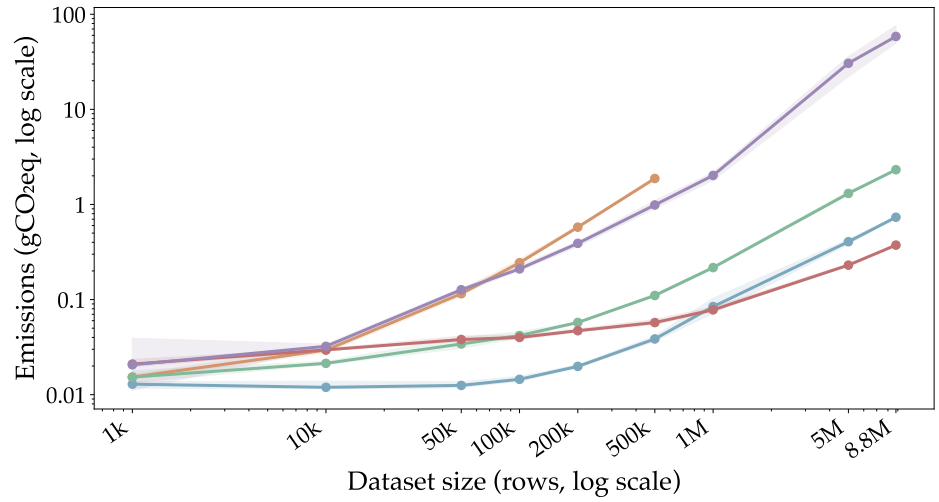


Figure 5.8: XGB break-even point comparison between different subset sizes of the HIGGS dataset.

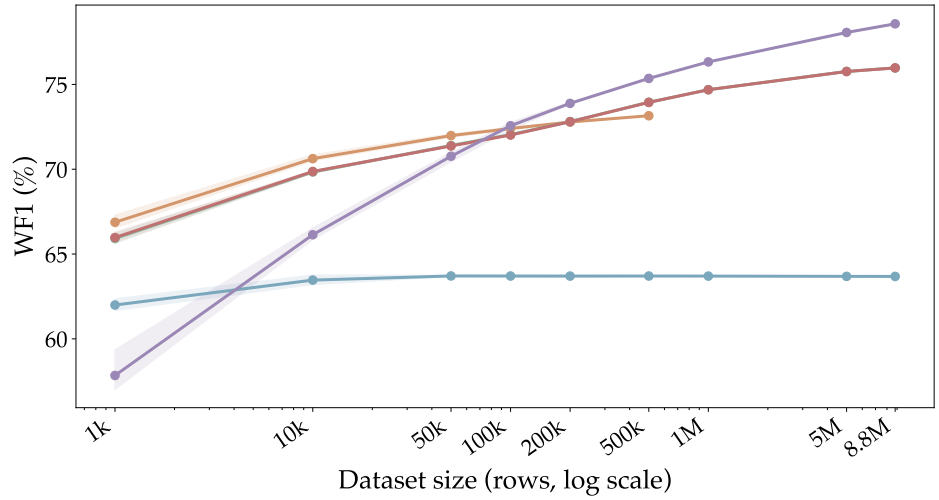
5.6 HIGGS Dataset Scaling

Exp. 2, directly addressing RQ 4, tracks emissions and predictive performance across nine HIGGS subsets ranging from 1,000 to 8,800,000 rows (Figure 5.9). Each data point is the mean across three independent random seeds. This replication averages out subsample-specific noise and yields monotonically well-behaved scaling curves. By holding the dataset complexity constant and isolating the volume of data, the scaling analysis exposes the exact thresholds at which the ecological cost of processing more data eclipses the predictive benefit.

Capacity Saturation in Simple Models At the lower end of the scaling curve, the linear boundary of LR hits a hard capacity ceiling remarkably early. The model's WF1 improves from 62.0 % at 1,000 rows to 63.7 % at 100,000 rows, but remains essentially flat across all subsequent, larger subsets. Because the model lacks the representational capacity to capture the non-linear intricacies of the HIGGS dataset, additional data yields virtually no predictive return. Consequently, any compute expended to train this model beyond 100,000 rows is ecologically wasted, demonstrating that data volume must always be matched to model capacity.



(a) Emissions (gCO_2eq, log scale)



(b) WF1

Figure 5.9: Emissions (log scale) and WF1 across nine HIGGS subsets for all models. Each point is the mean of three independent random-seed runs. RF excluded above 500,000 rows.

The Architectural Bottleneck of Ensembles As the dataset scales to 500,000 rows, a striking reversal in the expected cost of model complexity emerges. RF, despite its conceptual simplicity relative to deep learning, becomes the most ecologically expensive model in the study. At this scale, RF emits 1.88 gCO_2eq to achieve a WF1 of 73.2 %. In stark contrast, the MLP achieves a higher WF1 of 75.3 % while emitting just over half the CO_2eq (0.99 g) and XGB CPU achieves 74.0 % for just 0.11 g. RF is ultimately excluded from runs above 500,000 rows due to prohibitive training times. The underlying reason is architectural: training an ensemble of independent, fully-grown decision trees relies on coarse-grained parallelism that scales super-linearly with data volume and cannot easily exploit GPU acceleration (see Section 2.3.2). This proves that a model's ecological footprint is not determined by its abstract mathematical complexity, but by how efficiently its training procedure maps onto available hardware parallelism.

Diminishing Returns in the High-Data Regime In the high-data regime above 1,000,000 rows, all non-linear models face the fundamental tension of empirical learning curves (introduced in Section 2.4): predictive performance scales sub-linearly, while energy consumption scales linearly or worse. The observed WF1 curves across all models are qualitatively consistent with the concave power-law shape $P(n) \propto n^{\alpha}$ described there [46]: gains are steep at small subset sizes and flatten progressively as the dataset approaches its full scale. The MLP illustrates this trade-off most dramatically. Expanding its training data from 1 million to 8.8 million rows yields the largest absolute performance gain in this regime, improving the WF1 from 76.3 % to 78.6 %. However, this 2.2 percentage point improvement requires a 29-fold increase in emissions. Framed as a stopping decision, a practitioner who caps MLP training at one million rows instead of 8.8 million reduces emissions by 97 % while accepting a 2.9 % relative loss in WF1: a compelling case for treating dataset size as a tunable budget rather than a fixed input. Conversely, XGB GPU navigates this regime with extreme efficiency, reaching a WF1 of 76.0 % at 8.8 million rows while emitting only 0.37 gCO_2eq, roughly one sixth of its CPU counterpart (2.32 g).

Synthesis: Rethinking the Big Data Dogma Assessed through the CF1 lens, the diverse scaling patterns compress into a single, clarifying trend: as dataset size grows, the emissions gap between hardware-aligned models and inefficient models drastically widens. XGB GPU sustains the lowest CF1 across every scale above its hardware break-even point, proving that large-scale learning is only ecologically viable when software is perfectly aligned with SIMT acceleration. At five million rows it achieves a CF1 of 3.04 against XGB CPU's 17.25 at the same WF1 of 75.8 %, a gap that widens further at the full 8.8 million row dataset.

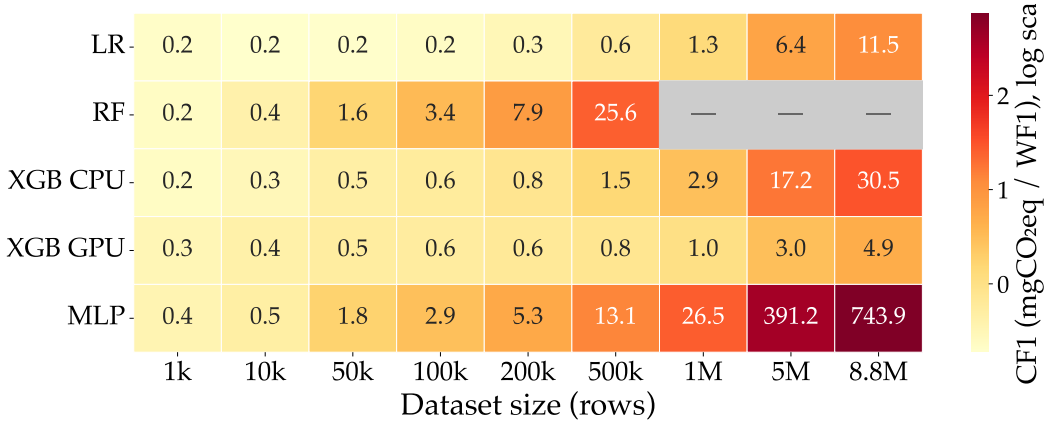


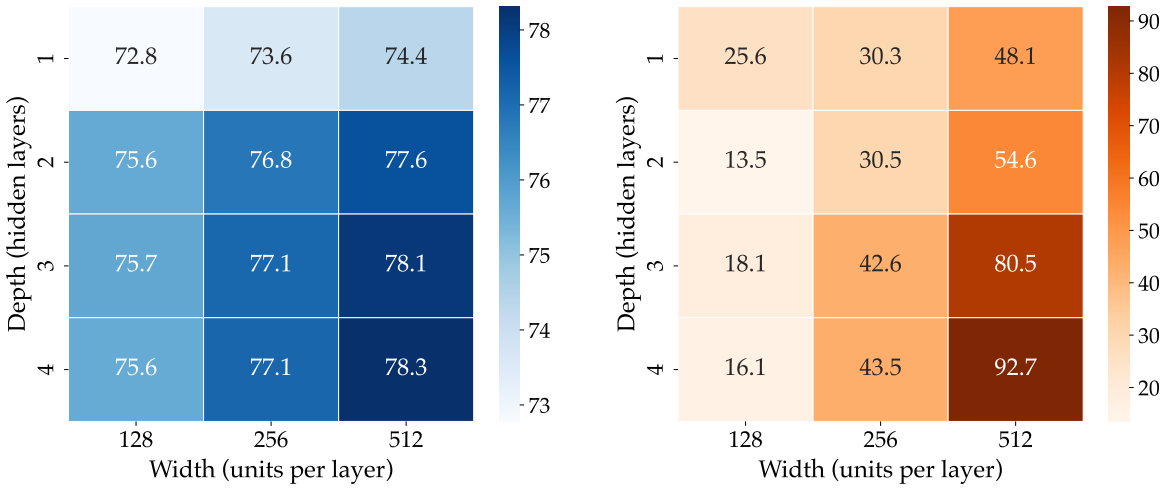
Figure 5.10: CF1 ($mgCO_2eq$ per WF1) for all model-size combinations. Colour encodes log-scaled CF1: yellow is most efficient, dark red is least. Cell annotations show raw CF1 values. Grey cells indicate runs not performed (RF excluded above 500,000 rows).

More profoundly, these scaling curves challenge the pervasive assumption that more data is unconditionally better, which drives modern machine learning. In the pursuit of state-of-the-art performance, the default industrial practice is to feed models the largest dataset available [2]. The empirical evidence here proves that this practice is often an ecological liability. At the tail end of a learning curve, forcing a model to ingest ten million additional rows to extract a marginal gain in WF1 results in a disproportionate explosion

of carbon emissions. True Green AI requires practitioners to abandon the assumption that all available data must be used. Instead, as recent data-centric AI studies argue, we must proactively identify the threshold where predictive returns diminish [12], treating dataset size not as a fixed asset, but as a hyperparameter that must be aggressively tuned to halt training before the ecological cost eclipses the scientific value.

5.7 MLP Architecture Variation

Exp. 3 directly addresses RQ 2 by training all twelve MLP configurations (depth $\in \{1, 2, 3, 4\}$, width $\in \{128, 256, 512\}$) on the full HIGGS dataset; Figure 5.11 shows the results.



(a) WF1 (%)

(b) Emissions (gCO_2eq)

Figure 5.11: MLP architecture variation on HIGGS for all depth-width combinations. Left: WF1, where darker cells indicate higher WF1 (better). Right: training emissions in gCO_2eq, where lighter cells indicate lower emissions (better).

The variation grid reveals that width has a substantially stronger effect on performance than depth. Moving from 128 to 512 units per layer increases WF1 by up to 2.7 percentage points across all depth levels, whereas adding layers beyond two yields marginal gains of at most 0.7 percentage points at the widest setting and near zero for narrower networks. The predictive ceiling is reached early: the 2×512 architecture achieves a WF1 of 77.6 %, trailing the best-observed configuration (4×512, WF1 = 78.3 %) by only 0.7 percentage points.

The emissions picture is more nuanced. For narrow networks (width = 128), emissions actually *decrease* as the network gets deeper, falling from 26 gCO_2eq at depth 1 to 18 gCO_2eq at depth 3. This reversal is mechanically driven by early stopping. Deep, narrow networks lack the representational capacity required for complex tabular features, causing them to rapidly overfit and trigger the early stopping patience criterion in fewer epochs. Consequently, the total compute per training run is lower despite the larger parameter count. For wider networks, this effect vanishes. At width = 512, emissions increase monotonically with depth from 48 gCO_2eq to 93 gCO_2eq, as the network successfully utilizes its capacity and trains for more epochs before saturating.

Taken together, these results highlight a structural mismatch between standard deep learning practice and tabular data. In computer vision or natural language processing, stacking layers is necessary to extract hierarchical spatial or sequential representations. Tabular datasets like HIGGS lack this strict local hierarchy, so excessive depth provides almost no predictive value and functions primarily as a computational overhead. The 4×512 configuration illustrates this directly: achieving the final 0.7 percentage point improvement in WF1 required scaling to maximum depth and width, nearly doubling the emission footprint from 55 gCO_2eq to 93 gCO_2eq for a gain well within normal run-to-run variance. Framed as an architecture selection decision, choosing 2×512 over 4×512 reduces emissions by 41 % while sacrificing less than one percentage point of WF1 in relative terms. The data here corroborate the broader Green AI observation that predictive returns on model scale follow diminishing returns, while energy costs do not [2].

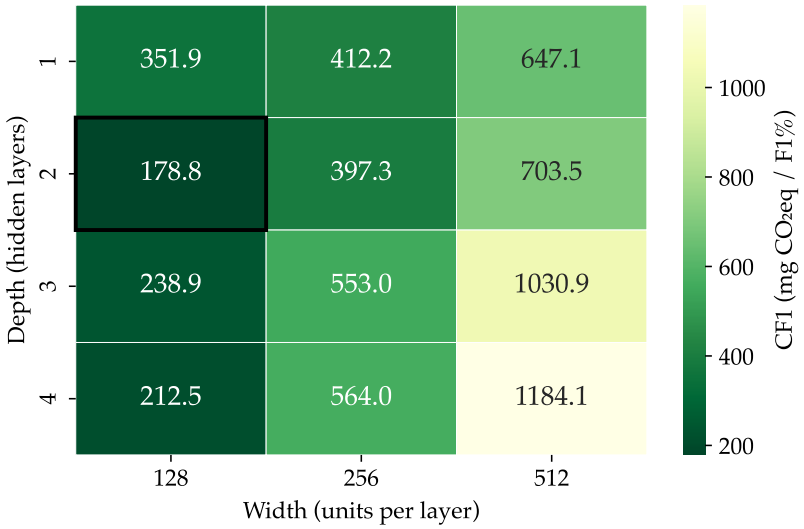


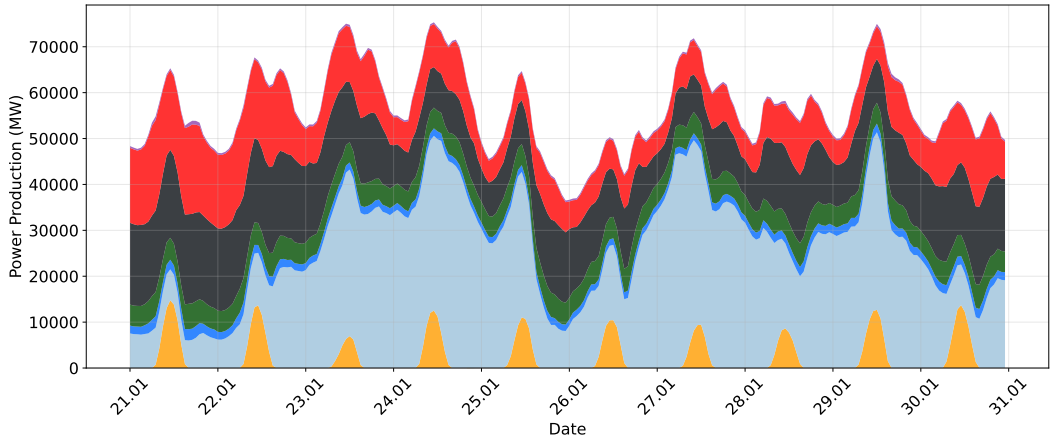
Figure 5.12: MLP architecture variation on HIGGS: CF1 for all depth-width combinations. Darker cells indicate lower (better) values.

Evaluating the twelve configurations through the CF1 lens, as seen in Figure 5.12 makes this trade-off explicit. Shallower, narrower architectures achieve lower CF1 scores: their modest performance reductions are more than offset by their lower training costs. The 2×128 configuration achieves the lowest CF1, entirely displacing the 4×512 configuration that posted the highest raw WF1 but pays a far higher carbon cost per WF1 point. For neural networks applied to tabular data, this suggests treating depth not as a default but as a resource justified only by proportional predictive returns.

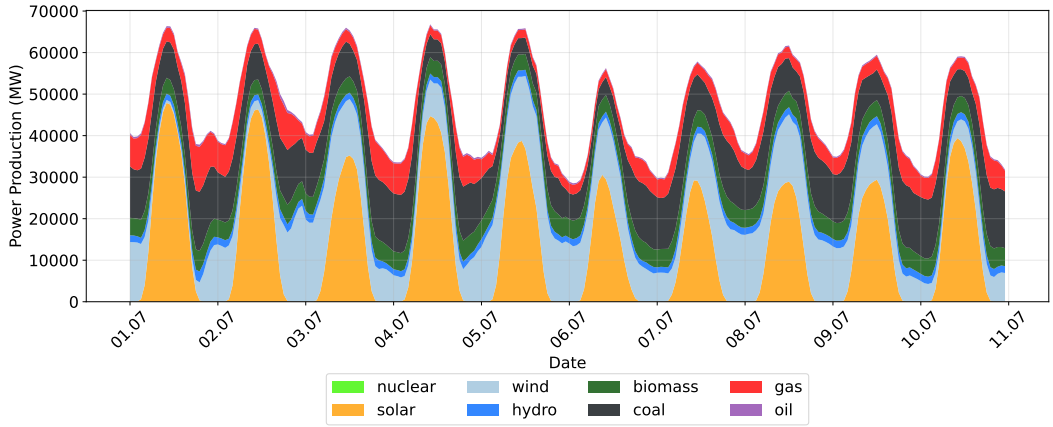
5.8 Carbon Intensity Analysis

The analyses conducted in the preceding sections treat emissions as a fixed function of the training run: given the same model, dataset and hardware, the reported figure does not change. This is a consequence of how CodeCarbon works: it converts measured energy into gCO_2eq using a static intensity factor of 381 $\frac{gCO_2eq}{kWh}$ for Germany, a calculated average that cannot reflect when a run actually takes place (see Section 2.2.5). The German grid,

however, draws on a fundamentally different fuel mix in January than in July and during peak evening demand than during solar-rich midday hours, a structure visible directly in the hourly ElectricityMaps data used in this analysis.



(a) January 2025 (21–30 Jan). Gas, coal and nuclear dominate, with solar generation near zero.



(b) July 2025 (1–10 Jul). Solar becomes the dominant midday source, sharply reducing the fossil fuel share during daylight hours.

Figure 5.13: Electricity production mix in Germany for representative ten-day windows in winter and summer. Each data point represents a one-hour average of power production in MW, sampled at hourly intervals and reported by ElectricityMaps. The legend shown in (b) applies to both panels.

Figures 5.13a and 5.13b illustrate this contrast directly. In January, dispatchable sources (gas, coal and nuclear) form the base of the stack, with wind providing a variable but significant contribution and solar generation remaining negligible throughout the day. In July, the picture reverses: solar becomes the dominant midday source, regularly exceeding 20,000 MW, while gas and coal drop to a much smaller share. It is this structural shift in fuel composition that drives the seasonal variation in carbon intensity quantified in the analysis below.

If this temporal variation is large enough, then the timing of a training run is itself a meaningful ecological decision, one that requires no changes to code, hardware, or model architecture. This section presents the results of Exp. 4, which evaluates that hypothesis by counterfactually re-assigning the energy of every training run to different timing scenarios derived from one year of hourly ElectricityMaps data, as described in Section 4.10.4.

Annual Spread and Scenario Intensities The first result is somewhat reassuring for the existing measurements: as shown in Figure 5.14, the static CodeCarbon factor of 381 $\frac{gCO_2eq}{kWh}$ is a defensible approximation of the annual mean of the year 2024, which the empirical data places at 341 $\frac{gCO_2eq}{kWh}$. A workload executed at a uniformly random point during the year would therefore be assigned only a modest systematic error by CodeCarbon. The more important finding is the spread around that average. The best recorded hour of 2024 reaches 115 $\frac{gCO_2eq}{kWh}$ and the worst reaches 658 $\frac{gCO_2eq}{kWh}$, a factor of 5.7× between two hours. The absolute extremes are not practically exploitable, since no practitioner can guarantee their run coincides with the single cleanest or dirtiest hour of the year. The structurally informed seasonal-diurnal combinations are the actionable range: summer midday hours average 193 $\frac{gCO_2eq}{kWh}$ while winter evening hours average 389 $\frac{gCO_2eq}{kWh}$, a factor of two achievable through deliberate scheduling alone.

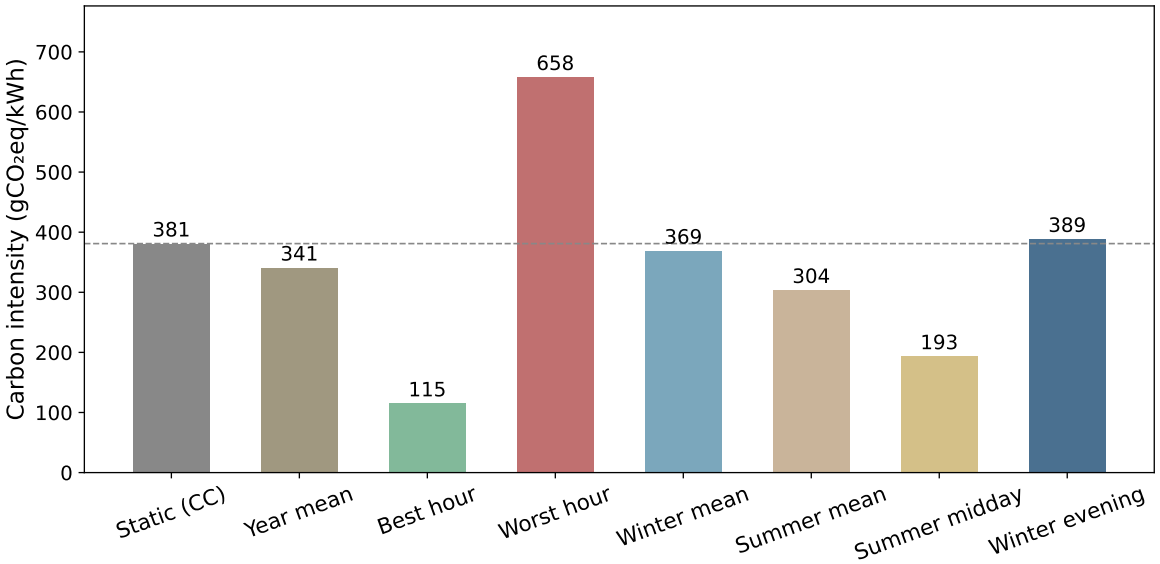


Figure 5.14: Mean grid carbon intensity in gCO_2eq/kWh for eight timing scenarios derived from one year of hourly ElectricityMaps data for Germany (2024). The dashed line marks the static CodeCarbon baseline of 381 gCO_2eq/kWh. Summer midday hours are roughly half as carbon-intensive as winter evening hours, a factor of two achievable through deliberate scheduling alone.

Per-Run Emissions Counterfactuals Because training energy is a fixed physical quantity for any given run, every percentage difference in grid intensity translates directly into the same percentage difference in emissions. Figure 5.15 expresses this proportionality relative to the static CodeCarbon baseline, making the stakes concrete. Scheduling during summer mean conditions would reduce the reported emissions of this study by approximately 20 % compared to the static estimate, requiring no modification to the experimental setup whatsoever. The best-hour scenario pushes this reduction to 70 %, while the worst-hour scenario reverses it into a 73 % increase. These percentages are identical for every model and dataset, because the intensity factor scales uniformly. What differs is the absolute magnitude of the effect. For lightweight runs such as LR on Wine, the difference between any two scenarios is negligible in absolute terms. For the MLP trained on HIGGS, the

gap between best-hour and worst-hour timing amounts to approximately 208 gCO_2eq, a quantity that exceeds the total training footprint of every Wine and Credit run in this study combined. This asymmetry reveals an important practical principle: grid-aware scheduling is most valuable precisely where model and hardware efficiency decisions also matter most, namely on large, compute-intensive workloads where the absolute energy expenditure is high enough for timing to make a meaningful difference.

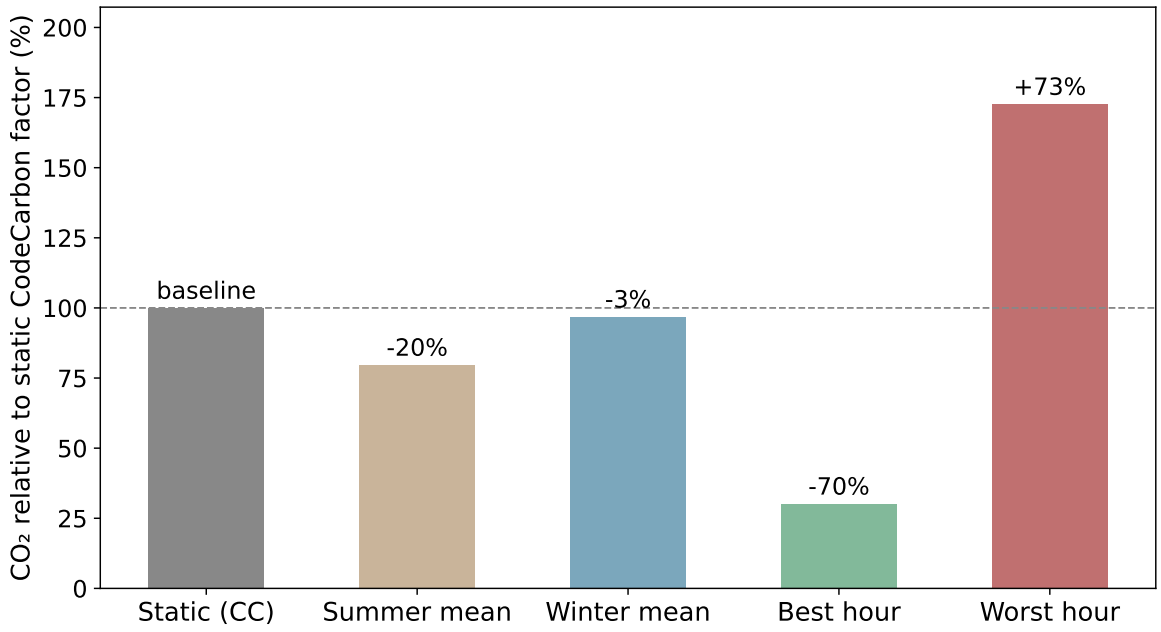


Figure 5.15: Counterfactual training emissions expressed as a percentage of the static CodeCarbon baseline (= 100 %). Because grid intensity scales uniformly across all runs, the bars are identical for every model and dataset. The dashed line marks the CodeCarbon reference; values below it represent emission reductions achievable through timing alone, without any change to code, hardware, or model architecture.

The Diurnal Dimension To understand why the diurnal dimension matters as much as the seasonal one, consider the solar duck curve visible in Figure 5.16 for 24 July 2024. Carbon Intensity (I_C) declines from roughly 300 $\frac{gCO_2eq}{kWh}$ in the morning as solar generation ramps up, reaches a minimum of 163 $\frac{gCO_2eq}{kWh}$ near midday, then recovers sharply to a peak of 461 $\frac{gCO_2eq}{kWh}$ once solar output falls in the evening. The static CodeCarbon factor of 381 $\frac{gCO_2eq}{kWh}$ intersects this curve only during the early morning and late evening hours, the two dirtiest windows of the day on a summer date with strong solar generation. During the core daylight hours, the static factor overestimates the actual intensity by up to a factor of two. This discrepancy empirically validates recent arguments that relying on generation-based average rates is inadequate for true consequential carbon accounting, which instead requires time-specific marginal emissions data [5]. This is not an anomaly specific to this particular date: it is the structural consequence of Germany's growing solar capacity, which consistently depresses midday intensity throughout the summer months. A practitioner who runs training jobs during office hours on summer days is inadvertently benefiting from this effect, while their CodeCarbon reports attribute a higher carbon cost than actually occurred.

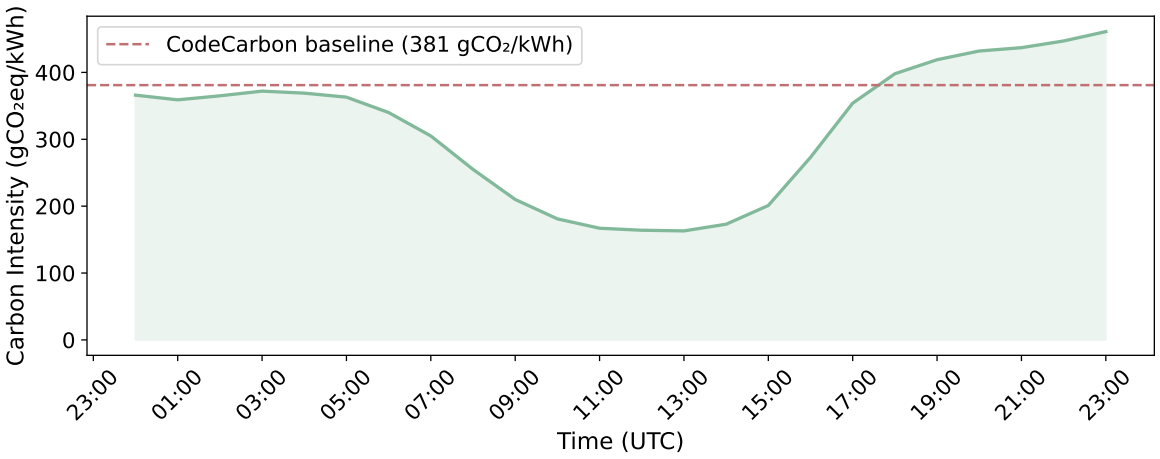
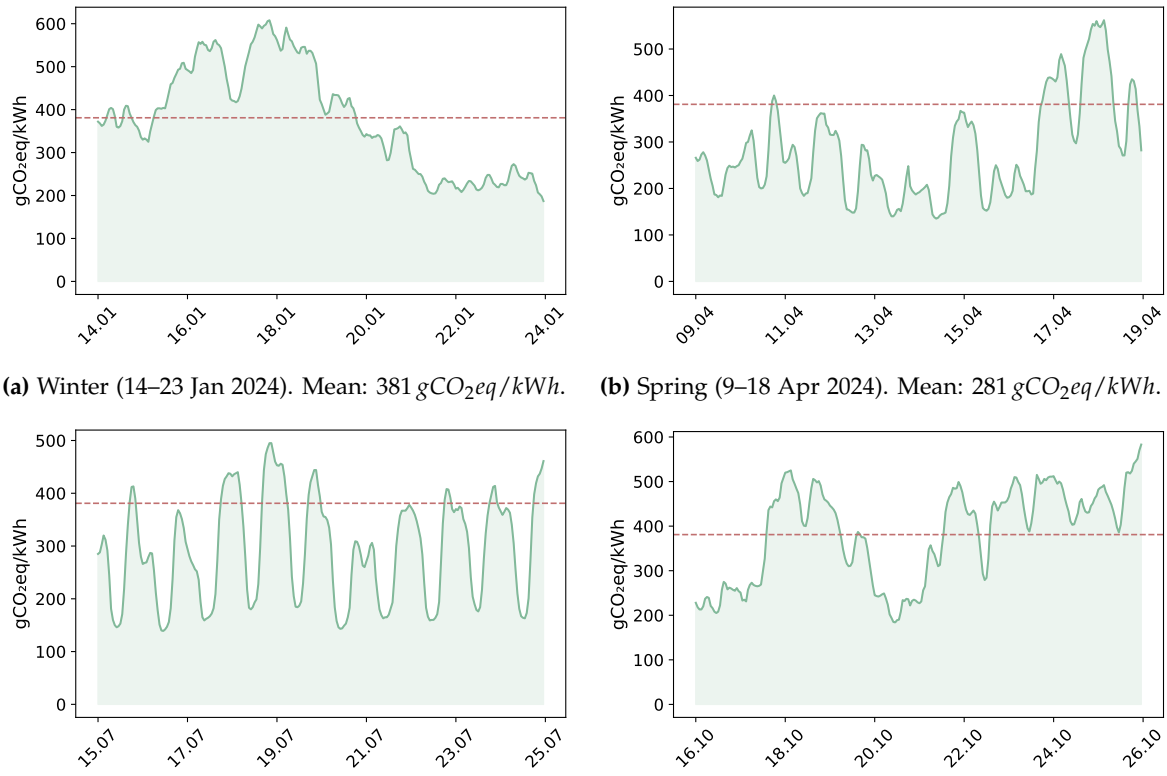


Figure 5.16: Hourly I_C in gCO_2eq/kWh for the German bidding zone on 24 July 2024, retrieved from the ElectricityMaps API (`/v3/carbon-intensity/past-range`). Each data point represents the I_C for one clock hour as computed by ElectricityMaps from the hourly generation mix. The dashed line marks the static CodeCarbon baseline of 381 gCO_2eq/kWh.

The Seasonal Dimension The seasonal dimension adds a second layer of variation on top of the diurnal one. The ten-day windows shown in Figure 5.17 reveal that winter and autumn both sit above the static CodeCarbon factor on average, meaning CodeCarbon systematically *underestimates* winter emissions rather than overestimating them. The winter window averages 474 $\frac{gCO_2eq}{kWh}$, nearly 25 % above the static factor, while spring (347 $\frac{gCO_2eq}{kWh}$) and summer (361 $\frac{gCO_2eq}{kWh}$) fall meaningfully below it. This directional asymmetry matters for transparency: the runs in this study were conducted during periods of varying grid intensity and the static factor provides no indication of whether a given run's reported footprint is an over- or underestimate. An equally important finding is that within-season variance is large enough to undermine coarse seasonal scheduling as a reliable strategy. The winter window alone spans from 311 to 614 $\frac{gCO_2eq}{kWh}$ across ten consecutive days, a range wider than the gap between winter and summer means. A practitioner who decides to schedule training in July rather than January improves their expected footprint, but cannot assume low intensity on any given July day without consulting real-time grid data.



(a) Winter (14–23 Jan 2024). Mean: 381 gCO_2eq/kWh.

(b) Spring (9–18 Apr 2024). Mean: 281 gCO_2eq/kWh.

(c) Summer (15–24 Jul 2024). Mean: 293 gCO_2eq/kWh.

(d) Autumn (16–25 Oct 2024). Mean: 384 gCO_2eq/kWh.

Figure 5.17: Hourly I_C in gCO_2eq/kWh for the German bidding zone across representative ten-day windows per season (2024), retrieved from the ElectricityMaps API (`/v3/carbon-intensity/past-range`). Each data point is the I_C for one clock hour. The dashed line marks the static CodeCarbon baseline of 381 gCO_2eq/kWh. Winter and autumn exceed the baseline on average; spring and summer fall below it.

Predictable Structure and Actionable Scheduling The year-long heatmap in Figure 5.18 brings both dimensions together and makes their joint structure immediately legible. The lowest-intensity region of the entire year is summer midday: a consistent band of green running from May through August between approximately 10:00 and 15:00, reflecting the hours when solar generation is both abundant and at its seasonal peak. The highest-intensity region is winter evenings: the band of red and orange concentrated between 17:00 and 21:00 from November through February, when solar output has already dropped to near zero and space heating drives gas and coal generation. What the heatmap makes clear is that these two extremes are not separated by chance. They are structurally determined by the interaction of Germany's solar build-out with the seasonal daylight cycle and heating demand patterns. A practitioner optimizing scheduling on local infrastructure is therefore not searching for a random low-intensity moment. They are navigating a predictable structure where the best and worst times are largely known in advance. The summer-midday to winter-evening gradient spans the full two-to-one ratio identified earlier and represents the maximum emissions reduction achievable through scheduling on the German grid without any hardware or model changes. Exploiting these predictable patterns

through elastic, carbon-aware workload scheduling thus emerges as a primary strategy for decarbonizing ML [5, 21].

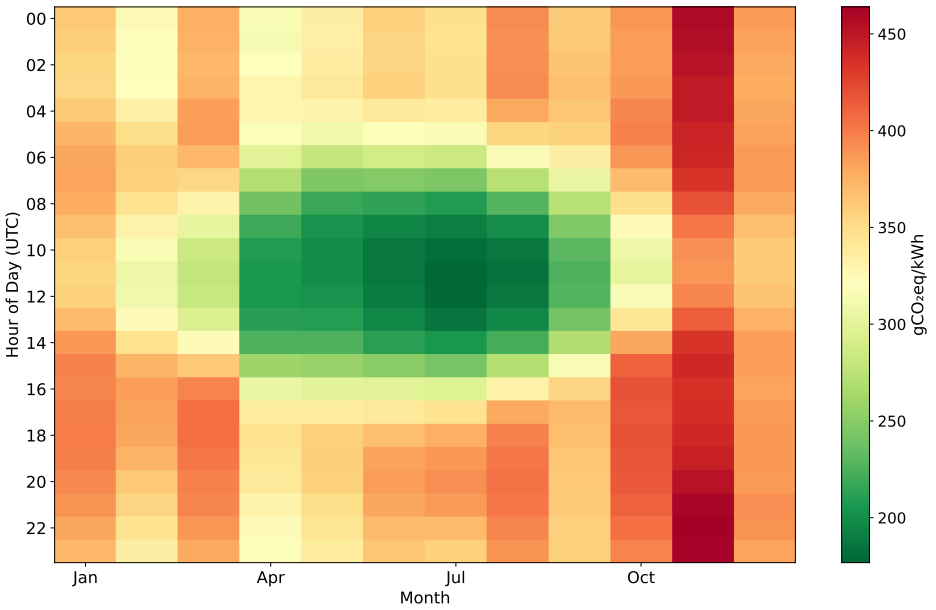


Figure 5.18: Mean carbon intensity ($\frac{gCO_2eq}{kWh}$) by hour of day and calendar month for the German grid (2024), derived from 8,760 hourly observations.

Synthesis Taken together, these findings introduce a dimension of ecological efficiency that the main benchmark cannot capture: timing. The static CodeCarbon factor is not grossly wrong as an annual average, but it flattens a distribution that is wide enough to change the practical interpretation of the results. It is worth noting that this intensity overestimation of roughly 12 % (381 versus 341 $\frac{gCO_2eq}{kWh}$) acts in the *opposite* direction to the TDP-based underestimation documented in Section 5.2, which inflates the corrected value by a median factor of 1.87. The intensity error is small enough that the energy-measurement error dominates in every case. The net effect of uncorrected CodeCarbon is still substantial undercounting. The emissions reported in this study should therefore be understood as contingent on when the runs were executed, not as intrinsic properties of the models. For small datasets such as Wine and Credit, this contingency is inconsequential in absolute terms, as the runs are too cheap for timing to shift the total by a meaningful amount. For large-scale workloads such as the MLP on HIGGS, grid-aware scheduling would have reduced the training footprint by the equivalent of the entire Credit dataset's combined model training budget. At that scale, scheduling is no longer a marginal consideration but a first-order efficiency lever, comparable in impact to moderate algorithmic improvements and achievable without touching a single line of model code.

Figure 5.19 extends this picture across six complete calendar years (2020–2025), confirming that the diurnal and seasonal structure identified above is not an artifact of a single year but a stable, recurring property of the German grid.

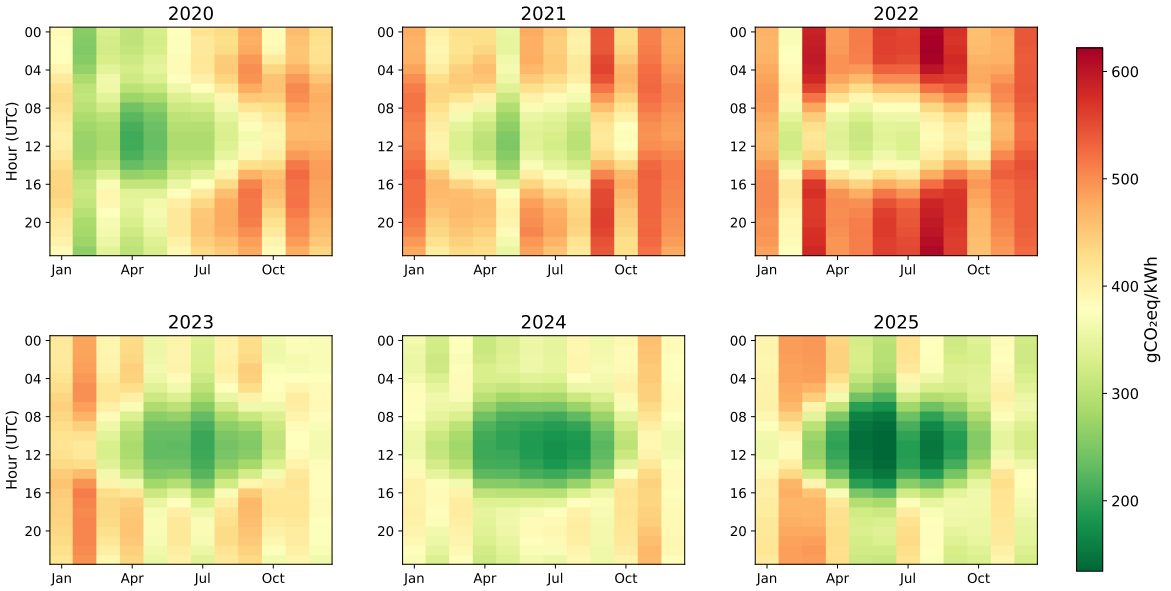


Figure 5.19: Mean carbon intensity ($\frac{gCO_2eq}{kWh}$) by hour of day and calendar month for the German grid, 2020–2025. Each panel is derived from 8,760 hourly observations (8,784 in leap years 2020 and 2024). The shared color scale allows direct cross-year comparison. The summer-midday low-intensity band and winter-evening peak are consistent across all six years.

5.9 Lifecycle Break-Even Analysis

Exp. 5 directly addresses RQ 5: Figure 5.20 shows for each model-dataset combination the number of predictions at which cumulative inference CO_2eq surpasses the one-time training footprint.

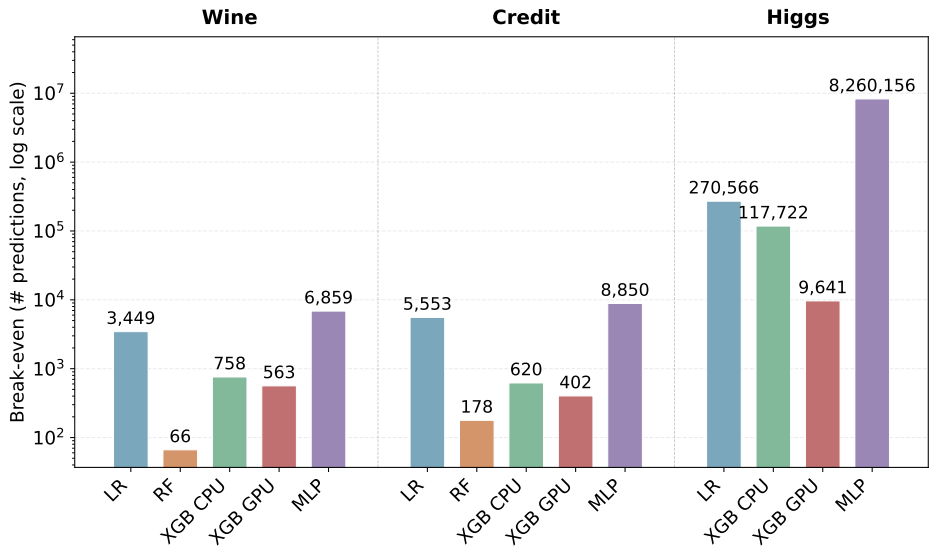


Figure 5.20: Break-even number of predictions at which cumulative inference emissions surpass the one-time training footprint (log scale).

The lowest break-even point is RF on Wine, at 66 predictions. RF inference is slow relative to other classifiers: each prediction requires sequential traversal of the learned tree ensemble, producing a per-prediction cost of approximately 236 µgCO_2eq. Against a

Table 5.6: Lifecycle break-even analysis. TE = hardware-corrected training CO_2eq; IE = per-prediction inference CO_2eq (directly measured CPU energy, same grid intensity). Pred./30 s = throughput in a 30-second window. Break-even = predictions until cumulative IE equals TE. RF not run on HIGGS. Bold = best per column per dataset.

Dataset	Model	WF1 (%)	TE (gCO_2eq)	IE (μgCO_2eq)	Pred./30 s	Break-even
Wine	LR	97.2	**0.0126**	**3.65**	**49,650**	3,449
	RF	97.2	0.0156	235.9	768	66
	XGB (CPU)	97.2	0.0130	17.11	20,113	758
	XGB (GPU)	97.2	0.0170	30.27	11,333	563
	MLP	97.2	0.0387	5.65	33,125	6,858
Credit	LR	77.0	0.0207	**3.73**	**46,870**	5,551
	RF	79.7	0.0446	251.2	742	178
	XGB (CPU)	79.8	**0.0206**	33.20	10,408	620
	XGB (GPU)	79.8	0.0223	55.48	6,123	402
	MLP	**80.2**	0.0722	8.16	24,327	8,851
HIGGS	LR	63.7	0.9610	**3.55**	**48,669**	270,538
	XGB (CPU)	76.0	2.7630	23.47	14,442	117,683
	XGB (GPU)	76.0	**0.3926**	40.71	8,380	9,643
	MLP	**78.5**	58.4610	7.08	29,011	8,261,330

training footprint of only 0.016 gCO_2eq, the inference cost accumulates to parity within a few minutes of active serving. For any realistic RF deployment on tabular data at this scale, inference is the dominant ecological cost by a wide margin.

The opposite extreme is MLP on HIGGS, which does not reach break-even until approximately 8.3 million predictions. MLP inference is fast: the forward pass through a fixed-depth network requires no tree traversal, producing a per-prediction cost of only 7.1 μgCO_2eq. Against a training footprint of 58.5 gCO_2eq, that rate of accumulation reaches parity only after roughly 8 million queries. A system serving this model on a low-traffic application would find that training represents the dominant ecological cost throughout its entire operational lifetime.

Between these extremes, break-even points organize around two drivers: training scale and inference latency. LR on Credit breaks even at 5,551 predictions because both costs are modest in absolute terms. XGB GPU on HIGGS breaks even at 9,643 predictions: its training footprint is contained at 0.39 gCO_2eq, but inference at 40.7 μgCO_2eq per prediction accumulates faster than MLP's 7.1 μgCO_2eq, so the budget is recovered at a far lower deployment volume. The contrast between XGB GPU (9,643) and MLP (8.3 million) on the same dataset illustrates a deeper trade-off in model design. MLP's fast inference pushes the break-even point to a scale that most low-traffic applications will never reach, concentrating the ecological cost at training time. XGB GPU's slower inference distributes the lifecycle cost more evenly across the deployment period.

Cross-Model Lifecycle Comparison on HIGGS The individual break-even points quantify when each model's inference cost overtakes its own training investment. A complementary and more deployment-relevant question is which of two competing models accumulates less total lifecycle emissions at a given query volume. Figure 5.21 addresses this directly for XGB GPU and MLP on HIGGS, the two models that dominate the HIGGS result in opposite directions of the efficiency-performance trade-off.

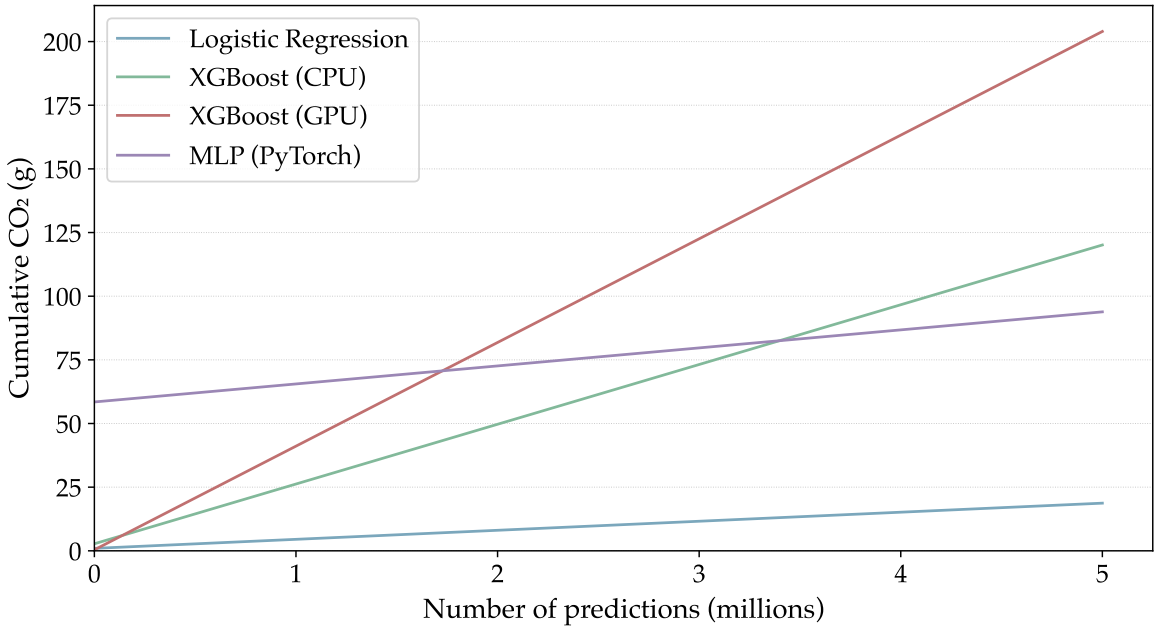


Figure 5.21: Cumulative lifecycle emissions for all four HIGGS models (LR, XGB CPU, XGB GPU, MLP) as a function of prediction volume. Models with cheap training but slow inference (XGB GPU, XGB CPU) accumulate cost faster than models with expensive training but fast inference (LR, MLP).

XGB GPU enters deployment with a training footprint of 0.39 gCO_2eq. MLP carries a training debt of 58.5 gCO_2eq, a factor of 149 larger. The two cumulative cost curves begin from these starting points and converge because inference accumulates at different rates: 40.7 μgCO_2eq per prediction for XGB GPU against 7.1 μgCO_2eq for MLP, a factor of 5.8 in the opposite direction. The cross-over occurs at approximately 1.73 million predictions, the query volume at which MLP's lower per-prediction cost has fully amortized its much larger training investment. Beyond that threshold, MLP is the more ecologically efficient of the two for every additional prediction served.

Whether that threshold is reached depends entirely on deployment context. A batch scoring job processing ten million records in a single run would cross the threshold within that run and favor MLP on lifecycle grounds despite its 149-fold higher training cost. A low-traffic internal tool serving a few thousand queries per month would never approach the threshold, leaving XGB GPU as the lifecycle-optimal choice throughout its operational lifetime. The cross-over point therefore acts as a deployment-scale decision boundary that is orthogonal to predictive performance and invisible in any training-only benchmark table.

The roughly five orders of magnitude separating these extremes reflect a structural divide rather than an accident of scale. Models with cheap training and slow inference front-load their ecological cost at the point of training, then spend that cost again with every subsequent prediction. Models with expensive training and fast inference invert this: the initial investment is large, but it is amortized across many cheap queries. Which profile is ecologically preferable depends entirely on how many predictions the deployed system will serve, a figure that has nothing to do with performance or training efficiency [56].

This creates a direct inversion of the training-only rankings for high-volume deployments [8]. RF on Wine achieves a favorable CF1 precisely because it is cheap to train. Yet at ten

thousand predictions it has already accumulated roughly 95 times its training budget in inference emissions, while MLP at the same query count has barely touched its much larger training investment. For a practitioner deploying at scale, the CF1 ranking is not merely incomplete at this point. It points in the wrong direction. Ecological model selection therefore requires a deployment volume estimate as an explicit input. Below the break-even threshold, optimizing training emissions is the right lever. Above it, inference latency per prediction governs the total lifecycle cost and no amount of training efficiency compensates for a slow prediction function at scale.

6

Conclusion and Outlook

6.1 Summary of Key Findings

This study empirically investigates the trade-off between predictive performance and ecological cost across classical models and deep NNs on tabular data. By synthesizing the results of Exp. 1 through Exp. 5, several core conclusions emerge that challenge conventional ML practices.

Ecological efficiency is context-dependent. Across the three datasets evaluated, no single model consistently led the efficiency rankings. The most ecologically viable choice shifted systematically with task scale and hardware alignment: LR on Wine, XGB CPU on Credit, and GPU-accelerated XGB on HIGGS. This pattern confirms that ecological efficiency is not a fixed property intrinsic to any model, but emerges from the interaction between computational structure, hardware, and data volume. To properly quantify this shifting dynamic, this study introduced and validated the CF1 metric. CF1 proved to be a highly effective absolute heuristic for exposing hidden ecological costs that traditional performance-only benchmark tables entirely obscure. On HIGGS, it penalized the MLP despite its highest raw WF1 in the entire study, and on Credit it surfaced the twofold emissions gap between RF and XGB despite near-identical predictive performance. Through this CF1 lens, it becomes evident that efficiency emerges strictly from the fit between a model's computational profile, the hardware it runs on and the volume of data it processes. This is the empirical answer to RQ 1.

Hardware acceleration requires a minimum dataset scale. The relationship between hardware and efficiency is sharply defined by dataset scale, challenging the assumption that GPUs are universally beneficial. This study identified a hard empirical GPU break-even point at approximately 80,000 rows for XGB on tabular data. Below this threshold, GPU acceleration acts as an ecological liability. On the Wine dataset, XGB GPU emitted 1.3 times as much CO_2 as its CPU counterpart to achieve the exact same WF1, because the fixed costs of device initialization and PCIe memory transfer completely dominated the actual computation. Once dataset scale surpasses the break-even point and saturates the parallel cores, the architecture pays off: at 11 million rows, the GPU variant was seven times more emissions-efficient and nine times faster than the CPU variant. The training-side advantage does not, however, survive into the deployment lifecycle. XGB GPU's per-prediction inference cost of 40.71 μgCO_2eq exceeds that of the CPU variant (23.47 μgCO_2eq) by a factor of 1.7, because every GPU prediction call incurs a fixed device-dispatch overhead that large training batches fully amortize but individual inference calls cannot. The cumulative lifecycle emission curves of the two variants cross at approximately 137,000 predictions, the point at which the training

footprint saved by GPU acceleration has been fully consumed by the higher per-prediction cost. Beyond that volume, the CPU variant is the more ecologically efficient choice for every additional query served. Hardware selection is therefore a two-dimensional decision: dataset scale determines whether GPU acceleration is justified for training, and deployment volume determines whether that advantage is preserved over the full operational lifetime. These findings from Exp. 1, Exp. 2, and Exp. 5 directly answer RQ 3.

Model structure outweighs hyperparameter tuning. Beyond hardware, the structural design of a model dictates efficiency far more rigidly than hyperparameter tuning, particularly for tabular data. On the Credit dataset, RF and XGB CPU reached a statistically indistinguishable WF1 (79.7 % versus 79.8 %), yet RF incurred more than twice the CO_2 cost. No hyperparameter configuration can eliminate the inherent per-tree structural overhead of coarse-grained ensemble construction that causes this gap. At the scale of HIGGS, this overhead compounds to the point at which RF had to be excluded from the largest dataset entirely, making it the only model in this benchmark that could not be evaluated on all three datasets and, by that criterion, the worst-scaling model in the study. When evaluating neural network architectures, the study found that width is the dominant axis for improving the MLP on HIGGS, whereas depth primarily acts as computational overhead. Increasing layer width from 128 to 512 neurons improved the WF1 meaningfully, but adding depth beyond two layers yielded near-zero predictive return while monotonically increasing the CO_2 cost from 48 gCO_2eq to 93 gCO_2eq. These architectural findings from Exp. 3 constitute the empirical answer to RQ 2.

Sublinear predictive returns impose a strict ecological boundary. These structural limitations are further compounded by the sublinear scaling of predictive returns, proving that the dataset scaling curve acts as a strict ecological boundary. As data volume increases, models quickly hit capacity saturation or severe diminishing returns. LR's performance on HIGGS plateaued before 100,000 rows, rendering any compute expended on the remaining 8.7 million rows of the training pool ecologically wasted. Even for complex non-linear models capable of absorbing more data, expanding the HIGGS training data from one million to 8.8 million rows improved the MLP's WF1 by only 2.2 percentage points but exacted a 29-fold increase in CO_2 emissions. This scaling pattern, documented in Exp. 2, is the empirical basis for answering RQ 4.

Hyperparameter tuning is a first-order ecological cost. Focusing solely on the final training run fundamentally misrepresents the true ecological footprint of ML systems. Hyperparameter tuning is a massive hidden first-order ecological cost. A deliberately constrained Optuna search for the MLP on HIGGS emitted 3,180 gCO_2eq and took over 72 hours, making the initialization phase approximately 54 times costlier than the 58.5 gCO_2eq benchmark training run it ultimately enabled. Standard benchmark tables entirely omit this ecological initialization cost that precedes every deployment.

Deployment volume governs the lifecycle footprint. The deployment lifecycle similarly overturns efficiency rankings based on inference latency. Because RF is computationally heavy during inference, it breaks even at just 66 predictions on the Wine dataset, exhausting its modest

training budget within minutes of active serving. In stark contrast, the MLP on HIGGS features extremely fast inference and does not reach its break-even point until approximately 8.3 million predictions. The cross-model comparison between XGB GPU and MLP on HIGGS makes the stakes concrete. Despite XGB GPU entering deployment with a training footprint 149 times smaller (0.393 g versus 58.5 g CO_2), its inference cost of 40.7 µg per prediction is 5.8 times higher than MLP's 7.1 µg. The two models reach equivalent total lifecycle CO_2 at approximately 1.73 million predictions. Beyond that point, MLP is the more ecologically efficient choice for every additional query served. For high-volume deployments, inference latency per prediction entirely governs the lifecycle footprint, meaning that optimizing solely for training efficiency can lead to the wrong ecological choice. These findings from Exp. 5 directly answer RQ 5.

Measurement fidelity and grid timing are primary efficiency levers. Finally, the accuracy of ecological benchmarking depends heavily on measurement methodology and external grid factors. Relying on software estimation tools like CodeCarbon can lead to systematic underestimation, particularly on CPU-intensive workloads. Hardware-corrected measurements in this study exceeded raw CodeCarbon figures across every model-dataset combination, with a median underestimation factor of 1.92. This distortion is worst for short training runs, meaning published studies using uncorrected estimators likely report only a fraction of their true emissions. Even when accurately measured, absolute emissions remain highly volatile due to temporal grid variations. The carbon intensity of the German grid fluctuates predictably, and the counterfactual analysis proves that grid-aware scheduling, for example running workloads during summer midday (193 $\frac{gCO_2eq}{kWh}$) instead of winter evenings (389 $\frac{gCO_2eq}{kWh}$), can reduce emissions by up to 70 % without altering a single line of code. For large-scale workloads, this zero-cost scheduling optimization saves more absolute CO_2 than the combined training footprints of the smaller datasets, establishing timing as a primary lever for Green AI, as quantified in Exp. 4.

6.2 Implications for Green AI

The empirical findings of this study, combined with the broader sustainability literature, provide several actionable implications for practitioners and researchers aiming to align machine learning development with Green AI principles. Ecological efficiency requires moving away from unconditional defaults and adopting a holistic, lifecycle-aware approach to model design.

Data and model selection. A fundamental reorientation is required in how datasets and model architectures are chosen. Dataset size must be treated as an aggressively tunable hyperparameter rather than a fixed asset to be unconditionally maximized. Because predictive returns on additional data generally follow a concave curve while energy costs grow linearly or faster [46], practitioners should identify the elbow point at which WF1 plateaus and halt data ingestion there. Assuming that all available data must be used is an ecological liability rather than a methodological virtue [12]. Model selection must follow the same principle of matching computational profile to task scale. On medium-scale tabular data in this study,

XGB CPU and RF achieved virtually identical WF1 while differing by a factor of two in CO_2, a structural gap that no amount of tuning can close [2]. Unconditionally using a GPU for small or medium tabular workloads compounds this problem further. The break-even analysis in this study demonstrates that below approximately 80,000 rows, device initialization overhead dominates the computation and increases rather than reduces emissions [21]. Even above that threshold, where GPU training is ecologically justified, the advantage does not automatically extend to inference: per-prediction dispatch overhead that large training batches fully amortize cannot be amortized by individual prediction calls, as the lifecycle analysis in Section 5.9 demonstrates for XGB. Training on GPU and serving predictions on CPU is therefore a practical strategy that captures the training-side efficiency gains without incurring the inference-side overhead. For neural network architectures on tabular data specifically, excessive depth acts as a computational penalty rather than a predictive asset, since tabular datasets lack the strict local structure that makes deep hierarchies beneficial in image and text domains [3].

Lifecycle costs and deployment decisions. Evaluating models exclusively on training efficiency obscures the true environmental cost of deployment. As inference can account for 80 to 90 percent of the total machine learning energy demand in production [8], ecological model selection must incorporate an explicit deployment volume estimate from the outset. The break-even prediction counts derived in this study make this concrete: RF exhausts its training carbon budget after as few as 66 predictions on Wine, while the MLP on HIGGS does not reach break-even until approximately 8.3 million predictions. A training-efficient model that is inference-heavy is not a green choice at scale. Post-training optimization techniques such as model distillation, pruning, and quantization can substantially lower per-prediction energy without requiring retraining and should be considered a standard part of the deployment pipeline for NN-based models such as the MLP [55]. For tree-based models these techniques are largely inapplicable, and the primary inference-side efficiency lever there is model selection at training time, as the gap between RF and XGB inference costs in this study illustrates. Published efficiency comparisons must also account for the ecological cost of hyperparameter tuning, which this study found to be approximately 54 times larger than the final training run for the MLP on HIGGS. Any benchmark that omits this overhead misrepresents the initialization cost of the models it compares [1]. A further consideration is the rebound effect: as models become cheaper and more efficient to run, their usage often scales up, driving aggregate energy consumption higher. This dynamic, known as Jevons' Paradox, has been identified as a structural risk in the AI efficiency literature [56]. This study does not directly observe or measure rebound effects. The mechanism is included as a theoretical implication drawn from that literature rather than an empirical result. The lifecycle framing developed here does, however, make the dynamic concrete: if the per-prediction efficiency improvements documented in Section 5.9 were to lower the cost barrier and expand deployment volume proportionally, aggregate emissions could rise even as individual prediction costs fall. Efficiency gains must therefore be paired with deliberate usage constraints to produce net ecological benefits.

Measurement and infrastructure. Accurate ecological accounting requires rigor at both the measurement and infrastructure levels. Software-based estimation tools that rely on TDP values consistently undercount actual energy usage. The median hardware-corrected factor of 1.92 observed in this study implies that benchmark figures produced with uncorrected CodeCarbon on comparable hardware capture roughly 52 % of actual emissions. Direct hardware sensor readings should be the standard for any work making quantitative ecological claims [6]. Once measurement is reliable, infrastructure timing becomes a powerful zero-cost lever. Shifting training jobs temporally to align with low-intensity grid hours, when solar or wind generation peaks, can reduce absolute emissions substantially without altering a single line of code. Beyond coarse rescheduling, carbon-aware training systems can monitor real-time carbon intensity over the course of a run and dynamically throttle or pause GPU compute whenever the grid mix turns carbon-heavy, lowering the training footprint while keeping the runtime penalty small [69]. For large-scale workloads, the counterfactual analysis in this study shows this timing difference can exceed the combined training footprints of smaller datasets, making carbon-aware scheduling one of the most cost-effective actions available to practitioners on renewable-heavy grids [5]. In cloud environments, spatial shifting is often even more powerful. Because model training is rarely latency-bound, workloads can be routed to data centers in regions with predominantly renewable energy supply. The difference between a fossil-fuel-dependent and a hydro-heavy region can reduce emissions by a factor of up to 30 [16]. Operational optimization must nevertheless be paired with an acknowledgment of embodied carbon. The manufacturing of specialized AI hardware, including raw material extraction and the substantial water footprint of semiconductor fabrication, is increasingly recognized as a dominant factor in the overall environmental impact of AI systems [15], [21]. Sustainable AI therefore requires not only efficient use of existing hardware but also deliberate decisions about hardware procurement and lifecycle extension.

Systemic and community-level implications. Long-term sustainability in AI ultimately requires systemic shifts in how the research community structures its incentives and tools. Because many practitioners rely on default parameters in machine learning libraries, shifting those defaults towards more energy-efficient configurations would reduce aggregate emissions across the entire community with minimal effort per user [16]. The community should additionally adopt standardized reporting frameworks and energy leaderboards that track computational and carbon costs alongside traditional performance metrics [55]. Benchmark tables that list only performance figures reward parameter count and training time while rendering ecological cost invisible. By aligning academic and industrial incentives to recognize algorithmic efficiency as a first-class evaluation criterion alongside raw predictive performance, the field can foster a virtuous cycle of innovation that does not treat environmental impact as an afterthought.

6.3 Limitations

Infrastructure. Every absolute emission and energy figure was produced on one specific machine connected to the German electricity grid. The GPU break-even threshold of approximately 80,000 rows, for example, is a direct function of this GPU's initialization overhead and PCIe transfer cost. A datacenter-class GPU with higher parallelism and faster interconnect would place that threshold elsewhere, possibly by a large margin. This constraint is especially relevant for cloud deployments, which dominate production ML infrastructure: cloud GPU instances are provisioned on shared hardware and billed by the second, their initialization overhead and per-compute carbon intensity are structurally different from a locally-attached consumer card, and the operational carbon intensity of the host data center can differ by an order of magnitude from the German grid assumed here [16]. The relative efficiency rankings should therefore be treated as hypotheses for other hardware contexts rather than universally applicable rules. Direct replication on representative cloud instance types would be required before these conclusions can inform cloud procurement decisions.

Operating system. The hardware-corrected CPU measurement relies on LibreHardwareMonitor, which is Windows-only and requires administrator privileges, making the corrected emission values non-reproducible on Linux or macOS without reimplementing the pipeline against Intel RAPL. Linux exposes equivalent processor energy counters through the RAPL interface without such requirements, and reimplementing the same correction against RAPL would remove the platform constraint and enable the methodology to travel to cloud-based research infrastructure where Windows instances are rarely available.

Emission scope coverage. The Greenhouse Gas Protocol distinguishes Scope 1 (direct emissions from a company's own activities), Scope 2 (emissions associated with purchased electricity, steam, heating or cooling) and Scope 3 (all other indirect emissions, spanning the supply chain and downstream product use, including the manufacturing of computing hardware), a framework that has been adapted to both the carbon and the water footprint of AI [5], [70]. Measured against this framework, the present study captures only part of Scope 2 and omits Scope 1 and Scope 3 entirely. RAM energy is not measured directly but estimated from the number of installed DIMM slots at a fixed wattage per slot, a heuristic that approximates rather than reflects actual memory power draw. Data loading, preprocessing and the train-test split are deliberately excluded from the emissions tracker so that all reported figures reflect training cost alone and remain comparable across models. The true end-to-end footprint of a run on HIGGS, where loading eleven million rows and applying a stratified split is itself non-trivial, is higher than the training-only figure suggests.

Most broadly, no embodied carbon is accounted for anywhere in the study. The manufacturing emissions of the CPU, GPU and RAM are excluded entirely. For short training runs on small datasets, embodied carbon can represent the dominant component of the total lifecycle footprint [15, 21], which narrows what conclusions can be drawn from the operational figures alone.

Task. The second scope boundary is task type. All experiments cover tabular classification on three datasets. The conclusions about depth, width, hardware selection and dataset scaling do not extend without re-evaluation to computer vision, NLP or regression, where model architectures map onto hardware in fundamentally different ways. The study's three datasets also represent a deliberately chosen size spectrum, not a broad sample of tabular problems: different feature dimensionalities, class structures or noise profiles could alter the efficiency rankings observed here.

Measurement asymmetries. Three measurement asymmetries affect the direct comparability of emission values across models. The first concerns CPU utilization. LR's SAG solver is single-threaded and runs at roughly 10 % CPU load, while RF and XGB saturate all eight cores via `n_jobs=-1`. The `CPUPowerMonitor` records total CPU Package Power including the processor's static idle draw. At low utilization this idle component dominates the measured wattage, so LR's reported emissions systematically overstate its true algorithmic energy cost relative to the multithreaded models. Subtracting an idle baseline would correct this, but a reliable baseline cannot be captured for runs that complete in under one second, where transient system load spikes can exceed the signal. Cleanly separating idle draw from active workload power is a recognized limitation of energy measurement for ML systems more broadly [71].

The second asymmetry concerns inference. The per-prediction CO_2 estimate is derived from directly measured CPU power, but both XGB GPU and MLP route prediction through the GPU (via `device="cuda"` and CUDA-enabled PyTorch respectively). What the `CPUPowerMonitor` records for these two models is therefore only the CPU-side overhead of host-device data transfer and synchronization, not the GPU's active inference compute energy. The reported per-prediction costs and break-even prediction counts for XGB GPU and MLP are therefore lower bounds on the true figures: both models reach break-even sooner than the table indicates.

The third asymmetry is temporal. CodeCarbon converts measured energy to CO_2 using a fixed annual average of 381 $\frac{gCO_2eq}{kWh}$ for Germany, which cannot reflect the actual carbon intensity at the time each run was executed. The relative rankings within any single dataset are unaffected because the same factor applies uniformly, but absolute CF1 values cannot be directly compared across datasets whose training runs span very different durations and therefore different exposure to grid fluctuation.

Deliberate design trade-offs. LR was not subjected to hyperparameter tuning, because its value in this study lies in representing the zero-tuning-cost end of the spectrum. As a consequence, the WF1 gap between LR and the tuned gradient-boosting models is partly a gap between an untuned linear model and optimized non-linear ones. A regularization search might close some of that gap, particularly on Credit. The reported LR results should therefore be read as the performance of a zero-overhead baseline and not as the ceiling of what a linear classifier can achieve.

The constrained Optuna budget was held uniform across all models to ensure that search effort is symmetric, and no model enters the benchmark with a hidden advantage from more thorough optimization. The cost of this symmetry is that the MLP's conditional search space,

whose effective dimensionality is five to eight depending on sampled depth, is substantially harder to map in limited trials than the low-dimensional spaces of RF or XGB. Reported MLP performance figures are therefore a lower bound on what a more exhaustive search could produce. The MLP implementation also deliberately omits Automatic Mixed Precision (AMP) training to maintain hardware-independent comparability. In an optimized production pipeline AMP would reduce training time and energy substantially, so the reported MLP emissions represent an upper bound on what a deployment-grade implementation would incur.

RF was excluded from HIGGS training runs above 500,000 rows because the per-run cost at full scale was ecologically unjustifiable for the insight it would have added. In the scaling experiment, hyperparameters were carried over from the full 11-million-row tuning run and not re-optimized per subset, which isolates the pure effect of data volume but means that reported WF1 figures at 1,000 or 10,000 instances are conservative lower bounds for models configured for a much larger dataset.

Statistical robustness. Every model-dataset combination was evaluated exactly once on one fixed 80/20 split with a single random seed. No confidence intervals were computed, and no significance tests were applied to any pairwise comparison. Differences such as XGB CPU WF1 of 79.8 % against RF WF1 of 79.7 % on Credit are argued from structural reasoning, not established through hypothesis testing. Repeated runs with different random seeds or a repeated k-fold evaluation would have produced a distribution of results and allowed variance to be distinguished from systematic differences.

This limitation is most consequential for Wine, where approximately 36 test samples yield a point estimate too noisy to distinguish performance differences across models. In practice, all five Wine models returned an identical WF1 of 97.2 %, so the statistical uncertainty means this result can only be read as evidence that any true performance differences fall below the resolution the test set provides. The scaling experiment is better controlled: every model-subset combination was replicated under three independent random seeds and the reported WF1 is the mean across those runs. No variance or standard deviation around those means is reported in the table, so the remaining uncertainty at small subset sizes, where a single atypical subsample draw can shift the mean, is not communicated.

6.4 Future Work

The directions opened by this work fall into four concentric rings: methodological refinement, scope expansion, model extensions, and a systems-level synthesis that would transform ecological benchmarking from a post-hoc activity into a design discipline.

6.4.1 Measurement and Reproducibility

Hardware and grid context. Every absolute emission figure and break-even threshold in this study was produced on one hardware configuration connected to the German electricity grid. Reproducing the benchmark on laptop-class CPUs, datacenter-grade GPUs, and low-power inference chips, and across grids with fundamentally different carbon profiles such

as Nordic hydro-heavy or U.S. Midwest coal-heavy mixes [5], would reveal which findings are hardware-specific and which reflect structural properties of the models that generalize. The GPU break-even threshold of approximately 80,000 rows is particularly likely to shift across GPU generations, because the fixed initialization overhead varies with architecture, and establishing the family of curves to which it belongs would give practitioners a directly applicable decision rule rather than a single data point. Wright et al. [49] formalize this hardware-context sensitivity as a fundamental discrepancy between compute efficiency, energy efficiency, and carbon efficiency, three quantities that diverge most sharply when hardware utilization is misaligned with workload scale. That is precisely the condition this study documents for GPU acceleration below the break-even threshold, and verifying whether the discrepancy persists on datacenter-class GPUs, where initialization overhead and interconnect costs operate at a different scale, would determine how far the single consumer-hardware data point obtained here generalizes.

Platform independence. Reproducibility also requires solving the platform dependency that currently limits the correction pipeline. The hardware-corrected measurements presented here rely on LibreHardwareMonitor, a Windows-only tool requiring administrator privileges. Linux exposes equivalent processor energy counters through the RAPL interface without such requirements, and reimplementing the same correction against RAPL would remove the platform constraint, allow independent cross-validation of the 1.92 underestimation factor on a second hardware context, and enable the methodology to travel to cloud-based research infrastructure where Windows instances are rarely available.

Dynamic carbon intensity. A further refinement concerns the carbon intensity inputs themselves. The counterfactual scheduling analysis in Section 5.8 relies on average hourly grid carbon intensity for Germany. Consequential carbon accounting argues that the relevant quantity is dynamic carbon intensity: the emissions rate of the generation unit that would actually be dispatched in response to an additional unit of demand [5]. Dynamic carbon intensity is more volatile than the average and frequently higher during demand peaks when fossil backup capacity is activated. Rerunning the scheduling analysis with dynamic carbon intensity data would test whether the 70 percent emissions reduction identified under average carbon intensity assumptions survives a more demanding accounting standard, and it would yield a more precise estimate of the true climate impact of carbon-aware scheduling decisions.

6.4.2 Model and Domain Scope

Gradient-boosting variants. Once the measurement foundation is more robust, extending the model landscape is most valuable when each addition is tied to a specific structural hypothesis rather than used purely to increase coverage. The most targeted extension is a direct comparison of XGB against LightGBM [72] and CatBoost [73]: both operate within the gradient-boosting family but use leaf-wise growth and native categorical encoding, structural choices that alter memory access patterns and core utilization in ways that could shift the efficiency profile relative to XGB's level-wise algorithm. This would test whether

the efficiency gap between gradient-boosting and ensemble methods is a property of the family or of XGB's specific implementation.

Attention-based tabular architectures. A second targeted opportunity is offered by attention-based tabular architectures such as TabNet [74] and FT-Transformer [4]: their sequential feature selection and self-attention mechanisms create fundamentally different compute graphs from tree-based and dense-layer models, and their per-prediction latency and scaling behavior under the lifecycle break-even framework could yield results that neither family analyzed here would anticipate.

Beyond tabular data. The most direct scope expansion is an evaluation of the same benchmarking framework on image classification and natural language tasks. The depth-as-overhead finding in this study is structurally conditioned on the absence of local spatial or sequential feature dependencies in tabular data. In convolutional and transformer-based architectures applied to images and text, hierarchical depth is not overhead but the primary mechanism by which useful representations are formed: removing layers degrades performance in a way that has no analog in the tabular MLP experiments reported here [3]. Applying the CF1 framework to a ResNet or BERT variant would therefore test whether depth remains a net ecological liability once it carries genuine predictive weight, or whether the relationship inverts entirely. The GPU break-even analysis is equally likely to shift in these domains. Vision and language models are typically an order of magnitude larger than the tabular classifiers studied here [55], and their training procedures are designed around GPU-native tensor operations. The fixed initialization overhead that constitutes an ecological penalty at small tabular scale may be negligible against the compute budget of a full pretraining run. Replicating the lifecycle and scaling analyses in these settings would reveal which efficiency conclusions reflect structural properties of the models and which are artifacts of the tabular domain.

6.4.3 Model Extensions

Carbon-aware hyperparameter optimization. This study found that a deliberately constrained 40-trial Optuna search for the MLP on HIGGS emitted 3,180 gCO_2eq, approximately 54 times the final training run's footprint, and that number reflects a search space kept narrow by design. Incorporating CO_2eq or total energy as a secondary objective within the HPO loop would allow practitioners to navigate the WF1-emissions trade-off during model selection rather than discovering it after the optimization budget has already been spent. Paired with early stopping on per-trial energy expenditure and a tuning-inclusive version of CF1 as the optimization target, carbon-aware HPO could substantially reduce the ecological initialization cost of large-scale training without sacrificing competitive WF1. Wright et al. [49] identify, however, a behavioral dimension to this problem: practitioners given a more efficient training pipeline frequently respond by running larger hyperparameter searches rather than banking the savings, effectively reinstating the original carbon budget. Measuring whether that behavioral rebound occurs in practice, for instance by tracking whether a twofold reduction in per-trial cost leads to a proportionally larger trial count,

would establish whether carbon-aware HPO produces a genuine net reduction or whether the efficiency gain is absorbed by expanded search.

Data-efficient learning. The complementary problem lies on the data side. This study identifies the scaling elbow at which additional data yields diminishing predictive returns relative to CO_2eq cost but does not examine how to extract more signal from fewer examples once that elbow is known. Active learning, core-set selection, and data distillation methods could potentially shift the WF1 curve above the observed saturation point without expanding the training set, and evaluating them on HIGGS subsets at the saturation points documented in Section 5.6 would quantify whether the elbow is a movable threshold or a model capacity ceiling.

6.4.4 Systems-Level Synthesis

Total carbon ownership framework. The most ambitious direction is to move from isolated efficiency measurement toward what Wright et al. [49] describe as systems thinking in sustainable AI: constructing a total carbon ownership framework that models the causal interactions between all identified efficiency levers simultaneously rather than analyzing them in sequence after the fact. Model selection, hardware configuration, dataset size, tuning budget, temporal grid scheduling, and deployment volume were each analyzed in isolation throughout this work. A unified framework would map these dimensions jointly: given a dataset, a deployment traffic estimate, and a target WF1, it would project a full lifecycle CO_2eq budget across every pipeline phase before the first training run begins. That transition from post-hoc reporting to pre-deployment carbon budgeting would represent a genuine shift in how ecological impact is addressed in machine learning.

Inference optimization. The inference side of the lifecycle remains equally open. Post-training optimization techniques such as quantization, pruning, and knowledge distillation reduce per-prediction compute without requiring a full retraining cycle [55], and measuring the CO_2eq cost of those compression steps alongside the inference speedup they provide would extend the lifecycle model developed in Section 5.9 with a compression phase, potentially shifting the break-even prediction counts significantly for the heavier models.

Embodied carbon. A complete lifecycle framework must also address the embodied carbon dimension that Wright et al. [49] identify as a third structural discrepancy: the gap between operational emissions and the manufacturing footprint of the underlying hardware. As acknowledged in Section 6.3, those costs are entirely absent from the accounting in this study. Amortizing the hardware manufacturing footprint across training runs and inference volume would place procurement decisions on the same carbon ledger as model and hardware choices, and could fundamentally alter break-even thresholds for hardware-intensive models on short-horizon deployments.

Closing Remark

This work set out to ask a deceptively simple question: which model is the most ecologically efficient choice for tabular classification, and under what conditions does that answer change? The answer that emerged is clear enough to be actionable but complex enough to resist any single formula. Ecological efficiency is not an intrinsic property of a model. It emerges from the fit between a model's computational structure, the hardware it runs on, the volume of data it processes, the grid it draws power from, and the scale at which it will eventually serve predictions. No benchmark table that reports performance alone can capture this. The contribution of this study is not a ranked list of preferred classifiers. It is a set of empirical instruments, a corrected measurement methodology, a context-sensitive efficiency metric, and a lifecycle accounting framework, that together make the question answerable in a given deployment context rather than in the abstract. Green AI does not require a fundamentally different kind of engineering. It requires the same rigorous, constraint-aware reasoning that good engineering has always demanded, applied consistently to a dimension that the field has for too long treated as external to the problem.

A

Full HIGGS Scaling Experiment Results

Tables A.1 and A.2 provide the complete numeric results for the HIGGS subset scaling experiment introduced in Section 4.10.2 and discussed in Section 5.6. Figure 5.9 in the main text shows these values as line charts; the tables below give exact figures for reproducibility. RF is excluded from subset sizes above 500,000 due to prohibitive training times. All CO_2 values are hardware-corrected (see Section 4.7).

Table A.1: WF1 (%) per model and HIGGS subset size. Values are means across three random seeds. Dashes indicate runs that were not performed.

Rows	Log. Reg.	RF	XGBoost (CPU)	XGBoost (GPU)	MLP
1,000	62.0	66.9	65.9	66.0	57.8
10,000	63.5	70.6	69.8	69.9	66.1
50,000	63.7	72.0	71.4	71.4	70.8
100,000	63.7	72.4	72.0	72.0	72.6
200,000	63.7	72.8	72.8	72.8	73.9
500,000	63.7	73.2	74.0	73.9	75.4
1,000,000	63.7	—	74.7	74.7	76.3
5,000,000	63.7	—	75.8	75.8	78.1
8,800,000	63.7	—	76.0	76.0	78.6

Table A.2: Emissions (gCO_2eq) and training time (s) per model and HIGGS subset size. Values are means across three random seeds. The MLP at 8.8M rows (58.5 gCO_2eq) is the study's highest single training cost.

	Log. Reg.		**RF**		**XGBoost (CPU)**		**XGBoost (GPU)**		**MLP**	
Rows	Emissions (gCO_2eq)	Time (s)	Emissions (gCO_2eq)	Time (s)	Emissions (gCO_2eq)	Time (s)	Emissions (gCO_2eq)	Time (s)	Emissions (gCO_2eq)	Time (s)
1,000	0.013	0.0	0.015	0.5	0.015	0.2	0.021	0.7	0.021	0.9
10,000	0.012	0.0	0.029	1.2	0.021	0.6	0.030	1.2	0.032	2.1
50,000	0.013	0.2	0.115	6.5	0.034	1.2	0.038	1.6	0.127	8.5
100,000	0.015	0.4	0.244	14.2	0.042	1.8	0.040	1.8	0.210	15.2
200,000	0.020	1.1	0.577	35.0	0.057	2.7	0.047	2.3	0.390	33.6
500,000	0.039	3.3	1.876	118.3	0.110	6.0	0.057	2.6	0.988	93.7
1,000,000	0.084	7.9	—	—	0.217	12.8	0.078	3.6	2.020	194.2
5,000,000	0.406	46.7	—	—	1.307	85.4	0.230	10.1	30.537	3,005
8,800,000	0.734	86.5	—	—	2.319	156.1	0.374	16.7	58.452	6,006

B

Full MLP Architecture Variation Results

Table B.1 gives the complete numeric results for all twelve MLP architecture configurations evaluated in the variation experiment (Sections 4.10.3 and 5.7). The heatmaps in Figures 5.11 and 5.12 visualize the same data. All runs use the full 11M-row HIGGS dataset. CF1 is computed as CF1 $= \frac{C}{\text{WF1}}$, where C is Emissions in mgCO_2eq and WF1 is expressed as a percentage; lower values indicate a more ecologically efficient configuration.

Table B.1: Complete results for the 12 MLP architecture configurations on HIGGS (11M rows). Configurations are labelled *depth*×*width*. Training time is single-run wall-clock time. CF1 is mg CO_2 per WF1 percentage point; lower is better. Bold marks the best value per column.

Config	WF1 (%)	Emissions (mgCO_2eq)	Time (h)	CF1 (mgCO_2eq/WF1)
1×128	72.8	25.6	0.75	352
2×128	75.6	**13.5**	**0.39**	**179**
3×128	75.7	18.1	0.54	239
4×128	75.6	16.1	0.48	213
1×256	73.6	30.3	0.95	412
2×256	76.8	30.5	0.93	397
3×256	77.1	42.6	1.26	553
4×256	77.2	43.5	1.28	564
1×512	74.4	48.1	1.33	647
2×512	77.6	54.6	1.39	704
3×512	78.1	80.5	2.16	1,031
4×512	**78.3**	92.7	2.17	1,184

C

Experimental Setup Reference

C.1 Dataset Characteristics

Table C.1 summarizes the key properties of the three datasets. The feature count excludes the target column; for Credit the identifier column (`ID`) is additionally dropped before training. Class distribution is measured on the full dataset before any train/test split.

Table C.1: Key properties of the three benchmark datasets.

Dataset	Samples	Features	Classes	Class distribution
Wine	178	13	3	33.1 % / 39.9 % / 27.0 %
Credit	30,000	23	2	77.9 % (no default) / 22.1 % (default)
HIGGS	11,000,000	28	2	47.0 % (background) / 53.0 % (signal)

C.2 Python Software Environment

Table C.2 lists the versions of all core libraries used in this study. A complete `requirements.txt` is included in the accompanying code repository. All experiments were run on Python 3.13.0 on Windows 11.

Table C.2: Core Python library versions used in all experiments.

Library	Version
Python	3.13.0
scikit-learn	1.8.0
XGBoost	3.2.0
PyTorch	2.7.0+cu128
Skorch	1.3.1
Optuna	4.8.0
CodeCarbon	3.2.3
HardwareMonitor	1.1.0
pandas	3.0.2
NumPy	2.4.4
matplotlib	3.10.8
seaborn	0.13.2
fastparquet	2026.3.0

D

Supplementary Numeric Results

D.1 Full Main Benchmark Results

Table D.1 consolidates all 14 model-dataset combinations from Section 5.4. Per-dataset tables in the main text present the same data separately; this table provides a single-page reference for cross-dataset comparison.

Table D.1: Main benchmark results for all 14 model-dataset combinations. Emissions are hardware-corrected. Bold marks the best value per column within each dataset.

Dataset	Model	WF1 (%)	Emissions (gCO_2eq)	Time (s)	CF1 ($\frac{mgCO_2eq}{WF1}$)
Wine	LR	97.2	**0.013**	**0.01**	**0.130**
	RF	97.2	0.016	0.4	0.161
	XGB (CPU)	97.2	**0.013**	0.11	0.133
	XGB (GPU)	97.2	0.017	0.31	0.175
	MLP	97.2	0.039	2.3	0.399
Credit	LR	77.0	**0.021**	0.8	0.269
	RF	79.7	0.045	1.7	0.560
	XGB (CPU)	79.8	**0.021**	**0.41**	**0.258**
	XGB (GPU)	79.8	0.022	0.66	0.279
	MLP	**80.2**	0.072	5.0	0.899
HIGGS	LR	63.7	0.961	84.7	15.09
	XGB (CPU)	76.0	2.763	155	36.38
	XGB (GPU)	76.0	**0.393**	**16.4**	**5.17**
	MLP	**78.5**	58.461	4,754	744.7

D.2 XGBoost CPU vs. GPU

Table D.2 gives exact values for the XGBoost CPU vs. GPU comparison discussed in Section 5.5; Figure 5.7 visualizes the same data. Both variants implement the same algorithm and produce equivalent predictive performance; any sub-0.001 difference in WF1 across runs is attributable to floating-point precision differences between CPU (float64) and GPU (float32) arithmetic and carries no practical meaning. Bold marks the better value within each dataset.

Table D.2: Efficiency results for XGBoost CPU and GPU variants across all three datasets. Emissions are hardware-corrected; inference latency is mean per-sample wall-clock time. WF1 is included for completeness; differences are sub-0.001 and carry no practical meaning.

Dataset	Variant	WF1 (%)	Emissions (gCO_2eq)	Time (s)	Latency (μs)
Wine	XGBoost (CPU)	97.2	**0.013**	**0.1**	**1464**
	XGBoost (GPU)	97.2	0.017	0.3	2468
Credit	XGBoost (CPU)	79.8	**0.021**	**0.4**	**2785**
	XGBoost (GPU)	79.8	0.022	0.7	4688
HIGGS	XGBoost (CPU)	76.0	2.763	155.2	**2021**
	XGBoost (GPU)	76.0	**0.393**	**16.4**	3491

D.3 Monthly Carbon Intensity Statistics: German Grid 2024

Table D.3 summarizes the carbon intensity of the German electricity grid for each calendar month of 2024, derived from 8,760 hourly observations obtained via the ElectricityMaps API. These values underpin the seasonal counterfactual analysis in Section 5.8. The winter months (October–January) consistently show higher mean intensity than the summer months, driven by reduced solar generation and higher gas and coal dispatch. November records the highest monthly mean; June records the single lowest hourly minimum.

Table D.3: Descriptive statistics of hourly carbon intensity ($\frac{gCO_2eq}{kWh}$) for the German bidding zone, January–December 2024. All values are rounded to the nearest integer.

Month	Mean	Median	Min	Max	SD
January	382	380	159	650	127
February	344	338	167	576	101
March	359	354	143	612	122
April	288	272	129	562	101
May	290	278	134	509	92
June	298	288	119	525	108
July	294	289	127	520	107
August	320	315	115	536	117
September	322	322	127	546	103
October	380	389	147	606	105
November	435	456	143	642	117
December	380	383	152	658	128

E

Full Counterfactual CO_2 Scenario Results

Tables E.1 to E.3 give exact CO_2 values for all models under the eight carbon intensity scenarios introduced in Section 5.8. The *Static (CC)* row is the hardware-corrected energy multiplied by CodeCarbon's built-in static intensity for Germany; the remaining seven rows use scenario intensities derived from the 2024 ElectricityMaps data. All values are in gCO_2eq.

Table E.1: Counterfactual emissions (gCO_2eq) for the Wine dataset under eight carbon intensity scenarios.

Scenario	Log. Reg.	Rand. Forest	XGB (CPU)	XGB (GPU)	MLP
Static (CC)	0.0137	0.0288	0.0205	0.0512	0.0587
Year mean	0.0123	0.0258	0.0183	0.0458	0.0526
Year best	0.0041	0.0087	0.0062	0.0155	0.0177
Year worst	0.0237	0.0498	0.0353	0.0885	0.1014
Winter mean	0.0133	0.0279	0.0198	0.0496	0.0569
Summer mean	0.0109	0.0230	0.0163	0.0408	0.0468
Summer noon	0.0070	0.0146	0.0104	0.0260	0.0298
Winter evening	0.0140	0.0294	0.0209	0.0522	0.0599

Table E.2: Counterfactual emissions (gCO_2eq) for the Credit dataset under eight carbon intensity scenarios.

Scenario	Log. Reg.	Rand. Forest	XGB (CPU)	XGB (GPU)	MLP
Static (CC)	0.061	0.832	0.043	0.052	0.633
Year mean	0.054	0.744	0.039	0.046	0.567
Year best	0.018	0.251	0.013	0.016	0.191
Year worst	0.105	1.436	0.075	0.089	1.093
Winter mean	0.059	0.806	0.042	0.050	0.613
Summer mean	0.048	0.663	0.035	0.041	0.505
Summer noon	0.031	0.422	0.022	0.026	0.321
Winter evening	0.062	0.848	0.044	0.053	0.646

Table E.3: Counterfactual emissions (gCO_2eq) for the HIGGS dataset under eight carbon intensity scenarios. RF was not run on the full 8.8M-row dataset.

Scenario	Log. Reg.	XGB (CPU)	XGB (GPU)	MLP
Static (CC)	5.148	14.650	2.104	145.892
Year mean	4.607	13.110	1.883	130.558
Year best	1.554	4.422	0.635	44.036
Year worst	8.891	25.301	3.634	251.960
Winter mean	4.987	14.192	2.038	141.329
Summer mean	4.103	11.676	1.677	116.269
Summer noon	2.614	7.438	1.068	74.075
Winter evening	5.251	14.941	2.146	148.792

Statutory Declaration

I herewith declare that I have composed the present thesis myself and without use of any other than the cited sources and aids. Sentences or parts of sentences quoted literally are marked as such; other references with regard to the statement and scope are indicated by full details of the publications concerned. The thesis in the same or similar form has not been submitted to any examination body and has not been published. This thesis was not yet, even in part, used in another examination or as a course performance.

Karlsruhe, June 10, 2026

Eron Qorri

Declaration on the Use of AI-Assisted Tools

In the preparation of this thesis, **Claude Code** (Anthropic, 2026) was used to support the development of implementation code, to paraphrase and refine passages of written text and to assist with translation between German and English.

Karlsruhe, June 10, 2026

Eron Qorri

References

[1] E. Strubell, A. Ganesh, and A. McCallum, "Energy and Policy Considerations for Deep Learning in NLP," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, A. Korhonen, D. Traum, and L. Màrquez, Eds., Florence, Italy: Association for Computational Linguistics, Jul. 2019, pp. 3645–3650. DOI: 10.18653/v1/P19-1355 Accessed: May 27, 2026.

[2] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green AI," *Commun. ACM*, vol. 63, no. 12, pp. 54–63, Nov. 2020, ISSN: 0001-0782. DOI: 10.1145/3381831 Accessed: Mar. 19, 2026.

[3] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" *Advances in Neural Information Processing Systems*, vol. 35, pp. 507–520, Dec. 2022. Accessed: Apr. 29, 2026.

[4] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, "Revisiting Deep Learning Models for Tabular Data," in *Advances in Neural Information Processing Systems*, vol. 34, Curran Associates, Inc., 2021, pp. 18 932–18 943. Accessed: Mar. 26, 2026.

[5] J. Dodge et al., "Measuring the Carbon Intensity of AI in Cloud Instances," in *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency*, ser. FAccT '22, New York, NY, USA: Association for Computing Machinery, Jun. 2022, pp. 1877–1894, ISBN: 978-1-4503-9352-2. DOI: 10.1145/3531146.3533234 Accessed: Mar. 11, 2026.

[6] C. Rodriguez, L. Degioanni, L. Kameni, R. Vidal, and G. Neglia, *Evaluating the Energy Consumption of Machine Learning: Systematic Literature Review and Experiments*, Aug. 2024. DOI: 10.48550/arXiv.2408.15128 arXiv: 2408.15128 [cs]. Accessed: Mar. 11, 2026.

[7] R. Fischer, *Ground-Truthing AI Energy Consumption: Validating CodeCarbon Against External Measurements*, Sep. 2025. DOI: 10.48550/arXiv.2509.22092 arXiv: 2509.22092 [cs]. Accessed: May 6, 2026.

[8] R. Desislavov, F. Martínez-Plumed, and J. Hernández-Orallo, "Trends in AI inference energy consumption: Beyond the performance-vs-parameter laws of deep learning," *Sustainable Computing: Informatics and Systems*, vol. 38, p. 100 857, Apr. 2023, ISSN: 22105379. DOI: 10.1016/j.suscom.2023.100857 Accessed: Apr. 10, 2026.

[9] M. F. Stefan Aeberhard, *Wine*, 1992. DOI: 10.24432/C5PC7J Accessed: Mar. 19, 2026.

[10] I-Cheng Yeh, *Default of Credit Card Clients*, 2009. DOI: 10.24432/C55S3H Accessed: Mar. 19, 2026.

[11] D. Whiteson, *HIGGS*, 2014. DOI: 10.24432/C5V312 Accessed: Mar. 19, 2026.

[12] R. Verdecchia, L. Cruz, J. Sallou, M. Lin, J. Wickenden, and E. Hotellier, "Data-Centric Green AI An Exploratory Empirical Study," in *2022 International Conference on ICT for Sustainability (ICT4S)*, Plovdiv, Bulgaria: IEEE, Jun. 2022, pp. 35–45, ISBN: 978-1-6654-8286-8. DOI: 10.1109/ICT4S55073.2022.00015 Accessed: Mar. 26, 2026.

[13] A. van Wynsberghe, "Sustainable AI: AI for sustainability and the sustainability of AI," *AI and Ethics*, vol. 1, no. 3, pp. 213–218, Aug. 2021, ISSN: 2730-5961. DOI: 10.1007/s43681-021-00043-6 Accessed: Jun. 2, 2026.

[14] *Green AI Position Paper*, https://greensoftware.foundation/policy/research/green-ai-position-paper/. Accessed: Mar. 30, 2026.

[15] T. Eilam, P. Bello-Maldonado, B. Bhattacharjee, C. Costa, E. K. Lee, and A. Tantawi, "Towards a Methodology and Framework for AI Sustainability Metrics," in *Proceedings of the 2nd Workshop on Sustainable Computer Systems*, ser. HotCarbon '23, New York, NY, USA: Association for Computing Machinery, Aug. 2023, pp. 1–7, ISBN: 979-8-4007-0242-6. DOI: 10.1145/3604930.3605715 Accessed: Apr. 10, 2026.

[16] P. Henderson, J. Hu, J. Romoff, E. Brunskill, D. Jurafsky, and J. Pineau, "Towards the Systematic Reporting of the Energy and Carbon Footprints of Machine Learning," *Journal of Machine Learning Research*, vol. 21, no. 248, pp. 1–43, 2020, ISSN: 1533-7928. Accessed: May 6, 2026.

[17] A. Lacoste, A. Luccioni, V. Schmidt, and T. Dandres, *Quantifying the Carbon Emissions of Machine Learning*, Nov. 2019. DOI: 10.48550/arXiv.1910.09700 arXiv: 1910.09700 [cs]. Accessed: Mar. 19, 2026.

[18] *Software Carbon Intensity (SCI) Specification*, https://sci.greensoftware.foundation/. Accessed: Jun. 2, 2026.

[19] J. Nickolls and W. J. Dally, "The GPU Computing Era," *IEEE Micro*, vol. 30, no. 2, pp. 56–69, Mar. 2010, ISSN: 1937-4143. DOI: 10.1109/MM.2010.41 Accessed: May 4, 2026.

[20] K. N. Khan, M. Hirki, T. Niemi, J. K. Nurminen, and Z. Ou, "RAPL in Action: Experiences in Using RAPL for Power Measurements," *ACM Transactions on Modeling and Performance Evaluation of Computing Systems*, vol. 3, no. 2, pp. 1–26, Jun. 2018, ISSN: 2376-3639, 2376-3647. DOI: 10.1145/3177754 Accessed: Apr. 15, 2026.

[21] C.-J. Wu et al., "Sustainable AI: Environmental Implications, Challenges and Opportunities," *Proceedings of Machine Learning and Systems*, vol. 4, pp. 795–813, Apr. 2022. Accessed: May 9, 2026.

[22] R. Mitchell and E. Frank, "Accelerating the XGBoost algorithm using GPU computing," *PeerJ Computer Science*, vol. 3, e127, Jul. 2017, ISSN: 2376-5992. DOI: 10.7717/peerj-cs.127 Accessed: Apr. 29, 2026.

[23] N. Bannour, S. Ghannay, A. Névéol, and A.-L. Ligozat, "Evaluating the carbon footprint of NLP methods: A survey and analysis of existing tools," in *Proceedings of the Second Workshop on Simple and Efficient Natural Language Processing*, Virtual: Association for Computational Linguistics, 2021, pp. 11–21. DOI: 10.18653/v1/2021.sustainlp-1.2 Accessed: Mar. 26, 2026.

[24] L. Lannelongue, J. Grealey, and M. Inouye, "Green Algorithms: Quantifying the Carbon Footprint of Computation," *Advanced Science*, vol. 8, no. 12, p. 2 100 707, 2021, ISSN: 2198-3844. DOI: 10.1002/advs.202100707 Accessed: May 4, 2026.

[25] L. F. W. Anthony, B. Kanding, and R. Selvan, *Carbontracker: Tracking and Predicting the Carbon Footprint of Training Deep Learning Models*, Jul. 2020. DOI: 10.48550/arXiv.2007.03051 arXiv: 2007.03051 [cs]. Accessed: May 4, 2026.

[26] B. Courty et al., *Mlco2/codecarbon: V2.4.1*, Zenodo, May 2024. DOI: 10.5281/zenodo.11171501 Accessed: Apr. 15, 2026.

[27] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*. Springer, 2017, ISBN: 978-0-387-84858-7. Accessed: May 4, 2026.

[28] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014, ISSN: 1533-7928. Accessed: Apr. 18, 2026.

[29] J. H. Friedman, "Greedy function approximation: A gradient boosting machine.," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, Oct. 2001, ISSN: 0090-5364, 2168-8966. DOI: 10.1214/aos/1013203451 Accessed: Apr. 19, 2026.

[30] D. R. Cox, "The Regression Analysis of Binary Sequences," *Journal of the Royal Statistical Society. Series B (Methodological)*, vol. 20, no. 2, pp. 215–242, 1958, ISSN: 0035-9246. JSTOR: 2983890. Accessed: Apr. 8, 2026.

[31] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, no. 85, pp. 2825–2830, 2011, ISSN: 1533-7928. Accessed: Mar. 19, 2026.

[32] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001, ISSN: 1573-0565. DOI: 10.1023/A:1010933404324 Accessed: Mar. 19, 2026.

[33] L. Breiman, "Bagging predictors," *Machine Learning*, vol. 24, no. 2, pp. 123–140, Aug. 1996, ISSN: 1573-0565. DOI: 10.1007/BF00058655 Accessed: Apr. 29, 2026.

[34] E. Scornet, G. Biau, and J.-P. Vert, "Consistency of Random Forests," *The Annals of Statistics*, vol. 43, Apr. 2014. DOI: 10.1214/15-AOS1321

[35] G. Biau and E. Scornet, "A random forest guided tour," *TEST*, vol. 25, no. 2, pp. 197–227, Jun. 2016, ISSN: 1863-8260. DOI: 10.1007/s11749-016-0481-7 Accessed: Apr. 29, 2026.

[36] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16, New York, NY, USA: Association for Computing Machinery, Aug. 2016, pp. 785–794, ISBN: 978-1-4503-4232-2. DOI: 10.1145/2939672.2939785 Accessed: Mar. 19, 2026.

[37] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015, ISSN: 1476-4687. DOI: 10.1038/nature14539 Accessed: Jun. 3, 2026.

[38] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Networks*, vol. 2, no. 5, pp. 359–366, Jan. 1989, ISSN: 0893-6080. DOI: 10.1016/0893-6080(89)90020-8 Accessed: Apr. 29, 2026.

[39] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, Oct. 1986, ISSN: 1476-4687. DOI: 10.1038/323533a0 Accessed: Apr. 19, 2026.

[40] D. P. Kingma and L. J. Ba, "Adam: A Method for Stochastic Optimization," 2015. Accessed: Apr. 19, 2026.

[41] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, Mar. 1994, ISSN: 1941-0093. DOI: 10.1109/72.279181 Accessed: Apr. 29, 2026.

[42] G. Hinton, V. Nair, and Y. Rachmad, "Rectified Linear Units Improve Restricted Boltzmann Machines Vinod Nair," vol. 27, pp. 807–814, Jun. 2010.

[43] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *Proceedings of the 32nd International Conference on Machine Learning*, PMLR, Jun. 2015, pp. 448–456. Accessed: Apr. 18, 2026.

[44] S. Santurkar, D. Tsipras, A. Ilyas, and A. Madry, *How Does Batch Normalization Help Optimization?* Apr. 2019. DOI: 10.48550/arXiv.1805.11604 arXiv: 1805.11604 [stat.ML]. Accessed: Jun. 10, 2026.

[45] L. Prechelt, "Early Stopping — But When?" In *Neural Networks: Tricks of the Trade: Second Edition*, G. Montavon, G. B. Orr, and K.-R. Müller, Eds., Berlin, Heidelberg: Springer, 2012, pp. 53–67, ISBN: 978-3-642-35289-8. DOI: 10.1007/978-3-642-35289-8_5 Accessed: Apr. 29, 2026.

[46] P. Koshute, J. Zook, and I. McCulloh, *Recommending Training Set Sizes for Classification*, https://arxiv.org/abs/2102.09382v1, Feb. 2021. Accessed: Mar. 25, 2026.

[47] A. Althnian et al., "Impact of Dataset Size on Classification Performance: An Empirical Evaluation in the Medical Domain," *Applied Sciences*, vol. 11, no. 2, p. 796, Jan. 2021, ISSN: 2076-3417. DOI: 10.3390/app11020796 Accessed: Apr. 16, 2026.

[48] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, Jul. 2009, ISSN: 0306-4573. DOI: 10.1016/j.ipm.2009.03.002 Accessed: May 4, 2026.

[49] D. Wright, C. Igel, G. Samuel, and R. Selvan, "Efficiency Is Not Enough: A Critical Perspective on Environmentally Sustainable AI," *Communications of the ACM*, vol. 68, no. 7, pp. 62–69, Jun. 2025, ISSN: 0001-0782. DOI: 10.1145/3724500 Accessed: May 28, 2026.

[50] L. Yang and A. Shami, "On hyperparameter optimization of machine learning algorithms: Theory and practice," *Neurocomputing*, vol. 415, pp. 295–316, Nov. 2020, ISSN: 0925-2312. DOI: 10.1016/j.neucom.2020.07.061 Accessed: May 4, 2026.

[51] J. Bergstra and Y. Bengio, "Random Search for Hyper-Parameter Optimization," *Journal of Machine Learning Research*, vol. 13, no. 10, pp. 281–305, 2012, ISSN: 1533-7928. Accessed: Apr. 19, 2026.

[52] G. C. Cawley and N. L. C. Talbot, "On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation," *Journal of Machine Learning Research*, vol. 11, no. 70, pp. 2079–2107, 2010, ISSN: 1533-7928. Accessed: May 16, 2026.

[53] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for Hyper-Parameter Optimization," in *Advances in Neural Information Processing Systems*, vol. 24, Curran Associates, Inc., 2011. Accessed: Apr. 19, 2026.

[54] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ser. KDD '19, New York, NY, USA: Association for Computing Machinery, Jul. 2019, pp. 2623–2631, ISBN: 978-1-4503-6201-6. DOI: 10.1145/3292500.3330701 Accessed: Apr. 1, 2026.

[55] D. Patterson et al., *Carbon Emissions and Large Neural Network Training*, Apr. 2021. DOI: 10.48550/arXiv.2104.10350 arXiv: 2104.10350 [cs]. Accessed: May 9, 2026.

[56] S. Luccioni, Y. Jernite, and E. Strubell, "Power Hungry Processing: Watts Driving the Cost of AI Deployment?" In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency*, ser. FAccT '24, New York, NY, USA: Association for Computing Machinery, Jun. 2024, pp. 85–99, ISBN: 979-8-4007-0450-5. DOI: 10.1145/3630106.3658542 Accessed: May 9, 2026.

[57] D. Hernandez and T. B. Brown, *Measuring the Algorithmic Efficiency of Neural Networks*, May 2020. DOI: 10.48550/arXiv.2005.04305 arXiv: 2005.04305 [cs]. Accessed: May 9, 2026.

[58] C. Coleman et al., "Analysis of DAWNBench, a Time-to-Accuracy Machine Learning Performance Benchmark," *SIGOPS Oper. Syst. Rev.*, vol. 53, no. 1, pp. 14–25, Jul. 2019, ISSN: 0163-5980. DOI: 10.1145/3352020.3352024 Accessed: May 9, 2026.

[59] P. Mattson et al., "MLPerf Training Benchmark," *Proceedings of Machine Learning and Systems*, vol. 2, pp. 336–349, Mar. 2020. Accessed: May 9, 2026.

[60] E. Qorri, *Ecological Efficiency of ML Classification*, github.com/EronQorri/bachelor, 2026.

[61] *XGBoost Documentation — xgboost 3.2.0 documentation*, https://xgboost.readthedocs.io/en/stable/index.html. Accessed: Jun. 7, 2026.

[62] A. Paszke et al., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, Dec. 2019. DOI: 10.48550/arXiv.1912.01703 arXiv: 1912.01703 [cs]. Accessed: Mar. 19, 2026.

[63] B. Ghosh, *Skorch: The Bridge Between PyTorch and Scikit-Learn*, Sep. 2024. Accessed: Mar. 26, 2026.

[64] P. Baldi, P. Sadowski, and D. Whiteson, "Searching for Exotic Particles in High-Energy Physics with Deep Learning," *Nature Communications*, vol. 5, no. 1, p. 4308, Jul. 2014, ISSN: 2041-1723. DOI: 10.1038/ncomms5308 arXiv: 1402.4735 [hep-ph]. Accessed: Mar. 19, 2026.

[65] A. Bagnall and G. C. Cawley, *On the Use of Default Parameter Settings in the Empirical Evaluation of Classification Algorithms*, Mar. 2017. DOI: 10.48550/arXiv.1703.06777 arXiv: 1703.06777 [cs]. Accessed: Apr. 10, 2026.

[66] N. Feix, *Snip3rnick/PyHardwareMonitor*, Mar. 2026. Accessed: May 8, 2026.

[67] *Electricity Maps API: Carbon Intensity History*. Accessed: Apr. 20, 2026.

[68] European Environment Agency, *Monitoring of CO2 emissions from passenger cars, 2024 - Provisional data*, 2025. DOI: 10.2909/5814203D-F36D-4FEE-90D4-64BB348D15A6 Accessed: May 18, 2026.

[69] Z. Yang, L. Meng, J.-W. Chung, and M. Chowdhury, *Chasing Low-Carbon Electricity for Practical and Sustainable DNN Training*, Apr. 2023. DOI: 10.48550/arXiv.2303.02508 arXiv: 2303.02508 [cs.LG]. Accessed: Jun. 10, 2026.

[70] P. Li, J. Yang, M. A. Islam, and S. Ren, "Making AI Less 'Thirsty'," *Communications of the ACM*, vol. 68, no. 7, pp. 54–61, Jun. 2025, ISSN: 0001-0782. DOI: 10.1145/3724499 Accessed: Jun. 10, 2026.

[71] A. Tschand et al., "MLPerf Power: Benchmarking the Energy Efficiency of Machine Learning Systems from μWatts to MWatts for Sustainable AI," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Mar. 2025, pp. 1201–1216. DOI: 10.1109/HPCA61900.2025.00092 Accessed: Jun. 10, 2026.

[72] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Advances in Neural Information Processing Systems*, vol. 30, Curran Associates, Inc., 2017. Accessed: May 27, 2026.

[73] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, ser. NIPS'18, Red Hook, NY, USA: Curran Associates Inc., Dec. 2018, pp. 6639–6649. Accessed: May 25, 2026.

[74] S. Ö. Arik and T. Pfister, "TabNet: Attentive Interpretable Tabular Learning," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 8, pp. 6679–6687, May 2021, ISSN: 2374-3468. DOI: 10.1609/aaai.v35i8.16826 Accessed: May 27, 2026.